

FM-Agent 第二次增量自验证 Bug 清单

生成口径：依据本次 incremental run 写回的 `logic_verification_results/`、`bug_validation/summary.json`、逐条 validator 报告与 extracted-function [SPEC] 汇总。
`confirmed` 仅表示 probe 复现了“实现与生成 SPEC 不一致”，不等于已经确认产品源码存在真实 bug。

总括

本轮是从 `95fd9ed9d4d147aeb6f37cf8aa78d4c908f572f2` 到 `20748637e3dde2ba5b17eeb3812be08f0faf74cc` 的真正增量验证。流水线正常结束并写回 target，耗时 **6247.88 秒**（约 **1 小时 44 分 08 秒**）；运行期间未发现 connection error、timeout、SQLite lock、Python traceback 或自嵌套调用 FM-Agent 的 probe。

项目	本轮结果
基线提交	<code>95fd9ed9d4d147aeb6f37cf8aa78d4c908f572f2</code>
当前提交	<code>20748637e3dde2ba5b17eeb3812be08f0faf74cc</code>
变更源文件	14
新增 / 修改 / 删除函数	16 / 26 / 0
当前 extracted functions	396
与本轮 intent 相关的函数	122
重新生成或更新的 SPEC	91
进入 reasoner 的函数	98
MATCH / MISMATCH / ERROR	25 / 73 / 0
已产出 bug-validator 结论	73 / 73
confirmed / not_confirmed / error	64 / 9 / 0

Reasoner 共产生 **73** 个 bug 候选。Bug validator 已对全部候选形成结果：**64 confirmed**、**9 not_confirmed**、**0 error**，覆盖率为 **100.00%**。原先未落盘的 Go `batch_extract` 已通过定向重跑补齐，并在第一次尝试中得到 `confirmed`。

将生成 SPEC、Pre-condition、reasoner 推导 POST、源码/调用方和 probe 一起人工复核后，73 条 MISMATCH 可按与 `bug_list.md` 一致的口径分层如下：

审计类别	数量	占 MISMATCH	含义
推理误判	30	41.10%	代码读取错误、反例违反真实输入约束、错误 mock/callee 语义；包含 validator 已 <code>not_confirmed</code> 的 9 项
SPEC 错误	21	28.77%	差异可能复现，但 SPEC 自造了不存在的行为、异常或格式约束
契约待确认	15	20.55%	实现与 SPEC 确有差异，但当前仓库证据不足以判定应当采用哪一种行为
实现缺陷候选	7	9.59%	有独立源码不变量、真实调用链或下游可观察影响支持，值得先补回归测试

因此，高置信度误报下界（推理误判 + SPEC 错误）为 **51/73 = 69.86%**，与全量审计的 **69.81%** 几乎一致，说明高误报主要是 SPEC/推理/validator 方法的系统性问题，而不是本次增量提交突然引入了大量缺陷。若把

尚无产品契约支持的 15 条“契约待确认”也按不可直接报 bug 处理，则当前不应直接修源码的比例为 $66/73 = 90.41\%$ ；可直接进入修复/补测候选队列的为 $7/73 = 9.59\%$ 。

Validator 的 64 条 **confirmed** 中，人工复核为 21 条推理误判、21 条 SPEC 错误、15 条契约待确认和 7 条实现缺陷候选。因而即使只看 **confirmed**，高置信度误报仍至少为 $42/64 = 65.63\%$ 。原因是 validator 通常把同一份 LLM SPEC 当作 expected：它能证明“实现与 SPEC 不同”，却没有独立证明 SPEC 就是正确的产品契约。

为便于按编号人工复核，分类索引如下：

- 推理误判 (30)：002、005、006、010、012、013、018、020、024、025、026、027、029、030、032、037、039、040、044、046、047、048、051、052、053、056、058、060、061、064。
- SPEC 错误 (21)：001、003、007、008、014、015、016、021、022、028、033、034、041、049、054、055、057、068、069、070、072。
- 契约待确认 (15)：004、009、011、017、019、023、031、035、038、042、043、050、059、062、063。
- 实现缺陷候选 (7)：036、045、065、066、067、071、073。具体优先级、独立依据和代码位置见文末“人工复核追加：最可能的实际 Bug”。

解读边界

- 本文完整保留 FM-Agent 生成的 SPEC 和验证证据，但不把 SPEC 自动视为产品 ground truth。
- confirmed** 表示 probe 支持该 SPEC/实现差异；仍需用 README、调用方、维护者意图或独立测试审查 SPEC 是否正确。
- not_confirmed** 表示本轮 probe 未能支持原 mismatch，通常应优先视为推理误判或 SPEC 证据不足。
- 本轮只验证 98 个增量相关/受影响函数，因此不能把这份文档当作 396 个函数的全量验证报告。

Validator 结果索引

编号	Bug ID	审计类别	Validator	触发摘要
001	config-py--Settings::settings_customise_sources	SPEC 错误	confirmed	_LayeredSource.call() omits py Field defaults, so fields like injer (default ") are resolved outside sources returned by settings_customise_sources.
002	config-py--_LayeredSource::__call__	推理误判	not_confirmed	_LayeredSource.call returns se directly without filtering model d but self._data is populated only TOML + env vars, so no model ever appear in the output.
003	config-py--_LayeredSource::__init__	SPEC 错误	confirmed	When path does not exist, the c prints the path as given (potenti relative) instead of the absolute required by the specification.
004	main-py--run_pipeline	契约待确认	confirmed	run_pipeline unconditionally cal generate_topdown_layers() whi resume parameter, causing pre completed stages to be re-exec when resume=True and fm_age

编号	Bug ID	审计类别	Validator	触发摘要
005	<code>src--cli_backend-py--build_agent_command</code>	推理 误判	confirmed	When files=['a.txt','b.txt'], AgentCommand.argv lacks file flags, violating the spec that requires each file path be attached as context to the backend invocation.
006	<code>src--cli_backend-py--cli_effort</code>	推理 误判	confirmed	settings.llm.effort is a bytes object (b'hello '); .strip() returns bytes (b'') instead of str, violating the spec's str type guarantee.
007	<code>src--cli_backend-py--resolve_model_backend</code>	SPEC 错误	confirmed	settings.llm.backend set to unrecognizable value 'foobar' — _normalize_backend passes it through unchanged, causing resolve_model_backend to return a non-canonical identifier.
008	<code>src--domain_knowledge-py--collect_domain_knowledge_paths</code>	SPEC 错误	confirmed	Passing a nonexistent file path to cli_paths causes ValueError instead of silently skipping that path per spec.
009	<code>src--domain_knowledge-py--resolve_domain_knowledge_paths</code>	契约 待确认	confirmed	A relative path containing '..' that traverses a symlink: os.path.abspath lexically normalizes it to a non-existent path, causing a spurious ValueError though the original candidate path exists on the filesystem.
010	<code>src--domain_knowledge-py--stage_domain_knowledge_files</code>	推理 误判	not_confirmed	The spec requires '/' as path separator regardless of platform; list_staged_domain_knowledge_files already does replace(os.sep, '/') in line 124, so no bug exists.
011	<code>src--entry_reasoning_pipeline-py--_extracted_file_to_source_relative_filename</code>	契约 待确认	confirmed	hyphen>0 guard incorrectly excludes directory components starting with hyphen (e.g., -cpp) from being recognized as extraction directories, causing fallback to return the component name instead of the correct source filename.
012	<code>src--entry_reasoning_pipeline-py--_fqn_to_ident</code>	推理 误判	confirmed	When the rightmost source-file component is the last FQN component and parts[j+1:] is empty, so "::-".join() returns an empty string, violating the non-empty return value requirement.
013	<code>src--entry_reasoning_pipeline-py--_select_functions_by_source</code>	推理 误判	not_confirmed	Bug claim asserted the function would not return without returning, but source code inspection confirms an explicit return statement in the empty-phase_files guard and an explicit return statement at function end.
014	<code>src--env_check-py--_check_codegraph_version</code>	SPEC 错误	confirmed	When codegraph binary executes successfully but returns empty string, empty string is falsy in 'if not go' guard, causing the code to report 'not installed'.

编号	Bug ID	审计类别	Validator	触发摘要
				of a version-mismatch error as by the specification.
015	<code>src--extract-py--run_extraction</code>	SPEC 错误	confirmed	<code>is_file_ready</code> requires exactly 2 INFO markers in strict order, so only 1 SPEC + 1 INFO is treated as ready and gets overwritten, violating spec that says "contains both [SPEC] marker lines and [INFO] marker lines" should be sufficient.
016	<code>src--file_utils-py--_get_phase_files</code>	SPEC 错误	confirmed	When an extracted-function subdirectory contains nested subdirectories, <code>_get_phase_files</code> yields root-level files before child directory files, producing an overall list that is not globally sorted by filename as the spec requires.
017	<code>src--generate_topdown_layers-py--_collect_phase_files</code>	契约待确认	confirmed	When a source file's basename ends with a dot (e.g., <code>'hidden.'</code>), <code>_collect_phase_files</code> fails with <code>AssertionError</code> because <code>last_dot > 0</code> and the guard <code>last_dot > 0</code> fails. The code looks for directory <code>'hidden.'</code> instead of the spec-correct <code>'-hidden'</code> .
018	<code>src--generate_topdown_layers-py--_strip_comments_from_source</code>	推理误判	not_confirmed	Verification tool claimed function incorrectly masks string literals, not comments. Empirical testing with Python <code>#</code> , C <code>/* */</code> comments shows all comment types are correctly replaced with <code>__</code> .
019	<code>src--git-py--frozen_worktree</code>	契约待确认	confirmed	<code>git add -A</code> skips untracked files not matching <code>.gitignore</code> patterns, so gitignored untracked files are omitted from the snapshot despite the spec requiring untracked files to be captured.
020	<code>src--git-py--frozen_worktree::_git</code>	推理误判	confirmed	<code>_git()</code> hardcodes <code>check=True</code> , <code>capture_output=True</code> , <code>text=True</code> . It passes <code>**kwargs</code> to subprocess, but passing any of these keys in <code>kwargs</code> raises <code>TypeError</code> due to duplicate keyword argument.
021	<code>src--incremental_reasoner-py--_extracted_files_by_method</code>	SPEC 错误	confirmed	Accessing a missing key on the <code>defaultdict(list)</code> and mutating the value permanently adds the key+value to the mapping, violating the spec that mutations to the returned list must not affect the mapping.
022	<code>src--incremental_reasoner-py--_reconcile_extracted_dir</code>	SPEC 错误	confirmed	<code>os.remove()</code> raises <code>PermissionError</code> if file in a non-writable subdirectory. Terminating the function immediately leaves the filesystem partially reconciled — one orphaned file deleted, another present.
023	<code>src--incremental_reasoner-py--_remove_stale_extracted</code>	契约待确认	confirmed	<code>_remove_stale_extracted</code> calls <code>_reconcile_extracted_dir</code> per spec.

编号	Bug ID	审计类别	Validator	触发摘要
		认		which removes stale files but ne removes the func_dir itself or pr empty parent directories above multiple deleted source files sha common parent, empty director
024	src--incremental_reasoner-py-- _update_specs_for_intent	推理 误判	not_confirmed	The seed set is populated from changed_targets (via seed.upda relevant_rel_files; the logic veri missed the seed.update(changed_targets.k call.
025	src--incremental_reasoner-py-- collect_relevant_function_scope	推理 误判	not_confirmed	Bug claim asserts changed_fun criterion is not incorporated; co inspection at lines 1158-1162 sh or any() clause IS present, and confirms it selects modules with files even when LLM assessme empty.
026	src--incremental_reasoner-py-- run_incremental_pipeline	推理 误判	not_confirmed	The code_evidence only cited t directory-removal lines and mis immediately-following artifact-re code (lines 231-243) that delete matching select_relevant_, rele and spec_update_* globs via gl os.remove — the spec is sati...
027	src--languages--c-py--batch_extract	推理 误判	confirmed	batch_extract uses truthiness c cg) instead of explicit None che non-None but falsy CodeGraph incorrectly returns {} instead of functions.
028	src--languages--c-py--function_spans	SPEC 错误	confirmed	For a valid proj_dir and a C file functions, function_spans return instead of an empty list because get_function_spans returns Non with no definitions and the code this through.
029	src--languages--codegraph-py-- CodeGraphExtractor::get_functions_by_file	推理 误判	confirmed	Passing a relative proj_dir to get_functions_by_file produces filesystem path keys in the retur instead of absolute paths as the specification requires.
030	src--languages--codegraph-py-- _bare_function_name	推理 误判	confirmed	When tail starts with 'operator' b contains no consecutive operat symbols (e.g. 'operatorFoo'), th falls through to regex matching returns 'operatorFoo' instead of required 'operator'.
031	src--languages--codegraph-py-- _codegraph_cmd	契约 待确 认	confirmed	When settings.codegraph.bin_c relative path and an executable 'codegraph' file exists at that pa

编号	Bug ID	审计类别	Validator	触发摘要
				<code>_codegraph_cmd()</code> returns the relative path instead of an absolute one.
032	<code>src--languages--codegraph-py--_extraction_ident</code>	推理 误判	confirmed	Whitespace-only scope qualified components (e.g., 'foo:::func') are not removed from empty strings from <code>_bare_function_name</code> , which pass through canonicalize and create scope components with '::-'-separated components in the spec, violating the spec's non-empty component requirement.
033	<code>src--languages--codegraph-py--_node_fqn_map</code>	SPEC 错误	confirmed	0-based counter 'c' used directly as a list index suffix; spec requires 1-indexed counter. Duplicate — second occurrence of <code>_2</code> instead of <code>_2</code> .
034	<code>src--languages--codegraph-py--_qualified_parts</code>	SPEC 错误	confirmed	Function strips whitespace from <code>qualified_name</code> before extracting components, so leading whitespace is lost: <code>qualified_name = 'bar::foo'</code> with <code>name='foo'</code> returns <code>['foo']</code> instead of spec-expected <code>['bar', 'foo']</code> .
035	<code>src--languages--codegraph-py--_warn_on_codegraph_version_mismatch</code>	契约 待确认	confirmed	The code removes leading 'v' on the configured version but not from the command output, causing a false WARNING when both versions are semantically identical (e.g., configured 'v1.0' and output 'v1.0').
036	<code>src--languages--codegraph-py--try_codegraph_init</code>	实现 缺陷 候选	confirmed	When <code>force=True</code> and <code>.codegraph/codegraph.db</code> exists, <code>codegraph_executable</code> is not found in <code>PATH</code> , <code>shutil.rmtree()</code> removes the <code>.codegraph</code> directory before the executable check, violating the requirement of no file modification if the executable is...
037	<code>src--languages--cpp-py--function_spans</code>	推理 误判	confirmed	Passing <code>None</code> as <code>proj_dir</code> causes <code>CodeGraphExtractor.from_project_dir</code> to raise <code>TypeError</code> via <code>os.path.abspath(None)</code> , which propagates instead of returning <code>None</code> as the spec requires.
038	<code>src--languages--erlang-py--ElpClient::_handle_server_message</code>	契约 待确认	confirmed	<code>params.get('status')</code> returns <code>None</code> if the 'status' key is absent, overwriting <code>status</code> instead of leaving it unchanged.
039	<code>src--languages--erlang-py--ElpClient::_next_message</code>	推理 误判	confirmed	<code>_next_message</code> returns any non- <code>BaseException</code> queue item without checking it is a dict, so a non-dict value (like a list) is returned in violation of the spec.

编号	Bug ID	审计类别	Validator	触发摘要
040	src--languages--erlang-py--ElpClient::_send	推理误判	confirmed	stdin is a non-None object but closed/unavailable; the None check passes so RuntimeError is not raised and write/flush throws ValueError
041	src--languages--erlang-py--ElpClient::_wait_for_response	SPEC 错误	confirmed	Response with error=null causes truthiness check to fail, returning instead of raising RuntimeError
042	src--languages--erlang-py--ElpClient::_initialize	契约待确认	confirmed	Deadline computed after request instead of at call entry; request extends effective timeout window spec limit.
043	src--languages--erlang-py--ElpClient::_open_document	契约待确认	confirmed	Path.resolve() follows symlinks, open_document sends the target instead of the path argument Unless path is a symbolic link.
044	src--languages--erlang-py--ElpClient::_request	推理误判	confirmed	Setting _MAX_CONTENT_MODIFIED to 0 causes range(0) to produce iterations, falling through to raise AssertionError("unreachable") which violates the spec's allowed outcomes (return, TimeoutError, RuntimeError)
045	src--languages--erlang-py--_SourceIndex::build	实现缺陷候选	confirmed	For source='hello\\world', splitlines(keepends=True) incorrectly splits on vertical tab (lv), which is a newline character, producing 2 instead of 1.
046	src--languages--erlang-py--_analyze_project_uncached	推理误判	not_confirmed	When no .erl files exist, ErlangAnalysis(functions={}, ed still has spans={} via dataclass field(default_factory=dict) — false positive from logic verifier analyzing extracted function without seeing defaults.
047	src--languages--erlang-py--_elp_argv	推理误判	confirmed	When settings.erlang.command calling .strip() raises AttributeError instead of returning a valid argv required by the specification.
048	src--languages--erlang-py--_source_for_range	推理误判	not_confirmed	LSP range with start character 2 character 1 on same line; Python source[start:end] returns empty string matching spec-implied empty string
049	src--languages--erlang-py--_timeout_seconds	SPEC 错误	confirmed	settings.erlang.timeout_s diverges from _DEFAULT_TIMEOUT_SECONDS (vs 180) with ELP_TIMEOUT_SECONDS unset; function returns settings instead of spec-required default
050	src--languages--go-py--batch_extract	契约待确	confirmed	When the codegraph database file_path entries with parent-dir

编号	Bug ID	审计类别	Validator	触发摘要
		误判		traversal (./), paths outside p can appear in the returned dict.
051	src--languages--go-py--function_spans	推理 误判	confirmed	function_spans hardcodes the l key 'go' in the call to get_function causing it to return None for non that are indexed by the codegra contain function definitions.
052	src--languages--javascript-py--function_spans	推理 误判	confirmed	function_spans returns the raw cg.get_function_spans without start_idx; when the backend ret unordered spans the spec's asc start_idx guarantee is violated.
053	src--languages--rust-py--batch_extract	推理 误判	confirmed	batch_extract passes through e values from get_functions_by_f filtering, violating the spec's req that each value be a non-empty tuples.
054	src--languages--rust-py--function_spans	SPEC 错误	confirmed	function_spans delegates to get_function_spans which retur 'function' and 'method' kinds, bu requires only top-level function declarations; methods inside im leak through unfiltered.
055	src--languages--typescript-py--batch_extract	SPEC 错误	confirmed	batch_extract delegates to get_functions_by_file which retu functions including nested ones spec requires only top-level fun definitions.
056	src--llm_client-py--_inject_targets	推理 误判	confirmed	INJECT_HOST env var is set to separated hosts but settings.inj is empty; function returns [] inst parsed env var values
057	src--llm_client-py--_messages_to_anthropic	SPEC 错误	confirmed	With 3+ system messages, the iterative .strip() on line 92 prem removes trailing whitespace from intermediate messages' content the spec requires concatenating contents first and stripping only result.
058	src--llm_client-py--_metadata_body	推理 误判	confirmed	Mutating settings.inject.id between to _metadata_body() causes the user_id to change, violating the specification's stability guarante
059	src--llm_client-py--_retry_create	契约 待确认	confirmed	Passing input that causes a Typ inside client.chat.completions.c triggers the catch-all except Exc handler, which retries it 5 times transient error instead of propag immediately.

编号	Bug ID	审计类别	Validator	触发摘要
060	src--llm_client-py--_stable_user_id	推理 误判	confirmed	When settings.inject.id is a truthy string (e.g., integer 5), Python's returns the non-string value as-is of a string, violating the spec requirement that the return value always be an empty string.
061	src--opencode_trace-py--_opencode_provider_config	推理 误判	confirmed	When settings.llm is None, the code dereferences None.api_key raising AttributeError instead of returning None as required by the spec.
062	src--opencode_trace-py--_start_opencode_process	契约 待确认	confirmed	When command has no stdin test (command_stdin returns None), subprocess.Popen receives stdin=0 causing the child to inherit the parent's stdin fd instead of disconnecting as spec.
063	src--pipeline_setup-py--_deduplicate_phases	契约 待确认	confirmed	When a module's source_files contains duplicate entries of a file that is removed, removed_files preserves duplicates instead of deduplicating.
064	src--pipeline_setup-py--_phase_plan_complete	推理 误判	confirmed	When phases.json is a FIFO (named file) containing valid schema-compliant JSON, _phase_plan_complete returns True instead of the spec-required False.
065	src--pipeline_setup-py--_phase_plan_schema_errors	实现 缺陷 候选	confirmed	Opening a file containing invalid UTF-8 bytes causes an unhandled UnicodeDecodeError instead of returning an error list.
066	src--pipeline_setup-py--_phases_cover_current_sources	实现 缺陷 候选	confirmed	Absolute file paths in phases.json bypass os.path.exists() check via os.path.abspath, allowing files outside project to be accepted when spec requires relative under proj_dir.
067	src--pipeline_setup-py--_run_generate_phases	实现 缺陷 候选	confirmed	When is_incremental=True and phases.json mtime changes but _phases_cover_current_sources returns False, the OR operator sets phase_plan_ready=True, incorrectly accepting the incomplete phase.
068	src--pipeline_setup-py--_setup_outputs_complete	SPEC 错误	confirmed	When phases.json exists but contains invalid JSON; _phase_plan_complete returns JSON validity+ schema, but spec requires file existence, so _setup_outputs_complete returns False instead of True.
069	src--prompts-py--_generate_block_post_condition	SPEC 错误	confirmed	Call _generate_block_post_condition while _llm_json_call raises a RuntimeError (simulating network error) — the exception propagates unhandled.

编号	Bug ID	审计类别	Validator	触发摘要
				instead of returning None as the spec requires.
070	src--prompts-py--_parse_spec_check_json	SPEC 错误	confirmed	Whitespace-only counterexample/offending_strings bypass the MATCH-verify ValueError check because _nonempty_string uses bool(value) instead of len(value) > 0.
071	src--reasoner-py--_compute_brace_depth_per_line	实现缺陷 候选	confirmed	Multi-line string not tracked: unbalanced double quote on line 0 causes block of subsequent lines to be counted instead of excluded.
072	src--reasoner-py--_split_into_blocks_braced	SPEC 错误	confirmed	C function body with opening brace on second line: entry_depth=1 but block starts at depth 0, violating spec requirement that segments begin at depth.
073	src--verification-py--streaming_reasoner	实现缺陷 候选	confirmed	Early exit when all spec_procs are done does not call is_file_ready() on remaining expected files; ready files are skipped and never submitted for verification.

逐条 MISMATCH 与完整 SPEC

每条均给出本轮生成 SPEC 的全文、reasoner 认定的冲突，以及 bug validator 的实际结论。链接均相对于 [fm_agent/bug_incremental_list.md](#)。

config-py

INCR-MISMATCH-001 — config-py--Settings::settings_customise_sources

- 人工审计：SPEC 错误。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：config-py/Settings::settings_customise_sources.py。
- Reasoner 结果：logic_verification_results/config-py/Settings::settings_customise_sources.json。
- 详细报告：config-py--Settings::settings_customise_sources.md。
- Probe：probe_config-py--Settings::settings_customise_sources.py。

完整生成 SPEC

```
[SPEC]
Unit: config.py

Settings.settings_customise_sources(cls, settings_cls, init_settings, env_settings,
dotenv_settings, file_secret_settings)

Pre-condition:
- settings_cls is a subclass of BaseSettings with typed fields declared via pydantic Field
  annotations
- init_settings is a PydanticBaseSettingsSource carrying keyword-argument values passed to
  Settings()
- _CONFIG_PATH is a pathlib.Path pointing to the fm-agent.toml configuration file

Post-condition:
```

- Returns a 2-tuple of PydanticBaseSettingsSource instances defining the complete field-resolution priority chain
 - The first element (init_settings) has highest priority; any field value provided via keyword arguments to Settings(...) is used as-is and never overridden by any other source
 - The second element is a _LayeredSource that resolves each field in the following order of descending priority:
 1. Process environment variables whose names appear as keys in _ENV_MAP
 2. Values from the TOML file at _CONFIG_PATH
 3. The pydantic Field default declared on the field in settings_cls
- The env_settings, dotenv_settings, and file_secret_settings sources are discarded; they do not participate in field resolution
- Every field of the Settings model is resolved through exactly one of the two returned sources

[SPEC]

Reasoner 差异

- SPEC claim:

- Returns a 2-tuple of PydanticBaseSettingsSource instances defining the complete field-resolution priority chain
 - The first element (init_settings) has highest priority; any field value provided via keyword arguments to Settings(...) is used as-is and never overridden by any other source
 - The second element is a _LayeredSource that resolves each field in the following order of descending priority:
 1. Process environment variables whose names appear as keys in _ENV_MAP
 2. Values from the TOML file at _CONFIG_PATH
 3. The pydantic Field default declared on the field in settings_cls
- The env_settings, dotenv_settings, and file_secret_settings sources are discarded; they do not participate in field resolution
- Every field of the Settings model is resolved through exactly one of the two returned sources

- 推导 actual behavior:

The function returns a tuple `(init\_settings, \_LayeredSource(settings\_cls, \_CONFIG\_PATH))`. The first element is the same `init\_settings` object (a `PydanticBaseSettingsSource` containing constructor keyword arguments). The second element is a `\_LayeredSource` instance which, when invoked, returns a `dict` mapping toplevel section names (type `str`) to subdicts of field names and their resolved values. Within that `\_LayeredSource` result, values are obtained with the following priority (highest first): (1) process environment variable overrides looked up via `\_ENV\_MAP` for the corresponding `(section, field)` pair; (2) values from the TOML file at `\_CONFIG\_PATH`. If `\_CONFIG\_PATH` does not point to an existing file, no TOML values are contributed and only environment overrides appear. Crucially, the returned tuple configures pydantic settings to use the resolution order: constructor keyword arguments (`init\_settings`) > environment overrides (via `\_ENV\_MAP`) > TOML file > pydantic `Field` defaults. Formally, for every field `f` of `settings\_cls`, the final value `v\_f` assigned during settings construction satisfies: `v\_f = init\_settings.get(f) if f in init\_settings else ( LS.get(section, f) if (section, f) in LS.data else default\_field\_value(f) )`, where `LS.data` represents the merged dictionary produced by `\_LayeredSource`, and `default\_field\_value(f)` is the Pythonlevel default declared on the pydantic `Field` annotation.

- Code evidence:

```
Line 11: return (init_settings, _LayeredSource(settings_cls, _CONFIG_PATH))
```

- Trigger condition:

The specification requires the `\_LayeredSource` to resolve each field using env > TOML > Field default, and mandates that every field is resolved through one of the two returned sources. However, the code's `\_LayeredSource` returns only env and TOML values and omits Field defaults, causing fields that rely solely on their default to be resolved outside the two sources, thereby violating the specification.

Bug validator

- Trigger summary: `\_LayeredSource.call()` omits pydantic Field defaults, so fields like inject.id (default "") are resolved outside the two sources returned by settings_customise_sources.

- Probe stdout:

CONFIRMED – inject.id Field default '' NOT present in _LayeredSource.__call__() (sections in source: ['llm', 'runtime', 'scope', 'erlang', 'codegraph']), yet Settings().inject.id resolves to ''. Field default resolved outside the two returned sources.

INCR-MISMATCH-002 — config-py--_LayeredSource::__call__

- 人工审计: 推理误判。
- Validator: **not_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: config-py/_LayeredSource::__call__.py。
- Reasoner 结果: logic_verification_results/config-py/_LayeredSource::__call__.json。
- 详细报告: config-py--_LayeredSource::__call__.md。
- Probe: probe_config-py--_LayeredSource::__call__.py。

完整生成 SPEC

```
[SPEC]
Unit: config.py

_LayeredSource.__call__(self) -> dict

Pre-condition:
- None.

Post-condition:
- Returns a dictionary representing the resolved configuration for a Pydantic
  BaseSettings model. Each top-level key corresponds to a Settings field name;
  the associated value is a nested dictionary of sub-field values for that
  nested model field.
- The returned dictionary contains only explicitly configured values (sourced
  from a TOML file and/or environment variables); no model-level defaults are
  included.
- Multiple calls on the same instance return the identical dictionary object
  with unchanged content.
- The call always succeeds; it never raises an exception.
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns a dictionary representing the resolved configuration for a Pydantic
  BaseSettings model. Each top-level key corresponds to a Settings field name;
  the associated value is a nested dictionary of sub-field values for that
  nested model field.
- The returned dictionary contains only explicitly configured values (sourced
  from a TOML file and/or environment variables); no model-level defaults are
  included.
- Multiple calls on the same instance return the identical dictionary object
  with unchanged content.
- The call always succeeds; it never raises an exception.
```

- 推导 actual behavior:

After execution, the __call__ method returns the value of the instance attribute self._data, which is expected to be a dict. The state of self._data remains unchanged. Formal: \result == self._data

- Code evidence:

```
Line 1:     def __call__(self) -> dict:
Line 2:         return self._data
```

- Trigger condition:

The code returns `self.\_data` directly without filtering out model defaults, allowing nonexplicitly configured values to appear in the output.

Bug validator

- Trigger summary: `_LayeredSource.call` returns `self._data` directly without filtering model defaults, but `self._data` is populated only from TOML + env vars, so no model defaults ever appear in the output.
- Probe stdout:

```
NOT CONFIRMED - no model defaults leaked; only TOML values present
Result: {'nested': {'count': 99}, 'llm': {'api_key': 'sk-88694ac2d6d84bef9b903056966b9d56',
'base_url': 'https://api.deepseek.com', 'backend': 'opencode', 'name': 'deepseek-v4-pro',
'effort': '', 'provider': 'deepseek'}, 'erlang': {'command': 'elp', 'timeout_s': '180'}}
```

INCR-MISMATCH-003 — `config-py--_LayeredSource::__init__`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `config-py/_LayeredSource::__init__.py`。
- Reasoner 结果: `logic_verification_results/config-py/_LayeredSource::__init__.json`。
- 详细报告: `config-py--_LayeredSource::__init__.md`。
- Probe: `probe_config-py--_LayeredSource::__init__.py`。

完整生成 SPEC

```
[SPEC]
Unit: config.py

_LayeredSource.__init__(self, settings_cls, path) -> None

Pre-condition:
- settings_cls is a subclass of BaseSettings with typed fields declared via pydantic Field annotations
- path is a pathlib.Path

Post-condition:
- Invoking the instance as a callable returns a dict whose top-level keys are str section names and whose values are dicts mapping str field names to their values
- When path refers to an existing regular file containing valid TOML, every key-value pair parsed from the file is present in the returned dict
- When path does not refer to an existing regular file, no file-sourced entries appear in the returned dict and a diagnostic message identifying the file name and absolute path is written to stderr
- For each supported process environment variable that is set, the returned dict contains the corresponding (section, field) entry; an environment-sourced entry replaces any file-sourced entry for the same field
- A field for which no entry exists in the returned dict - neither from the file nor from any matching environment variable - is absent from the returned dict, and pydantic resolves it to the Field default declared on the settings model
[SPEC]
```

Reasoner 差异

- SPEC claim:

- Invoking the instance as a callable returns a dict whose top-level keys are str section names and whose values are dicts mapping str field names to their values
- When path refers to an existing regular file containing valid TOML, every key-value pair parsed from the file is present in the returned dict
- When path does not refer to an existing regular file, no file-sourced entries appear in the returned dict and a diagnostic message identifying the file name and absolute path is written to stderr
- For each supported process environment variable that is set, the returned dict contains the corresponding (section, field) entry; an environment-sourced entry replaces any file-sourced entry for the same field
- A field for which no entry exists in the returned dict neither from the file nor from any matching environment variable is absent from the returned dict, and pydantic resolves it to the Field default declared on the settings model

• 推导 actual behavior:

On normal termination (no exception raised):

- The parent class constructor `super().__init__(settings_cls)` has completed successfully, storing the supplied settings class for later use by the settings source.
- `self._data` is a dictionary assembled according to the following rules:
 1. Initially an empty dictionary.
 2. If `path` refers to an existing regular file (`path.is_file()` is true), the dictionary is replaced with the result of parsing the file contents as TOML via `tomllib.loads(path.read_text())`. If `path` is not a file, a warning message ("FM-Agent: {path.name} not found at {path}; using built-in defaults.") is printed to `sys.stderr`, and the dictionary remains empty.
 3. For each entry `(env_name, (section, field))` in the mapping `_ENV_MAP`, if the environment variable `env_name` exists (i.e., `os.environ.get(env_name)` is not `None`), the value is stored in the dictionary under `data[section][field]`. If `data` does not yet have the key `section`, a new empty dictionary is created via `setdefault` before setting the field.
- The method returns `None`.

On abnormal termination (an exception is raised from any of `super().__init__`, `path.is_file()`, `path.read_text()`, or `tomllib.loads`):

- The exception propagates out of the method, and the instance `self` is left in a partially initialized state. The parent constructor may have completed (if the exception occurred after that call), but `self._data` is not assigned (or retains any pre-existing value if `__init__` was called on an already initialized object). No warning message is printed (unless the exception occurred after the `print` call, but in practice an exception during file reading/parsing prevents reaching the `print` branch).

Formal logic (Hoare-style):

Let `_ENV_MAP` be the mapping from environment variable names to (section, field) tuples.

Define `D_init` as the initial data dictionary:

```
D_init = tomllib.loads(path.read_text()) if path.is_file() and no exception occurs,
D_init = {} otherwise (with stderr output if path not a file).
```

Then after normal termination:

```
self._data = D_init { sec : { fld : os.environ[env] | (env, (sec, fld)) ENV_MAP
os.environ.get(env) None }
```

where ```` merges dictionaries with nested updates: for each `(env, (sec, fld))`, if `D_init` does not have key `sec`, it is added as `{}`; then `fld` is set to the environment value. In case of exception `E`, the post-condition is `E` is raised and `self._data` is undefined.

• Code evidence:

```
Line 11:         print(
            f"FM-Agent: {path.name} not found at {path}; using built-in defaults.",
            file=sys.stderr,
        )
```

• Trigger condition:

Specification requires the diagnostic message to include the absolute path, but the code prints the path object as given, which may be relative, failing to identify the absolute path.

Bug validator

- Trigger summary: When path does not exist, the diagnostic prints the path as given (potentially relative) instead of the absolute path as required by the specification.
- Probe stdout:

```
CONFIRMED – diagnostic prints relative path instead of absolute path | stderr: 'FM-Agent:
nonexistent_config_test.toml not found at nonexistent_config_test.toml; using built-in
defaults.\n'
```

main-py

INCR-MISMATCH-004 — main-py--run_pipeline

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `main-py/run_pipeline.py`。
- Reasoner 结果: `logic_verification_results/main-py/run_pipeline.json`。
- 详细报告: `main-py--run_pipeline.md`。
- Probe: `probe_main-py--run_pipeline.py`。

完整生成 SPEC

```
[SPEC]
Unit: main.py

run_pipeline(proj_dir, resume, required_source_files, domain_knowledge_files, submodules,
one_phase, extra_call_edges_path, only_spec) -> None

Pre-condition:
- proj_dir is a non-None string; if it does not reference an existing directory, the function
prints a diagnostic and calls sys.exit(1)
- If proj_dir is a directory but contains no files with an extension recognized by the
pipeline, the function prints a diagnostic and calls sys.exit(1)
- resume is a truthy/falsy value; when truthy and fm_agent/ exists under proj_dir, previously
completed pipeline work is preserved and only remaining work executes
- domain_knowledge_files is None or an iterable of path strings pointing to existing markdown
files
- submodules is None or a non-empty iterable of subdirectory names relative to proj_dir; when
provided, only those subdirectories are processed
- one_phase is a truthy/falsy value
- extra_call_edges_path is None or a path to a JSON file defining supplemental call-graph
edges in the format specified by engine conventions
- only_spec is a truthy/falsy value

Post-condition:
- On success (normal return): the full pipeline has executed across all source files under
proj_dir
- If only_spec is truthy: every function in the extracted call graph has a behavioral spec
([SPEC] block) prepended to its extracted-function file; no verification or bug validation runs
```

Reasoner 差异

- SPEC claim:

```
- On success (normal return): the full pipeline has executed across all source files under
proj_dir
- If only_spec is truthy: every function in the extracted call graph has a behavioral spec
([SPEC] block) prepended to its extracted-function file; no verification or bug validation runs
- If only_spec is falsy: specs are generated, then each specced function has a verification
result in fm_agent/logic_verification_results/, and each MISMATCH has a bug validation report in
fm_agent/bug_validation/
- The fm_agent/ work directory under proj_dir is created and populated; no file outside
fm_agent/ under proj_dir is modified
- If resume is truthy and fm_agent/ exists, previously completed stages are not re-executed;
if resume is falsy or fm_agent/ is absent, all prior fm_agent/ contents are removed before
starting
- User domain knowledge files are staged into
fm_agent/spec_prompts/domain_context/user_knowledge/ before any pipeline stage executes
- If no functions are found for verification (empty file_list), the function returns early
without generating specs
- On unrecoverable stage failure after all configured retries: prints a diagnostic identifying
the failed stage and the trace directory, then calls sys.exit(1)
- Pipeline stages execute sequentially: phases.json generation  domain context generation
```


function extraction spec generation (optionally) verification bug validation

- The function outputs status messages to stdout for each major stage transition
- In only_spec mode, the final summary does not print a confirmed-bug count

- 推导 actual behavior:

If `os.path.isdir(proj_dir)` is False, the program prints an error message containing 'proj_dir does not exist or is not a directory' and terminates via `sys.exit(1)`. Else if `_has_source_code(proj_dir, submodules)` returns False, the program prints an error message stating no source code files were found and terminates via `sys.exit(1)`. Otherwise the function proceeds: `work_dir` is set to `os.path.join(proj_dir, 'fm_agent')`, `input_dir` to `os.path.join(work_dir, 'extracted_functions')`, `output_dir` to `os.path.join(work_dir, 'logic_verification_results')`, and `script_dir` to the directory of the current file. `extra_call_edges` is loaded from `extra_call_edges_path` via `load_call_edges` (returning a dict or None). If `resume` is truthy and `work_dir` exists, `resume` remains truthy and the program prints a resume message; if `resume` is truthy but `work_dir` does not exist, `resume` becomes False. If `resume` is falsy (including after adjustment), `_clean_previous_run(work_dir)` is called, removing the `work_dir` tree if it exists. `os.makedirs(work_dir, exist_ok=True)` then guarantees `work_dir` exists. `stage_domain_knowledge_files` copies any existing markdown files from `domain_knowledge_files` (if provided) into `work_dir/spec_prompts/domain_context/user_knowledge/` and returns a list of project-relative paths; if that list is non-empty, a log message is printed. The pipeline then executes: `_run_generate_phases` (may call `sys.exit(1)` on unrecoverable failure), `_post_process_phases` (modifies `phases.json` if needed), `_run_generate_domain_context` (may call `sys.exit(1)` on unrecoverable failure), `collect_file_names`, and `generate_topdown_layers`. Subsequently, for each phase and layer, `_run_spec_generation_batch` is called to generate [SPEC] and [INFO] blocks in extracted function files, using `is_file_ready` to skip alreadyready files. Verification is performed by `streaming_reasoner` unless `only_spec` is truthy, in which case verification is skipped. If `one_phase` is truthy, only the first phase is processed; otherwise all phases are processed. Any unhandled I/O or other exceptions propagate. Upon normal completion, the `work_dir` contains the complete set of pipeline artifacts: `phases.json`, domain context files, topdown layer files, extracted function files enriched with specifications, and verification results (if not skipped). The program does not return a value.

Formally, let R be the initial state satisfying the precondition. Then the postcondition Q is:

```
Q (isdir(proj_dir) PrintedError('proj_dir does not exist') ExitWith(1))
  (isdir(proj_dir) _has_source_code(proj_dir, submodules) PrintedError('No source code
files found') ExitWith(1))
  (isdir(proj_dir) _has_source_code(proj_dir, submodules)
    (work_dir = proj_dir / 'fm_agent' ... [all assignments] ...
      (resume_effective = (resume isdir(work_dir) ? resume : False))
      (resume_effective clean(work_dir)) mkdir(work_dir)
      domain_knowledge_relpaths = stage(proj_dir, work_dir, domain_knowledge_files)
      (domain_knowledge_relpaths [] PrintedMsg)
      (generate_phases_succeeds ExitWith(1))
      (domain_context_succeeds ExitWith(1))
      (normal_termination (phases_artifacts specs_artifacts (only_spec ? no_verification :
verification_artifacts)))
    )
  )
```

- Code evidence:

Line 27-32: the resume handling only prints messages and may set `resume=False`, but does not skip subsequently called pipeline stages. The code unconditionally proceeds to execute `_run_generate_phases`, `_run_generate_domain_context`, etc., even when `resume` is True and `fm_agent/` exists, causing previously completed stages to be re-executed.

- Trigger condition:

The specification requires that 'If `resume` is truthy and `fm_agent/` exists, previously completed stages are not re-executed.' Condition A describes that the pipeline always runs `_run_generate_phases` and later stages, without any check to skip them when resuming. Thus, for an input where `resume=True` and `fm_agent/` already exists, the code re-executes phase generation (and likely other stages), violating the specification.

Bug validator

- Trigger summary: `run_pipeline` unconditionally calls `generate_topdown_layers()` which has no `resume` parameter, causing previously completed stages to be re-executed when `resume=True` and `fm_agent/` exists.

- Probe stdout:

CONFIRMED – generate_topdown_layers lacks resume param (params=['proj_dir', 'phase_numbers', 'extra_call_edges']), is called without resume guard in run_pipeline (line 113 in source), violating spec: 'previously completed stages are not re-executed'. Contrast: _run_generate_phases HAS resume param (['proj_dir', 'work_dir', 'script_dir', 'is_incremental', 'resume', 'submodules']) and uses _resume_skip internally.

src/cli_backend-py

INCR-MISMATCH-005 — src--cli_backend-py--build_agent_command

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/cli_backend-py/build_agent_command.py`。
- Reasoner 结果: `logic_verification_results/src/cli_backend-py/build_agent_command.json`。
- 详细报告: `src--cli_backend-py--build_agent_command.md`。
- Probe: `probe_src--cli_backend-py--build_agent_command.py`。

完整生成 SPEC

```
[SPEC]
Unit: fm_agent/extracted_functions/src/cli_backend-py/build_agent_command.py

build_agent_command(model, prompt, cwd, files=None, backend=None, effort=None) -> AgentCommand

Pre-condition:
- model is a non-empty string identifying an LLM model
- prompt is a non-empty string
- cwd is a path to an existing directory
- files is either None or a list of file path strings (may be empty)
- backend is either None or a string identifying a desired CLI backend
- effort is either None or a string

Post-condition:
- The effective backend is determined by normalizing the provided backend
  argument (resolving alias names) when non-None, or by reading the
  configured default model backend when backend is None; the sentinel
  value "auto" resolves to a concrete backend
- Raises ValueError with a message identifying the unsupported backend
  when the effective backend is not a supported CLI backend
- Returns an AgentCommand whose argv is a non-empty list of argument
  strings that, when executed via subprocess with cwd resolved to an
  absolute path as the working directory, invokes the effective backend
  to process prompt using model
- When files is a non-empty list, the prompt text and the contents of
  each listed file are combined into the AgentCommand's stdin field;
  each file path in files is attached as context to the backend
  invocation
- When files is None or an empty list, the AgentCommand's stdin field is
  None
- When effort is provided and non-empty, or when effort is None and a
  configured default effort is set and non-empty, the reasoning effort
  level is included in the backend invocation arguments
- The AgentCommand's backend field records the canonical name of the
  effective backend
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- The effective backend is determined by normalizing the provided backend
  argument (resolving alias names) when non-None, or by reading the
  configured default model backend when backend is None; the sentinel
  value "auto" resolves to a concrete backend
```

- Raises ValueError with a message identifying the unsupported backend when the effective backend is not a supported CLI backend
- Returns an AgentCommand whose argv is a non-empty list of argument strings that, when executed via subprocess with cwd resolved to an absolute path as the working directory, invokes the effective backend to process prompt using model
- When files is a non-empty list, the prompt text and the contents of each listed file are combined into the AgentCommand's stdin field; each file path in files is attached as context to the backend invocation
- When files is None or an empty list, the AgentCommand's stdin field is None
- When effort is provided and non-empty, or when effort is None and a configured default effort is set and non-empty, the reasoning effort level is included in the backend invocation arguments
- The AgentCommand's backend field records the canonical name of the effective backend

- 推导 actual behavior:

Let B, M, P, D, F, E denote the formal parameters backend, model, prompt, cwd, files, effort respectively. Define:

1. N = if B is not None then _normalize_backend(B) else resolve_model_backend()
2. R = if N == "auto" then resolve_model_backend() else N

If R {"codex-cli", "claude-cli"}, a ValueError is raised.

Otherwise, let:

```
C = os.path.abspath(D)
S = _compose_stdin(P, F if F is not None else [])
    (by spec, S is None when F is None or empty, else a string.)
M' = M.strip()                                (may be empty even if M was nonempty)
E' = (E if E is not None else cli_effort()).strip()
```

```
if R = "codex-cli":
    argv = ["codex", "exec", "--sandbox", "danger-full-access",
            "--dangerously-bypass-approvals-and-sandbox",
            "--skip-git-repo-check", "-C", C]
    + (["--model", M'] if M' != "" else [])
    + (["-c", f'model_reasoning_effort="{E}'"] if E' != "" else [])
    + ["-"]
    return a value r with r.argv = argv, r.stdin = S, r.backend = "codex-cli"
```

```
if R = "claude-cli":
    argv = ["claude", "-p", "--output-format", "text",
            "--no-session-persistence", "--dangerously-skip-permissions",
            "--permission-mode", "bypassPermissions", "--add-dir", C]
    + (["--model", M'] if M' != "" else [])
    + (["--effort", E'] if E' != "" else [])
    return a value r with r.argv = argv, r.stdin = S, r.backend = "claude-cli"
```

No other side effects occur.

- Code evidence:

Line 12: construction of argv for codex-cli does not attach files as context; Line 28: construction of argv for claude-cli does not attach files as context; the requirement to attach each file path as context to the backend invocation is not implemented anywhere in the function.

- Trigger condition:

The specification requires that when files is a non-empty list, each file path is attached as context to the backend invocation (i.e., appears in argv as arguments). The code only combines file contents into stdin but never adds the file paths to argv. With the given input (files=['a.txt', 'b.txt']), the returned AgentCommand.argv lacks any file-related flags like '--file a.txt', violating the specification.

Bug validator

- Trigger summary: When files=['a.txt', 'b.txt'], AgentCommand.argv lacks file path flags, violating the spec that requires each file path be attached as context to the backend invocation.

- Probe stdout:

CONFIRMED – file paths missing from argv for all backends: codex-cli: file paths MISSING from argv → spec violated → CONFIRMED | claude-cli: file paths MISSING from argv → spec violated → CONFIRMED

INCR-MISMATCH-006 — `src--cli_backend-py--cli_effort`

- 人工审计：推理误判。
- Validator: **confirmed**；尝试次数: 1。
- 原始函数与 SPEC: `src/cli_backend-py/cli_effort.py`。
- Reasoner 结果: `logic_verification_results/src/cli_backend-py/cli_effort.json`。
- 详细报告: `src--cli_backend-py--cli_effort.md`。
- Probe: `probe_src--cli_backend-py--cli_effort.py`。

完整生成 SPEC

```
[SPEC]
Unit: fm_agent/extracted_functions/src/cli_backend-py/cli_effort.py

cli_effort() -> str

Pre-condition:
- `settings.llm.effort` is a string

Post-condition:
- Returns the value of `settings.llm.effort` with leading and trailing
  whitespace removed
- If the value is empty or consists only of whitespace, returns an empty
  string
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns the value of `settings.llm.effort` with leading and trailing
  whitespace removed
- If the value is empty or consists only of whitespace, returns an empty
  string
```

- 推导 actual behavior:

The function returns a string that is the result of stripping leading and trailing whitespace from the original ``settings.llm.effort``. The original ``settings.llm.effort`` is unchanged. Formal: let ``old = settings.llm.effort`` at function entry. Then the return value ``r`` satisfies ``r = old.strip()` `isinstance(r, str)` `settings.llm.effort = old``.

- Code evidence:

```
Line 2: return settings.llm.effort.strip()
```

- Trigger condition:

The code returns the result of `.strip()` on the current value of `settings.llm.effort`. If that value is a bytes object (e.g., `b' hello '`), `.strip()` returns a bytes object (`b'hello'`), which is not a string. The specification requires that the function return a string (e.g., an empty string for whitespace-only input), so returning a bytes value violates the type expectation.

- Trigger summary: `settings.llm.effort` is a bytes object (`b' hello '`); `.strip()` returns bytes (`b'hello'`) instead of str, violating the spec's return-type guarantee.
- Probe stdout:

CONFIRMED – actual: `b'hello'` (type: bytes) | expected str: `'hello'`

INCR-MISMATCH-007 — `src--cli_backend-py--resolve_model_backend`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/cli_backend-py/resolve_model_backend.py`。
- Reasoner 结果: `logic_verification_results/src/cli_backend-py/resolve_model_backend.json`。
- 详细报告: `src--cli_backend-py--resolve_model_backend.md`。
- Probe: `probe_src--cli_backend-py--resolve_model_backend.py`。

完整生成 SPEC

```
[SPEC]
Unit: src/cli_backend.py

resolve_model_backend() -> str

Pre-condition:
- settings.llm.backend contains a string value (the configured model backend)
- The process environment may or may not contain variables named
  FM_AGENT_HOST or FM_AGENT_CLIENT, whose values are any strings
- The process environment may or may not contain any of the markers
  CLAUDE_PLUGIN_ROOT, CLAUDE_CODE_ENTRYPOINT, CODEX_HOME,
  CODEX_SANDBOX, or CODEX_EXECUTION_MODE

Post-condition:
- Returns a canonical backend identifier string: one of "opencode",
  "codex-cli", or "claude-cli"
- The returned backend is first determined by normalizing
  settings.llm.backend via _normalize_backend; if the result is not
  "auto", that result is returned immediately
- When the normalized value of settings.llm.backend is "auto", the
  backend is determined by inspecting environment markers in a fixed
  priority order:
  1. FM_AGENT_HOST or FM_AGENT_CLIENT (whichever is set) is checked
     case-insensitively for "claude" or "codex" substrings
  2. The presence of any Claude-specific environment variable
     (CLAUDE_PLUGIN_ROOT, CLAUDE_CODE_ENTRYPOINT)
  3. The presence of any Codex-specific environment variable
     (CODEX_HOME, CODEX_SANDBOX, CODEX_EXECUTION_MODE)
- The first matching marker in this priority order determines the
  returned backend: "claude-cli" for Claude markers, "codex-cli" for
  Codex markers
- When no marker matches, returns "codex-cli" (the default fallback)
- The same input (settings.llm.backend value and environment state)
  always produces the same output (pure function with respect to its
  inputs at call time)

[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns a canonical backend identifier string: one of "opencode",
  "codex-cli", or "claude-cli"
- The returned backend is first determined by normalizing
  settings.llm.backend via _normalize_backend; if the result is not
  "auto", that result is returned immediately
- When the normalized value of settings.llm.backend is "auto", the
  backend is determined by inspecting environment markers in a fixed
```

priority order:

1. FM_AGENT_HOST or FM_AGENT_CLIENT (whichever is set) is checked case-insensitively for "claude" or "codex" substrings
2. The presence of any Claude-specific environment variable (CLAUDE_PLUGIN_ROOT, CLAUDE_CODE_ENTRYPOINT)
3. The presence of any Codex-specific environment variable (CODEX_HOME, CODEX_SANDBOX, CODEX_EXECUTION_MODE)

- The first matching marker in this priority order determines the returned backend: "claude-cli" for Claude markers, "codex-cli" for Codex markers
- When no marker matches, returns "codex-cli" (the default fallback)
- The same input (settings.llm.backend value and environment state) always produces the same output (pure function with respect to its inputs at call time)

- 推导 actual behavior:

The function returns a string that is the canonical backend identifier resolved from the configuration and environment. Let `normalized = _normalize_backend(settings.llm.backend)`. If `normalized != 'auto'`, the result is normalized. Otherwise, when `normalized == 'auto'`, let `H = (os.environ.get('FM_AGENT_HOST') or os.environ.get('FM_AGENT_CLIENT') or '').lower()`. If `'claude' H` or any of the environment variables `CLAUDE_PLUGIN_ROOT` or `CLAUDE_CODE_ENTRYPOINT` is set to a non-empty value, the result is `'claude-cli'`. In all other cases (including when `H` contains `'codex'` but not `'claude'`, when any of `CODEX_HOME`, `CODEX_SANDBOX`, or `CODEX_EXECUTION_MODE` is set, or when no environment hints are present), the result is `'codex-cli'`.

- Code evidence:

```
Line 2:     backend = _normalize_backend(settings.llm.backend)
Line 3:     if backend != "auto":
Line 4:         return backend
```

- Trigger condition:

The specification requires the function to return one of the canonical backend identifiers 'opencode', 'codex-cli', or 'claude-cli'. The code returns the result of `_normalize_backend` unchanged when it is not 'auto'. According to the provided behaviour of `_normalize_backend`, if the input is not a recognised alias, it returns the input as-is. Therefore, for an input like 'foobar', the function returns 'foobar', which is not one of the allowed identifiers, violating the specification.

Bug validator

- Trigger summary: `settings.llm.backend` set to unrecognized value 'foobar' — `_normalize_backend` passes it through unchanged, causing `resolve_model_backend` to return a non-canonical identifier.
- Probe stdout:

CONFIRMED — actual: 'foobar' | expected: one of ['claude-cli', 'codex-cli', 'opencode']

src/domain_knowledge-py

INCR-MISMATCH-008 — src--domain_knowledge-py--collect_domain_knowledge_paths

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/domain_knowledge-py/collect_domain_knowledge_paths.py`。
- Reasoner 结果: `logic_verification_results/src/domain_knowledge-py/collect_domain_knowledge_paths.json`。
- 详细报告: `src--domain_knowledge-py--collect_domain_knowledge_paths.md`。
- Probe: `probe_src--domain_knowledge-py--collect_domain_knowledge_paths.py`。

```
[SPEC]
Unit: src/domain_knowledge.py

collect_domain_knowledge_paths(cli_paths, base_dir, fallback_base_dir) -> list[str]

Pre-condition:
- cli_paths is None or an iterable of path-like strings (potentially nested)
- base_dir is a string referencing an existing directory
- fallback_base_dir is None or a string referencing an existing directory

Post-condition:
- Returns a list of resolved absolute file path strings, one per distinct domain-knowledge
  markdown file
- The returned list includes paths sourced from two origins: the FM_AGENT_DOMAIN_KNOWLEDGE
  environment variable (split on os.pathsep) and the flattened cli_paths
- An empty or unset FM_AGENT_DOMAIN_KNOWLEDGE environment variable contributes no paths
- cli_paths entries that are None or empty contribute no paths
- Each path in the returned list is absolute; if a source path is relative, it is resolved
  against base_dir, and if that resolution does not yield an existing file, it is resolved against
  fallback_base_dir as a secondary base
- The returned list contains no duplicate entries
- The returned list is empty if no valid domain-knowledge paths are found
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns a list of resolved absolute file path strings, one per distinct domain-knowledge
  markdown file
- The returned list includes paths sourced from two origins: the FM_AGENT_DOMAIN_KNOWLEDGE
  environment variable (split on os.pathsep) and the flattened cli_paths
- An empty or unset FM_AGENT_DOMAIN_KNOWLEDGE environment variable contributes no paths
- cli_paths entries that are None or empty contribute no paths
- Each path in the returned list is absolute; if a source path is relative, it is resolved
  against base_dir, and if that resolution does not yield an existing file, it is resolved against
  fallback_base_dir as a secondary base
- The returned list contains no duplicate entries
- The returned list is empty if no valid domain-knowledge paths are found
```

- 推导 actual behavior:

The function returns a list of absolute canonical file paths (strings). It first constructs a list `paths` by concatenating: (a) the result of splitting the environment variable `settings.runtime.domain\_knowledge\_paths` on `os.pathsep` if it is a non-empty string, else an empty list; and (b) the flattened list of path strings obtained from `cli\_paths` after removing nesting, `None`, and empty values, preserving order. Then it calls `resolve\_domain\_knowledge\_paths(paths, base\_dir, fallback\_base\_dir)`, which for each path: expands `~` to the user home directory, resolves relative paths against `base\_dir` (if the path is relative and not resolved via `base\_dir`, fallback to `fallback\_base\_dir`), checks that the resolved path exists as a regular file and has a recognized domain-knowledge markdown extension; if any path fails these checks, a `ValueError` is raised. If no error occurs, the function returns a list of the resolved absolute canonical paths with duplicates removed (keeping the first occurrence according to the constructed order). If the list `paths` is empty, the function returns an empty list. Base_dir and fallback_base_dir (if not None) are pre-existing directories.

Formally: Let $E = (\text{settings.runtime.domain_knowledge_paths is not None and } \text{str}(\text{settings.runtime.domain_knowledge_paths}).\text{strip}() \neq '') ? \text{split}(\text{str}(\text{settings.runtime.domain_knowledge_paths}), \text{os.pathsep}) : []$; let $C = \text{_flatten_paths}(\text{cli_paths})$; let $P = E + C$. Then the return value $R = \text{resolve_domain_knowledge_paths}(P, \text{base_dir}, \text{fallback_base_dir})$. R satisfies: for each p in P , if $\text{resolve}(p)$ fails, a `ValueError` is raised; otherwise, $R = \text{distinct_canonical}([\text{resolve}(p) \text{ for } p \text{ in } P])$ preserving order, where $\text{resolve}(p)$ expands user home and uses `base_dir` or `fallback_base_dir` for relative paths, and requires p to be an existing regular file with a valid markdown extension; if P is empty, $R = []$.

- Code evidence:

```
Line 6-10: return resolve_domain_knowledge_paths(  
    paths,  
    base_dir=base_dir,  
    fallback_base_dir=fallback_base_dir,  
)
```

- Trigger condition:

The code raises a `ValueError` when any source path cannot be resolved to a valid domain-knowledge markdown file, but the specification requires invalid paths to be silently ignored and only valid domain-knowledge paths to appear in the returned list (or an empty list if none are found). This is a concrete mismatch in error-handling behavior.

Bug validator

- Trigger summary: Passing a nonexistent file path in `cli_paths` causes `ValueError` instead of silently skipping that path per spec.
- Probe stdout:

CONFIRMED – `ValueError` raised on invalid path (spec requires silent skip): domain knowledge file does not exist: `/nonexistent_xyz_file_that_does_not_exist.md`

INCR-MISMATCH-009 — `src--domain_knowledge-py--resolve_domain_knowledge_paths`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/domain_knowledge-py/resolve_domain_knowledge_paths.py`。
- Reasoner 结果: `logic_verification_results/src/domain_knowledge-py/resolve_domain_knowledge_paths.json`。
- 详细报告: `src--domain_knowledge-py--resolve_domain_knowledge_paths.md`。
- Probe: `probe_src--domain_knowledge-py--resolve_domain_knowledge_paths.py`。

完整生成 SPEC

```
[SPEC]  
Unit: src/domain_knowledge.py  
  
resolve_domain_knowledge_paths(paths, base_dir, fallback_base_dir=None) -> list[str]  
  
Pre-condition:  
- paths is an iterable of path-like strings, potentially nested (may contain inner iterables of path strings)  
- base_dir is a string referencing an existing directory  
- fallback_base_dir is None or a string referencing an existing directory  
  
Post-condition:  
- Returns a list of absolute file path strings, one per distinct valid domain-knowledge markdown file  
- Each input entry is expanded for user home directories and flattened from any nesting  
- If an expanded entry is absolute, it is used directly; otherwise it is resolved against base_dir  
- When fallback_base_dir is not None and the base_dir-resolved path does not exist on the filesystem, the entry is resolved against fallback_base_dir as a secondary base; if the fallback candidate also does not exist, the base_dir candidate is used (and will be checked later)  
- For each resolved path (the first existing candidate, or the base_dir candidate if none exist):  
    * If the path does not exist on the filesystem, a ValueError is raised with a message indicating the raw input path  
    * If the path exists but is not a regular file, a ValueError is raised with a message indicating the raw input path  
    * If the path exists but its file extension is not in VALID_DOMAIN_KNOWLEDGE_EXTENSIONS, a ValueError is raised with a message listing allowed extensions and the raw input path  
- The returned list contains no duplicate entries; deduplication is performed on the real (canonical) path  
- The returned list preserves the relative order of first occurrence among the input entries  
- If no paths are provided (or all entries are filtered out by deduplication after processing,
```


but note that invalid entries raise errors, so this only applies when input is empty or contains only duplicates of already-seen valid files), returns an empty list [SPEC]

Reasoner 差异

- SPEC claim:

```
- Returns a list of absolute file path strings, one per distinct valid domain-knowledge markdown file
- Each input entry is expanded for user home directories and flattened from any nesting
- If an expanded entry is absolute, it is used directly; otherwise it is resolved against base_dir
- When fallback_base_dir is not None and the base_dir-resolved path does not exist on the filesystem, the entry is resolved against fallback_base_dir as a secondary base; if the fallback candidate also does not exist, the base_dir candidate is used (and will be checked later)
- For each resolved path (the first existing candidate, or the base_dir candidate if none exist):
  * If the path does not exist on the filesystem, a ValueError is raised with a message indicating the raw input path
  * If the path exists but is not a regular file, a ValueError is raised with a message indicating the raw input path
  * If the path exists but its file extension is not in VALID_DOMAIN_KNOWLEDGE_EXTENSIONS, a ValueError is raised with a message listing allowed extensions and the raw input path
- The returned list contains no duplicate entries; deduplication is performed on the real (canonical) path
- The returned list preserves the relative order of first occurrence among the input entries
- If no paths are provided (or all entries are filtered out by deduplication after processing, but note that invalid entries raise errors, so this only applies when input is empty or contains only duplicates of already-seen valid files), returns an empty list
```

- 推导 actual behavior:

The function returns a list `resolved` of strings, where:

- Every element is an absolute path that exists, is a regular file, and has a lowercase extension in `VALID\_DOMAIN\_KNOWLEDGE\_EXTENSIONS`.
- For each raw path string `raw` produced by `\_flatten\_paths(paths)`, the function determines a candidate path via: let `e = os.path.expanduser(raw)`, then if `e` is absolute, `candidates = [e]`; otherwise `candidates = [os.path.join(base\_dir, e)] + ([os.path.join(fallback\_base\_dir, e)] if fallback\_base\_dir is not None else [])`. The candidate actually considered is `c = next((c for c in candidates if os.path.exists(c)), candidates[0])` and the final path is `p = os.path.abspath(c)`. If `p` exists, is a file, has an allowed extension, and `os.path.realpath(p)` has not been previously selected for any other `raw`, then `p` is appended to `resolved`; if any of the existence, file, or extension checks fail, the function raises a `ValueError`.
- The returned list contains exactly those final paths in the order their unique real paths were first encountered. Formally:

```
`resolved = [p_i | i <- [0..len(F)-1], let raw_i = F[i], e_i = os.path.expanduser(raw_i), cand_i = (if os.path.isabs(e_i) then [e_i] else [os.path.join(base_dir, e_i)] ++ (if fallback_base_dir is not None then [os.path.join(fallback_base_dir, e_i)] else [])), c_i = the first existing candidate in cand_i (cand_i[0] if none exist), p_i = os.path.abspath(c_i), where os.path.exists(p_i) os.path.isfile(p_i) os.path.splitext(p_i)[1].lower() VALID_DOMAIN_KNOWLEDGE_EXTENSIONS j < i, os.path.realpath(p_j) os.path.realpath(p_i) ]`.
```

- Code evidence:

```
Line 22: path = os.path.abspath(path)
```

- Trigger condition:

The code calls `os.path.abspath` on the candidate path before existence validation, performing lexical normalization (e.g., collapsing `..`) without considering symlinks. If the original candidate exists but the normalized string does not, the code incorrectly raises a `ValueError` for a non-existent file, whereas the specification requires using the candidate path directly for all checks.

Bug validator

- Trigger summary: A relative path containing '.' that traverses a symlink: `os.path.abspath` lexically normalizes it to a non-existent path, causing a spurious `ValueError` even though the original candidate path exists on the filesystem.
- Probe stdout:

```
CONFIRMED - ValueError raised for existing file: domain knowledge file does not exist:
link/./doc.md
  actual file path: /tmp/fm_probe_0_sv1xj0/doc.md
  actual file exists: True
  abspath'd path: /tmp/fm_probe_0_sv1xj0/base/doc.md
```

INCR-MISMATCH-010 — `src--domain_knowledge-py--stage_domain_knowledge_files`

- 人工审计: **推理误判**。
- Validator: **not_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/domain_knowledge-py/stage_domain_knowledge_files.py`。
- Reasoner 结果: `logic_verification_results/src/domain_knowledge-py/stage_domain_knowledge_files.json`。
- 详细报告: `src--domain_knowledge-py--stage_domain_knowledge_files.md`。
- Probe: `probe_src--domain_knowledge-py--stage_domain_knowledge_files.py`。

完整生成 SPEC

```
[SPEC]
Unit: src/domain_knowledge.py

stage_domain_knowledge_files(proj_dir, work_dir, markdown_paths=None) -> list[str]

Pre-condition:
- proj_dir is a directory path that exists on the filesystem.
- work_dir is the fm_agent/ workspace directory path.
- markdown_paths is None or an iterable of file path strings (relative or absolute).

Post-condition:
- When markdown_paths is falsy (None or empty): the staging directory
  <work_dir>/spec_prompts/domain_context/user_knowledge/ is NOT modified;
  any previously staged files are preserved. This supports resume runs.
- When markdown_paths is truthy and non-empty: the staging directory is
  atomically replaced to contain exactly copies of the resolved markdown
  files plus a manifest file recording which source files were staged.
- The replacement is atomic: a temporary directory is populated, then
  atomically swapped into place; a concurrent reader either sees the
  complete old state or the complete new state.
- Returns a sorted list of project-relative path strings, each prefixed
  with "fm_agent/", for all domain knowledge files that are currently
  staged under the work directory.
- The returned paths use "/" as the path separator regardless of platform.
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- When markdown_paths is falsy (None or empty): the staging directory
  <work_dir>/spec_prompts/domain_context/user_knowledge/ is NOT modified;
  any previously staged files are preserved. This supports resume runs.
- When markdown_paths is truthy and non-empty: the staging directory is
  atomically replaced to contain exactly copies of the resolved markdown
  files plus a manifest file recording which source files were staged.
- The replacement is atomic: a temporary directory is populated, then
  atomically swapped into place; a concurrent reader either sees the
  complete old state or the complete new state.
- Returns a sorted list of project-relative path strings, each prefixed
  with "fm_agent/", for all domain knowledge files that are currently
  staged under the work directory.
- The returned paths use "/" as the path separator regardless of platform.
```

- 推导 actual behavior:

If `markdown_paths` is falsy (None or an empty iterable), the function returns the result of `list_staged_domain_knowledge_relpaths(work_dir)` without modifying the filesystem. The staging directory (`work_dir/USER_KNOWLEDGE_REL_DIR`) and its contents remain exactly as before the call.

If `markdown_paths` is truthy and no exception occurs, the function atomically replaces the staging directory with a new set of domain knowledge files:

- `resolve_domain_knowledge_paths` resolves the provided paths to a list of absolute, valid file paths (raising `ValueError` otherwise).
- Each resolved source file is copied into a temporary directory (`<target_dir>.tmp`) under a unique safe name (via `_safe_staged_name`).
- A JSON manifest file (`USER_KNOWLEDGE_MANIFEST`) is written in the temporary directory containing an ordered list of objects, each with `'source_path'` (the original resolved absolute path) and `'staged_path'` (the path relative to the project root, prefixed with `'fm_agent/'`, using `'/'` separators).
- The temporary directory is then atomically moved to replace the final target directory (`work_dir/USER_KNOWLEDGE_REL_DIR`), deleting the previous staging directory if it existed.
- The function returns a sorted list of relative paths (as returned by `list_staged_domain_knowledge_relpaths(work_dir)`) corresponding to the newly staged files.

If `resolve_domain_knowledge_paths` raises a `ValueError` (e.g., due to a non-existent file, non-regular file, or invalid extension), that exception propagates and no changes are made to the staging directory (though a temporary directory may be left behind as a side effect).

If any other exception (e.g., `OSError` during file operations) occurs before the atomic rename, the staging directory remains unchanged, but a temporary directory may remain on disk. After a successful return, the staging directory contains exactly the files described by the manifest, with no remnants of the temporary directory.

Formally:

```
assert markdown_paths is None or hasattr(markdown_paths, '__iter__')
if not markdown_paths:
    result = list_staged_domain_knowledge_relpaths(work_dir)
    assert result == list_staged_domain_knowledge_relpaths(work_dir) # idempotent
    return result
else:
    resolved = resolve_domain_knowledge_paths(markdown_paths, base_dir=proj_dir,
    fallback_base_dir=os.getcwd())
    # If resolve_domain_knowledge_paths raises ValueError, the function does not return; staging
    directory is unmodified (mod tmp).
    # Otherwise, resolved is a list of absolute canonical paths of existing regular files with
    allowed extensions.

    target_dir = os.path.join(work_dir, USER_KNOWLEDGE_REL_DIR)
    tmp_dir = target_dir + '.tmp'

    # cleanup previous tmp_dir
    shutil.rmtree(tmp_dir, ignore_errors=True)
    os.makedirs(tmp_dir, exist_ok=True)

    used_names = set()
    entries = []
    for source_path in resolved:
        target_name = _safe_staged_name(source_path, used_names)
        target_path = os.path.join(tmp_dir, target_name)
        shutil.copy2(source_path, target_path)
        rel_to_work = os.path.join(USER_KNOWLEDGE_REL_DIR, target_name).replace(os.sep, '/')
        entries.append({
            'source_path': source_path,
            'staged_path': f'fm_agent/{rel_to_work}'
        })

    manifest_path = os.path.join(tmp_dir, USER_KNOWLEDGE_MANIFEST)
    with open(manifest_path, 'w', encoding='utf-8') as f:
        json.dump({'files': entries}, f, indent=2, ensure_ascii=False)

    parent_dir = os.path.dirname(target_dir)
    os.makedirs(parent_dir, exist_ok=True)

    # atomic replace
    shutil.rmtree(target_dir, ignore_errors=True)
    os.replace(tmp_dir, target_dir)

    result = list_staged_domain_knowledge_relpaths(work_dir)
    assert all(p.startswith('fm_agent/') for p in result)
    return result
```

- Code evidence:

```
Line 8: return list_staged_domain_knowledge_relpaths(work_dir); Line 38: return  
list_staged_domain_knowledge_relpaths(work_dir)
```

- Trigger condition:

The specification requires returned paths to use '/' as the path separator regardless of platform, but the code returns the raw output of `list_staged_domain_knowledge_relpaths`, which may use OS-specific separators (e.g., backslashes on Windows). No conversion is performed.

Bug validator

- Trigger summary: The spec requires '/' as path separator regardless of platform; `list_staged_domain_knowledge_relpaths` already does `replace(os.sep, '/')` at line 124, so no bug exists.
- Probe stdout:

```
--- Test 1: Linux native (os.sep='/') ---  
list_staged: ['fm_agent/spec_prompts/domain_context/user_knowledge/test.md']  
stage empty: ['fm_agent/spec_prompts/domain_context/user_knowledge/test.md']  
stage full: ['fm_agent/spec_prompts/domain_context/user_knowledge/input.md']  
PASS: Linux native test – all paths use '/' separators  
  
--- Test 2: Patched os (os.sep='\\') ---  
patched list_staged: ['fm_agent/spec_prompts/domain_context/user_knowledge/test.md']  
PASS: Patched test – replace(os.sep, '/') correctly converted backslashes  
  
NOT CONFIRMED – The code already converts paths to '/' separators  
through list_staged_domain_knowledge_relpaths's replace(os.sep, '/') call at line 124.  
Both stage_domain_knowledge_files return paths (lines 137 and 171) delegate to  
list_staged_domain_knowledge_relpaths, which handles the conversion.
```

src/entry_reasoning_pipeline-py

INCR-MISMATCH-011 — src--entry_reasoning_pipeline-py-- _extracted_file_to_source_rel

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/entry_reasoning_pipeline-py/_extracted_file_to_source_rel.py`。
- Reasoner 结果: `logic_verification_results/src/entry_reasoning_pipeline-py/_extracted_file_to_source_rel.json`。
- 详细报告: `src--entry_reasoning_pipeline-py--_extracted_file_to_source_rel.md`。
- Probe: `probe_src--entry_reasoning_pipeline-py--_extracted_file_to_source_rel.py`。

完整生成 SPEC

```
[SPEC]  
Unit: fm_agent/extracted_functions/src/entry_reasoning_pipeline-  
py/_extracted_file_to_source_rel.py
```

```
_extracted_file_to_source_rel(extracted_rel) -> str
```

Pre-condition:

- `extracted_rel` is a relative path string whose last path component is a function file name, and the path contains at least one directory component that was derived from a source filename by replacing the last dot with a hyphen followed by a known language extension (the "extraction directory component"). The extraction layout is such that the extraction directory component's suffix after its last hyphen matches a key in `EXT_TO_LANG`.

Post-condition:

- Returns the source-file relative path obtained by scanning the path components from right to left (skipping the filename) to find the extraction directory component. Once found, the component's last hyphen and its following extension are replaced by a dot and the extension (e.g., "loader-cpp" -> "loader.cpp"). The resulting filename is prepended with any leading directory prefix (components before the extraction directory component), and the function file component is dropped. If no extraction directory component is found (i.e., no component whose hyphen-suffix is in EXT_TO_LANG), falls back to using the immediate parent directory: its last hyphen is replaced with a dot (if a hyphen exists after the first character), and the result is prefixed with the parent directory of that parent, or used as is if no grandparent exists.
- The returned path uses the OS-native path separator (os.sep).

[SPEC]

Reasoner 差异

- SPEC claim:

- Returns the source-file relative path obtained by scanning the path components from right to left (skipping the filename) to find the extraction directory component. Once found, the component's last hyphen and its following extension are replaced by a dot and the extension (e.g., "loader-cpp" -> "loader.cpp"). The resulting filename is prepended with any leading directory prefix (components before the extraction directory component), and the function file component is dropped. If no extraction directory component is found (i.e., no component whose hyphen-suffix is in EXT_TO_LANG), falls back to using the immediate parent directory: its last hyphen is replaced with a dot (if a hyphen exists after the first character), and the result is prefixed with the parent directory of that parent, or used as is if no grandparent exists.
- The returned path uses the OS-native path separator (os.sep).

- 推导 actual behavior:

The function returns a relative path string `source_rel` that is the source file corresponding to `extracted_rel` under the extraction layout mapping. If the loop in lines 13-19 finds a component `comp = parts[i]` (with `i` ranging from `len(parts)-2` down to 0) that contains a hyphen at index `h>0` and `comp[h+1:]` is a key in `EXT_TO_LANG`, then the returned string is `os.path.join(src_dir, source_base)` (or `source_base` if `src_dir` is empty) where `src_dir = os.sep.join(parts[:i])` and `source_base = comp[:h] + '.' + comp[h+1:]`. In this case, `source_rel` is exactly the result of dropping all components of `extracted_rel` after the extraction directory component and replacing the extraction directory component `<base>-<ext>` with `<base>.<ext>`. If no such component is found, the fallback on lines 21-26 returns `os.path.join(src_dir, source_base)` (or `source_base`) where `func_dir = os.path.dirname(extracted_rel)`, `src_dir = os.path.dirname(func_dir)`, `dir_name = os.path.basename(func_dir)`, `h = dir_name.rfind('-')`, and `source_base = dir_name[:h] + '.' + dir_name[h+1:]` if `h > 0` else `dir_name`. Under the given pre-condition (that `extracted_rel` contains at least one extraction directory component whose suffix after the last hyphen matches a key in `EXT_TO_LANG`), the early return always executes, and the returned path satisfies the following formal property:

Let `P = split(extracted_rel, sep)` where `sep` is the OS path separator.

Then there exists an index `i` with `0 ≤ i ≤ len(P)-2` such that for `c = P[i]`, `h = c.rfind('-')`, we have `h > 0`, `c[h+1:] ∈ EXT_TO_LANG` and for all `j` with `i < j ≤ len(P)-2`, either `P[j]` does not contain `-` or `P[j][rfind('-')+1:] ∈ EXT_TO_LANG`.

The returned value `R` satisfies `R = join(P[:i], c[:h] + '.' + c[h+1:])` where `join` is `os.sep.join` except that if `P[:i]` is empty, the result is just `c[:h] + '.' + c[h+1:]`.

- Code evidence:

Line 16: if hyphen > 0 and comp[hyphen + 1:] in EXT_TO_LANG:

- Trigger condition:

The specification requires any component whose suffix after the last hyphen is in EXT_TO_LANG to be treated as the extraction directory, without requiring the hyphen index to be greater than 0. The code's condition `'hyphen > 0'` on line 16 excludes components like `'-cpp'` where the hyphen is

at the start. For input '-cpp/func.cpp', the specification would identify '-cpp' as the extraction directory and return '.cpp', but the code's loop skips it, falls back to the immediate parent, and returns '-cpp', violating the specification.

Bug validator

- Trigger summary: hyphen>0 guard incorrectly excludes directory components starting with a hyphen (e.g., -cpp) from being recognized as extraction directories, causing fallback to return the component name instead of the correct source filename.
- Probe stdout:

CONFIRMED — actual: '-cpp' | expected: '.cpp'

INCR-MISMATCH-012 — src--entry_reasoning_pipeline-py--_fqn_to_ident

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/entry_reasoning_pipeline-py/_fqn_to_ident.py`。
- Reasoner 结果: `logic_verification_results/src/entry_reasoning_pipeline-py/_fqn_to_ident.json`。
- 详细报告: `src--entry_reasoning_pipeline-py--_fqn_to_ident.md`。
- Probe: `probe_src--entry_reasoning_pipeline-py--_fqn_to_ident.py`。

完整生成 SPEC

```
[SPEC]
Unit: fm_agent/extracted_functions/src/entry_reasoning_pipeline-py/_fqn_to_ident.py

_fqn_to_ident(fqn) -> str

Pre-condition:
- fqn is a non-empty string of "::"-delimited components

Post-condition:
- Returns the class-qualified function identifier obtained by removing the
  path prefix up to and including the source-file component from fqn
- A component is recognized as a source-file component when it consists of
  a non-empty base name, a hyphen, and a suffix that is a recognized
  source-file language extension
- When fqn contains more than one source-file component, the rightmost one
  determines where the prefix ends
- When fqn contains no source-file component, returns the last component
  of fqn unchanged
- The returned string is non-empty
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns the class-qualified function identifier obtained by removing the
  path prefix up to and including the source-file component from fqn
- A component is recognized as a source-file component when it consists of
  a non-empty base name, a hyphen, and a suffix that is a recognized
  source-file language extension
- When fqn contains more than one source-file component, the rightmost one
  determines where the prefix ends
- When fqn contains no source-file component, returns the last component
  of fqn unchanged
- The returned string is non-empty
```

- 推导 actual behavior:

```
Let parts = fqn.split("::"), n = len(parts). Let i be the largest index in the range [0, n-1] such that there exists an integer pos > 0 where parts[i][pos] == '-' and parts[i][pos+1:] is a key in EXT_TO_LANG. If such an i exists, then result = "::".join(parts[i+1:]) (which may be an empty string if i == n-1). Otherwise, result = parts[-1].
```

- Code evidence:

```
Line 15: return "::".join(parts[i + 1:])
```

- Trigger condition:

When the rightmost source-file component is the last component of the FQN, parts[i+1:] is empty, so the code returns an empty string, violating the specification requirement that the returned string is non-empty.

Bug validator

- Trigger summary: When the rightmost source-file component is the last FQN component, parts[i+1:] is empty, so "::".join([]) returns an empty string, violating the non-empty return value requirement.
- Probe stdout:

```
CONFIRMED
Test 3 (src::storage-cpp):      actual='' | expected='storage-cpp'
Test 4 (storage-cpp):          actual='' | expected='storage-cpp'
Tests 1,2,5 (no-bug cases):    all passed
```

INCR-MISMATCH-013 — `src--entry_reasoning_pipeline-py--_select_functions_by_source`

- 人工审计: 推理误判。
- Validator: **not confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/entry_reasoning_pipeline-py/_select_functions_by_source.py`。
- Reasoner 结果: `logic_verification_results/src/entry_reasoning_pipeline-py/_select_functions_by_source.json`。
- 详细报告: `src--entry_reasoning_pipeline-py--_select_functions_by_source.md`。
- Probe: `probe_src--entry_reasoning_pipeline-py--_select_functions_by_source.py`。

完整生成 SPEC

```
[SPEC]
Unit: fm_agent/extracted_functions/src/entry_reasoning_pipeline-
py/_select_functions_by_source.py

_select_functions_by_source(proj_dir, entry_func, end_funcs, extra_call_edges=None) ->
(dict[str, set[str]], dict[str, set[str]])

Pre-condition:
- proj_dir is an existing directory (the project root)
- entry_func is a non-empty fully-qualified function name string
- end_funcs is an iterable of zero or more fully-qualified function name strings
- extra_call_edges, when provided, contributes supplemental call edges through a
  format recognized by _build_call_graph

Post-condition:
- proj_dir is never mutated; all mutations occur in a temporary sibling directory
  that is destroyed before this function returns
- Returns a tuple (all_by_source, keep_by_source) where:
  - all_by_source is a dict mapping each source-file relative path to the set of
    ALL function names that were extractable from that source file
```

- `keep_by_source` is a dict mapping each source-file relative path to the set of function names that are transitively reachable from `entry_func` in the static call graph; when `end_funcs` is non-empty, this set is further restricted to function names that lie on at least one call-chain path from `entry_func` to some member of `end_funcs`
- Raises `ValueError` when:
 - No extractable source files are found under `proj_dir`
 - No extractable functions are found under `proj_dir`
 - `entry_func` is not among the extracted functions
 - `end_funcs` is non-empty and no member of `end_funcs` is reachable from `entry_func` in the call graph
- When `extra_call_edges` is provided, its supplemental edges contribute to the call graph used for reachability analysis

[SPEC]

Reasoner 差异

- SPEC claim:

- `proj_dir` is never mutated; all mutations occur in a temporary sibling directory that is destroyed before this function returns
- Returns a tuple (`all_by_source`, `keep_by_source`) where:
 - `all_by_source` is a dict mapping each source-file relative path to the set of ALL function names that were extractable from that source file
 - `keep_by_source` is a dict mapping each source-file relative path to the set of function names that are transitively reachable from `entry_func` in the static call graph; when `end_funcs` is non-empty, this set is further restricted to function names that lie on at least one call-chain path from `entry_func` to some member of `end_funcs`
- Raises `ValueError` when:
 - No extractable source files are found under `proj_dir`
 - No extractable functions are found under `proj_dir`
 - `entry_func` is not among the extracted functions
 - `end_funcs` is non-empty and no member of `end_funcs` is reachable from `entry_func` in the call graph
- When `extra_call_edges` is provided, its supplemental edges contribute to the call graph used for reachability analysis

- 推导 actual behavior:

After executing the code block, exactly one of the following holds:

- 1. `**Exception propagation path.**`**
 - If ``_make_run_copy(proj_dir, sel_dir)`` raises an exception, it propagates; ``proj_dir`` is not modified, ``sel_dir`` may not exist or be partial.
 - If ``_enumerate_source_files(sel_dir)`` returns an empty list, a ``ValueError`` is raised with a message indicating no extractable source files; ``proj_dir`` is not modified, ``sel_dir`` exists and is a copy of ``proj_dir`` (up to the point of enumeration).
 - If any subsequent operation (``shutil.rmtree``, ``os.makedirs``, ``open`/`json.dump``, ``try_codegen_init``, ``run_extraction``, ``_collect_phase_files``) raises an exception, it propagates; ``proj_dir`` is not modified, and intermediate state under ``sel_dir`/`work_dir`` may exist.
- 2. `**Normal flow path.**`**
 - No exception is raised. ``proj_dir`` remains unmodified.
 - ``sel_dir`` (with name ``proj_dir` + '.fm-entry-select'``) exists and contains a full copy of ``proj_dir`` at the time of the call.
 - Inside ``sel_dir``, the directory ``fm_agent`` (``work_dir``) exists and is empty of previous extractions (any prior ``fm_agent`/`` was removed).
 - ``work_dir/phases.json`` contains a JSON object ``{"phases": [{"phase": 0, "name": "all", "modules": [{"name": "all", "source_files": source_files}]}]`` where ``source_files`` is the non-empty list of extractable source file paths returned by ``_enumerate_source_files(sel_dir)``.
 - If a codegraph index could be built for ``sel_dir``, it has been initialized (``try_codegen_init``); otherwise, extraction will have proceeded without it.
 - ``run_extraction(sel_dir, work_dir, force=True)`` has completed, writing extracted function files under ``work_dir/extracted_functions`/``.
 - ``phase_files`` is bound to the result of ``_collect_phase_files(work_dir, phase)``, which is a list of ``(extracted_file_relative_path, module_name)`` tuples. This list may be empty.
 - Execution point is immediately after the evaluation of the condition ``not phase_files``. The variable ``source_files`` is alive and non-empty, and ``sel_dir``, ``work_dir``, ``phase`` are in scope.

Formally, let ``SrcCopy(d)`` (copy of `d` at `sel_dir`` `proj_dir`` unchanged)`. Then the normal outcome satisfies:

```

proj_dir ExistingDirectory entry_func NonEmptyFQN end_funcs Iterable[FQN]
(source_files = _enumerate_source_files(sel_dir) source_files )
(phase = {phase:0,name:all,modules:[{name:all,source_files: source_files}])

```



```
(work_dir = se_dir + /fm_agent)
FilesExist(work_dir/phases.json)  jsonContent(work_dir/phases.json) = {phases:[phase]}
(try_codegraph_init(se_dir) has succeeded or skipped without error)
(post-run_extraction: work_dir/extracted_functions exists with extracted artifacts)
(phase_files = _collect_phase_files(work_dir, phase) : List[(file, module)])
````
```

If an exception occurs on any step, the traceback reflects that exception.

- Code evidence:

```
Line 40: if not phase_files: (and subsequent missing return statement)
```

- Trigger condition:

The code block never returns the required tuple (all\_by\_source, keep\_by\_source); after reaching line 40 the function falls off and returns None, violating the specification.

#### Bug validator

- Trigger summary: Bug claim asserted the function falls off without returning, but source code inspection confirms an explicit raise in the empty-phase\_files guard and an explicit return statement at function end.
- Probe stdout:

```
NOT CONFIRMED - has_return=True, found_guard=True, has_raise=True
```

## src/env\_check-py

### INCR-MISMATCH-014 — src--env\_check-py--\_check\_codegraph\_version

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: [src/env\\_check-py/\\_check\\_codegraph\\_version.py](#)。
- Reasoner 结果: [logic\\_verification\\_results/src/env\\_check-py/\\_check\\_codegraph\\_version.json](#)。
- 详细报告: [src--env\\_check-py--\\_check\\_codegraph\\_version.md](#)。
- Probe: [probe\\_src--env\\_check-py--\\_check\\_codegraph\\_version.py](#)。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/env_check.py

_check_codegraph_version(config) -> (bool, str | None)

Pre-condition:
- config provides access to a codegraph version string via settings.codegraph.version and a
 binary directory path via settings.codegraph.bin_dir

Post-condition:
- Returns (True, None) when the configured codegraph version, after stripping whitespace and a
 leading "v" prefix, is the empty string – no version is pinned so verification is skipped
- Otherwise, attempts to obtain the installed codegraph binary's version string by executing
 it with a --version flag and capturing its standard output
- Returns (False, message) when the version string could not be obtained (binary missing, not
 executable, or times out), with a message identifying the configured binary directory and
 instructing the user to re-run ./install.sh
- Returns (False, message) when the obtained version string does not equal the configured
 pinned version (after stripping whitespace and any leading "v" prefix), with a message stating
 the installed and pinned versions and instructing the user to re-run ./install.sh
- Returns (True, None) when the obtained version string equals the configured pinned version
```

```
- Never raises an exception: all error paths return (False, message) with a human-readable description [SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns (True, None) when the configured codegraph version, after stripping whitespace and a leading "v" prefix, is the empty string no version is pinned so verification is skipped
- Otherwise, attempts to obtain the installed codegraph binary's version string by executing it with a --version flag and capturing its standard output
- Returns (False, message) when the version string could not be obtained (binary missing, not executable, or times out), with a message identifying the configured binary directory and instructing the user to re-run ./install.sh
- Returns (False, message) when the obtained version string does not equal the configured pinned version (after stripping whitespace and any leading "v" prefix), with a message stating the installed and pinned versions and instructing the user to re-run ./install.sh
- Returns (True, None) when the obtained version string equals the configured pinned version
- Never raises an exception: all error paths return (False, message) with a human-readable description
```

- 推导 actual behavior:

After the execution of `_check_codegraph_version(config)`, the function returns a tuple (status, msg) where status is True if the installed codegraph version matches the pinned version (or if no version is pinned), and False otherwise. The behavior is defined as follows:

```
- Let w = config.settings.codegraph.version.strip().removeprefix('v').
- If w is empty, return (True, None).
- Otherwise, let cmd = _codegraph_cmd(). By its specification, cmd is an absolute path to an executable file under config.settings.codegraph.bin_dir if that file exists and is executable, else the string 'codegraph' (to be resolved via PATH).
- Attempt to obtain the installed version by executing subprocess.run([cmd, '--version'], capture_output=True, text=True, timeout=10). On success, let got = strip(stdout). On any OSError or subprocess.SubprocessError, let got = ''.
- Let bin_dir = os.path.expanduser(config.settings.codegraph.bin_dir) (used only in messages).
- If got == '', return (False, 'codegraph (pinned v{want}) is not installed at {bin_dir} run ./install.sh (C/C++ extraction falls back to the regex extractor otherwise)').
- Else if got != w, return (False, 'codegraph {got} is installed but v{w} is pinned in fm-agent.toml re-run ./install.sh to install the pinned build.').
- Else (got == w), return (True, None).
```

Formally:

```
config :
 let w = strip(removeprefix(config.settings.codegraph.version, 'v'))
 result =
 if w == '' then (True, None)
 else let cmd = _codegraph_cmd()
 let got = if out : subprocess.run([cmd, '--version'], capture_output=True, text=True, timeout=10) completes successfully out = it.stdout.strip()
 then out else '' (including cases of OSError subprocess.SubprocessError)
 let bin_dir = os.path.expanduser(config.settings.codegraph.bin_dir)
 if got == '' then (False, 'codegraph (pinned v{w}) is not installed at {bin_dir} run ./install.sh (C/C++ extraction falls back to the regex extractor otherwise).')
 else if got != w then (False, 'codegraph {got} is installed but v{w} is pinned in fm-agent.toml re-run ./install.sh to install the pinned build.')
 else (True, None)
```

- Code evidence:

Line 19: if not got:

- Trigger condition:

The code treats an empty version string (got='') the same as a failure to execute the binary, returning a 'not installed' error. The specification requires that if the binary executes successfully (even with empty output), the obtained version string (empty) should be compared against the pinned version. An empty string is not equal to a non-empty pinned version, so the correct behavior per spec is to return a version-mismatch error, not a missing-binary error.

- Trigger summary: When codegraph binary executes successfully but returns empty stdout, the empty string is falsy in 'if not got:' causing the code to report 'not installed' instead of a version-mismatch error as required by the specification.
- Probe stdout:

```
CONFIRMED – actual: (False, 'codegraph (pinned v1.2.3) is not installed at
/tmp/bug_probe_env_check_codegraph – run ./install.sh (C/C++ extraction falls back to the regex
extractor otherwise).') | expected: version-mismatch error, not 'not installed' error
```

## src/extract-py

### INCR-MISMATCH-015 — src--extract-py--run\_extraction

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 2。
- 原始函数与 SPEC: [src/extract-py/run\\_extraction.py](#)。
- Reasoner 结果: [logic\\_verification\\_results/src/extract-py/run\\_extraction.json](#)。
- 详细报告: [src--extract-py--run\\_extraction.md](#)。
- Probe: [probe\\_src--extract-py--run\\_extraction.py](#)。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/extract.py

run_extraction(proj_dir, work_dir=None, force=False, verbose=False) -> (int, int)

Pre-condition:
- proj_dir is a path to an existing directory
- work_dir (or proj_dir if work_dir is None) contains a phases.json file whose
 structure includes a list of phases, each with modules containing source_files
 entries that are relative paths from proj_dir

Post-condition:
- A function is extracted from every source file listed in phases.json that
 (a) has a file extension recognized as a supported language, and (b) does not
 match test-file heuristics
- Each extracted function is written as a separate file under
 work_dir/extracted_functions/; the output path is constructed by replacing
 the last dot in the source filename with a hyphen to form a directory, then
 placing the canonicalized function name with the original extension inside
- An output file that already exists and contains both [SPEC] marker lines and
```

#### Reasoner 差异

- SPEC claim:

```
- A function is extracted from every source file listed in phases.json that
 (a) has a file extension recognized as a supported language, and (b) does not
 match test-file heuristics
- Each extracted function is written as a separate file under
 work_dir/extracted_functions/; the output path is constructed by replacing
 the last dot in the source filename with a hyphen to form a directory, then
 placing the canonicalized function name with the original extension inside
- An output file that already exists and contains both [SPEC] marker lines and
 [INFO] marker lines is left unchanged and counted as skipped, unless force
 is True
- After all extractions complete, every function file in the output tree
 contains exactly one function body (validated)
- Returns (written_count, skipped_count): the number of function files newly
 written and the number of already-specced files skipped, both non-negative
```

- 推导 actual behavior:

After successful execution (no exception):

1. ``phases_path`` exists and was successfully read as a JSON object ``phases_data``.
2. ``source_files`` is a list of all relative source file paths obtained by iterating over ``phases_data['phases'][*]['modules'][*]['source_files'][*]``.
3. ``registry_langs`` is the second element of the tuple returned by ``batch_extract_all(proj_dir)``, representing the set of languages detected in the project.
4. For each ``src_rel`` in ``source_files``:
  - If ``_is_test_file(src_rel)`` returns ``True``, the file is skipped (no output, no contribution to ``skipped``).
  - Otherwise, ``src_path = os.path.join(proj_dir, src_rel)`` is formed. If ``src_path`` does not exist, a warning is logged and the file is skipped (no output, no contribution to ``skipped``).
  - Otherwise, the language ``lang_key`` is detected from the file extension of ``src_rel`` (by an external mapping), and ``funcs = extract_functions_from_file(src_path, lang_key)`` is called, returning a list of ``(func_name, func_source)`` pairs.
  - For each element in ``funcs``:
    - \* An output filename is generated by ``_safe_filename(func_name, ext)`` where ``ext`` is the extension of ``src_rel``.
    - \* The full output path is ``os.path.join(output_base, output_filename)`` with ``output_base = os.path.join(work_dir, 'extracted_functions')`` (the directory is created if it does not exist).
    - \* If ``force`` is ``False`` and the output path already exists as a regular file, the function is `**skipped**`: ``skipped`` is incremented by 1, and the existing file is not overwritten.
    - \* Otherwise (``force`` is ``True`` or the output file does not exist), the function source text is written to the output file (overwriting if it exists), and ``written`` is incremented by 1.
5. After processing all source files, ``_validate_extraction(output_base, registry_langs)`` is called. It returns a list of files under ``output_base`` that do not contain exactly one function body; the function may log or accumulate warnings based on this list but does `**not**` raise an exception.
6. The function returns a tuple ``(written, skipped)`` where:
  - ``written`` is the total number of function bodies written to files under ``output_base`` during this invocation.
  - ``skipped`` is the total number of function bodies that were `**not**` written because ``force=False`` and their corresponding output file already existed.

Formally, let:

- ``D = { src_rel in source_files | not _is_test_file(src_rel) and os.path.exists(os.path.join(proj_dir, src_rel)) }``
- For each ``src_rel`` in ``D``, let ``L_src = extract_functions_from_file(os.path.join(proj_dir, src_rel), lang_key_src)`` be the list of extracted functions.
- Define ``F = { (out_path, func_source) | src_rel in D, (name, body) in L_src, out_path = join(output_base, _safe_filename(name, ext_src)) }``.
- Then ``written = |{ (out_path, src) in F : force=True or not exists(out_path) }|`` and ``skipped = |{ (out_path, src) in F : force=False and exists(out_path) }|``.
- All other files under ``output_base`` are untouched except those created or overwritten by this process.

- Code evidence:

Line 30-... (after visible block): The loop skips writing when the output file already exists without checking for [SPEC]/[INFO] markers.

- Trigger condition:

Condition A skips any function whose output file already exists when `force=False`, but the specification requires skipping only if the existing file contains both [SPEC] and [INFO] marker lines. The example output file lacks those markers, so the specification demands it be overwritten, while the code skips it.

## Bug validator

- Trigger summary: `is_file_ready` requires exactly 2 SPEC + 2 INFO markers in strict order, so a file with only 1 SPEC + 1 INFO is treated as not-ready and gets overwritten, violating the spec that says "contains both [SPEC] marker lines and [INFO] marker lines" should be sufficient to skip.
- Probe stdout:

Extraction complete: 1 written, 0 skipped.  
 CONFIRMED - `is_file_ready` returned False for 1+1 markers, causing overwrite (written=1,

## src/file\_utils.py

### INCR-MISMATCH-016 — src--file\_utils.py--\_get\_phase\_files

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/file_utils.py/_get_phase_files.py`。
- Reasoner 结果: `logic_verification_results/src/file_utils.py/_get_phase_files.json`。
- 详细报告: `src--file_utils.py--_get_phase_files.md`。
- Probe: `probe_src--file_utils.py--_get_phase_files.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/file_utils.py/_get_phase_files.py

_get_phase_files(phases_data, phase_num, input_dir) -> list[str]

Pre-condition:
- phases_data is a dict conforming to the phases.json schema: it has a "phases" key
 whose value is a list of phase info dicts, each containing at least a "phase" field
 (integer phase number) and a "modules" field.
- phase_num is an integer that matches the "phase" field of exactly one dict in
 phases_data["phases"]. If no phase dict has a matching "phase" field, StopIteration
 is raised.
- Each module dict in the matched phase has a "source_files" key whose value is an
 iterable of string paths using "/" separators.
- input_dir is an existing directory path under which extracted function files are
 stored following the engine directory-layout convention.

Post-condition:
- Returns a list of relative path strings, each being the path from input_dir to a
 regular file located under an extracted-function subdirectory.
- Each returned path originates from a source file declared in the modules of the
 phase identified by phase_num; the mapping from a source file path to its
 extracted-function subdirectory follows the engine convention: the last "." in the
 source file's basename is replaced by "-", and the resulting name is used as a
 subdirectory under input_dir joined with the source file's directory portion.
- Source files whose corresponding extracted-function subdirectory does not exist
 under input_dir contribute no entries to the result (they are silently skipped).
- Within each extracted-function subdirectory, contained regular files appear in
 lexicographically sorted order by filename.
- The overall order of paths in the result preserves: the iteration order of
 phases_data["phases"], the iteration order of modules within the matched phase,
 and the iteration order of source_files within each module.
- The returned list may be empty when the matched phase has no modules, no source
 files, or none of its source files have an existing extracted-function directory.
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

- Returns a list of relative path strings, each being the path from input\_dir to a regular file located under an extracted-function subdirectory.
- Each returned path originates from a source file declared in the modules of the phase identified by phase\_num; the mapping from a source file path to its extracted-function subdirectory follows the engine convention: the last "." in the source file's basename is replaced by "-", and the resulting name is used as a subdirectory under input\_dir joined with the source file's directory portion.
- Source files whose corresponding extracted-function subdirectory does not exist under input\_dir contribute no entries to the result (they are silently skipped).
- Within each extracted-function subdirectory, contained regular files appear in lexicographically sorted order by filename.
- The overall order of paths in the result preserves: the iteration order of

phases\_data["phases"], the iteration order of modules within the matched phase, and the iteration order of source\_files within each module.

- The returned list may be empty when the matched phase has no modules, no source files, or none of its source files have an existing extracted-function directory.

- 推导 actual behavior:

If a phase dict with `phase == phase\_num` exists, the function returns a list of relative file paths (strings) from `input\_dir` for all regular files found inside the extracted directories that exist. Otherwise, `StopIteration` is raised; other exceptions (e.g., `OSError`) may propagate if filesystem operations fail. Formally:

```
p phases_data["phases"] : p["phase"] = phase_num
let M = {p | p phases_data["phases"] p["phase"] = phase_num} (singleton by pre-condition).
For each module M["modules"], for each src_file module["source_files"]:
 let base = basename(src_file), ext_idx = base.rfind("."),
 subdir = (base[:ext_idx] + "-" + base[ext_idx+1:]) if ext_idx > 0 else base,
 extracted_dir = join(input_dir, dirname(src_file), subdir).
 If is_dir(extracted_dir), then for every (root, dirs, files) in os.walk(extracted_dir) (top-
down, arbitrary order),
 for every fname sorted(files):
 let fpath = join(root, fname).
 If is_file(fpath), then append relpath(fpath, input_dir) to result list.
Return the final list (deterministic for a given filesystem state).

Otherwise (no matching phase), `StopIteration` is raised.
```

- Code evidence:

```
Line 20: for root, _dirs, fnames in os.walk(extracted_dir):
Line 21: for fname in sorted(fnames):
Line 22: fpath = os.path.join(root, fname)
Line 23: if os.path.isfile(fpath):
Line 24: phase_files.append(os.path.relpath(fpath, input_dir))
```

- Trigger condition:

Specification requires that all regular files contained within an extracted-function subdirectory appear sorted by filename. The code uses `os.walk` and sorts files per-directory, so when subdirectories exist the overall list is not globally sorted. In the counterexample, the code returns `['dir/file-cpp/z.txt', 'dir/file-cpp/sub/a.txt']` but the specification demands `['dir/file-cpp/sub/a.txt', 'dir/file-cpp/z.txt']`.

#### Bug validator

- Trigger summary: When an extracted-function subdirectory contains nested subdirectories, `os.walk` yields root-level files before child-directory files, producing an overall file list that is not globally sorted by filename as the spec requires.
- Probe stdout:

```
CONFIRMED — actual (os.walk order): ['src/file-cpp/z.txt', 'src/file-cpp/sub/a.txt'] | expected
(spec order): ['src/file-cpp/sub/a.txt', 'src/file-cpp/z.txt']
```

## src/generate\_topdown\_layers-py

### INCR-MISMATCH-017 — src--generate\_topdown\_layers-py--\_collect\_phase\_files

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/generate_topdown_layers-py/_collect_phase_files.py`。

- Reasoner 结果: `logic_verification_results/src/generate_topdown_layers-py/_collect_phase_files.json`。
- 详细报告: `src--generate_topdown_layers-py--_collect_phase_files.md`。
- Probe: `probe_src--generate_topdown_layers-py--_collect_phase_files.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/generate_topdown_layers-py/_collect_phase_files.py

_collect_phase_files(proj_dir, phase_data) -> list[tuple[str, str]]

Pre-condition:
- proj_dir is a path to an existing directory
- phase_data is a dict that may contain a "modules" key; if present, its value is an iterable of module dicts, each with a "name" (str) and optionally "source_files" (iterable of str relative paths)

Post-condition:
- Returns a list of (file_path, module_name) pairs, where module_name is the "name" of a module in phase_data
- For each source file declared in a module: the source file's basename extension is stripped by replacing the last "." with "-" (e.g., "loader.cpp" → "loader-cpp"), and the resulting directory name is resolved under proj_dir/extracted_functions/ alongside the source file's parent directory
- Every regular file found in such a directory is collected into the result, each paired with the name of the module that declared the source file
- Directories that do not exist on disk are skipped with no error raised
- Returns an empty list when phase_data has no "modules" key, the modules list is empty, or no extracted-function directories exist on disk
- The returned list preserves no guaranteed ordering across calls
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns a list of (file_path, module_name) pairs, where module_name is the "name" of a module in phase_data
- For each source file declared in a module: the source file's basename extension is stripped by replacing the last "." with "-" (e.g., "loader.cpp" → "loader-cpp"), and the resulting directory name is resolved under proj_dir/extracted_functions/ alongside the source file's parent directory
- Every regular file found in such a directory is collected into the result, each paired with the name of the module that declared the source file
- Directories that do not exist on disk are skipped with no error raised
- Returns an empty list when phase_data has no "modules" key, the modules list is empty, or no extracted-function directories exist on disk
- The returned list preserves no guaranteed ordering across calls
```

- 推导 actual behavior:

```
The function returns a list `results` such that:

results = [(file_path, module_name) for each module in phase_data.get('modules', []) if
'source_files' in module for each src_file in module['source_files'] where
os.path.isdir(func_dir) for each regular file (os.path.isfile) with path file_path found by
recursively walking func_dir via os.walk].

Here func_dir = os.path.join(proj_dir, 'extracted_functions', src_dir, dir_name) if src_dir
(os.path.dirname(src_file)) is non-empty, else os.path.join(proj_dir, 'extracted_functions',
dir_name). dir_name is derived from os.path.basename(src_file): if a last dot position > 0
exists, dir_name = basename[:last_dot] + '-' + basename[last_dot+1:]; otherwise dir_name =
basename.

file_path is the absolute path (given proj_dir) of each file inside the extracted directory
tree; module_name is the string module['name']. The order in results matches iteration order of
modules, then source_files, and for each directory the order of files from os.walk (depth-first,
top-down). The function has no side effects and raises no exceptions under the pre-condition.
```

- Code evidence:

```
Line 13: last_dot = src_base.rfind(".")
Line 14: if last_dot > 0:
Line 15: dir_name = src_base[:last_dot] + "-" + src_base[last_dot + 1:]
Line 17: dir_name = src_base
```

- Trigger condition:

When a source file's basename starts with a dot (e.g., '.hidden'), the last dot is at index 0, and the condition `last_dot > 0` fails, causing `dir_name` to remain '.hidden'. The specification replaces the last '.' with '-', which would produce '-hidden'. This leads the code to look for a directory '.hidden' instead of the expected '-hidden', missing files that should have been included.

#### Bug validator

- Trigger summary: When a source file's basename starts with a dot (e.g., '.hidden'), `last_dot` equals 0 and the guard `last_dot > 0` fails, so the code looks for directory '.hidden' instead of the spec-correct '-hidden'.
- Probe stdout:

```
CONFIRMED — actual:
{'/tmp/probe_collect_phase_files_cztzyoif/extracted_functions/.hidden/buggy_file.py'} | spec-
expected (look in -hidden/): {'/tmp/probe_collect_phase_files_cztzyoif/extracted_functions/-
hidden/correct_file.py'} | code looked in .hidden/ instead because last_dot>0 is False when
last_dot==0
```

---

#### INCR-MISMATCH-018 — `src--generate_topdown_layers-py--_strip_comments_from_source`

- 人工审计: 推理误判。
- Validator: **not confirmed**; 尝试次数: 2。
- 原始函数与 SPEC: `src/generate_topdown_layers-py/_strip_comments_from_source.py`。
- Reasoner 结果: `logic_verification_results/src/generate_topdown_layers-py/_strip_comments_from_source.json`。
- 详细报告: `src--generate_topdown_layers-py--_strip_comments_from_source.md`。
- Probe: `probe_src--generate_topdown_layers-py--_strip_comments_from_source.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/generate_topdown_layers-py/_strip_comments_from_source.py

_strip_comments_from_source(text, lang_key) -> str

Pre-condition:
- text is a string containing source code, which may be empty
- lang_key is a string identifying a programming language

Post-condition:
- Returns a string of the same length as text (same number of characters)
- Every character position that lies within a comment region in the input
 is replaced with a space character (' ') in the output, where a comment
 region is defined according to the language-specific comment syntax:
 * When the language configuration for lang_key has comment_prefix "#":
 a comment region spans from a '#' character (that is not inside a
 string literal) to the end of the same line, including the '#'.
 * When the language configuration for lang_key has comment_prefix
 "/*" (or when lang_key is not found, defaulting to "/*"): a line-
 comment region spans from "/*" to end of line; a block-comment
 region spans from "/*" to the next "*/" (non-nesting). The opening
 and closing markers are part of the comment region.
 * Characters within comment regions that are newline characters (\n)
 are never replaced; they remain unchanged.
```



- Every character position that lies within a string-literal region in the input is replaced with a space character in the output. A string-literal region includes its delimiting quote characters and all characters between them. Delimiters may be:
  - \* single-quote ('...'), double-quote ("..."),
  - \* triple-single-quote ('''...'''), triple-double-quote ("""...""").
- Backslash-escape handling: a backslash (\) and the immediately following character (if any) are treated as non-delimiting and are replaced with spaces; if the following character is a newline, only the backslash is replaced, the newline is preserved.
- Any character position that is neither within a comment region nor within a string-literal region is preserved unchanged (including whitespace, punctuation, identifiers, keywords, etc.).
- The order and positions of non-replaced characters are unchanged; the returned string has exactly the same length as the input text.

[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns a string of the same length as text (same number of characters)
- Every character position that lies within a comment region in the input is replaced with a space character ( ' ') in the output, where a comment region is defined according to the language-specific comment syntax:
  - \* When the language configuration for lang\_key has comment\_prefix "#": a comment region spans from a '#' character (that is not inside a string literal) to the end of the same line, including the '#'.
  - \* When the language configuration for lang\_key has comment\_prefix "/" (or when lang\_key is not found, defaulting to "/"): a line-comment region spans from "/" to end of line; a block-comment region spans from "/\*" to the next "\*/" (non-nesting). The opening and closing markers are part of the comment region.
  - \* Characters within comment regions that are newline characters (\n) are never replaced; they remain unchanged.
- Every character position that lies within a string-literal region in the input is replaced with a space character in the output. A string-literal region includes its delimiting quote characters and all characters between them. Delimiters may be:
  - \* single-quote ('...'), double-quote ("..."),
  - \* triple-single-quote ('''...'''), triple-double-quote ("""...""").
- Backslash-escape handling: a backslash (\) and the immediately following character (if any) are treated as non-delimiting and are replaced with spaces; if the following character is a newline, only the backslash is replaced, the newline is preserved.
- Any character position that is neither within a comment region nor within a string-literal region is preserved unchanged (including whitespace, punctuation, identifiers, keywords, etc.).
- The order and positions of non-replaced characters are unchanged; the returned string has exactly the same length as the input text.

- 推导 actual behavior:

After the code block finishes execution, the input source text has been processed to mask all string literals (single-quoted, double-quoted, and triple-quoted strings, with backslash-escaped characters handled) by replacing their characters with spaces. The variable 'result' is a list of characters of the same length as 'text'. For every index j, if the j-th character in the original 'text' lies inside a string literal (according to the quoting rules of the language and the processing logic), then result[j] equals a space ' '; otherwise result[j] equals text[j]. The index 'i' is equal to len(result), indicating that the scanning loop has completed. The variables 'lang\_cfg', 'comment\_prefix', and 'is\_hash\_comment' hold the derived language configuration. Formally: len(result) = len(text) AND (forall j in [0, len(text)-1] : (in\_string\_literal(text, j) -> result[j] = ' ') and (not in\_string\_literal(text, j) -> result[j] = text[j])) AND i = len(result) AND is\_hash\_comment = (comment\_prefix = '#') AND comment\_prefix = (LANG\_CONFIG.get(lang\_key, {}).get('comment\_prefix', '/')) AND lang\_cfg = LANG\_CONFIG.get(lang\_key, {})

- Code evidence:

Line 59-64 (hash comment handling) are either not executed or do not affect the behavior described by A. According to A, the code does not mask comments, violating the requirement in B that comment regions be replaced with spaces.

- Trigger condition:

Condition A states that only string literals are masked; characters not in string literals are preserved unchanged. Specification B requires that comment regions also be masked (replaced with spaces). Any input containing a comment (e.g., '# comment\n') will result in the comment characters being left as-is by A, whereas B requires them to be spaces. This is a direct violation.

#### Bug validator

- Trigger summary: Verification tool claimed function only masks string literals, not comments. Empirical testing with Python #, C++ //, and C /\* \*/ comments shows all comment types are correctly replaced with spaces.
- Probe stdout:

```
=== Test 1: Python hash comment ===
PASS: True
Input: 'x = 1 # this is a comment\n'
Result: 'x = 1 \n'
Expected: 'x = 1 \n'

=== Test 2: Python string + comment ===
PASS: True
Input: 'print("hello") # greet\n'
Result: 'print() \n'
Expected: 'print() \n'

=== Test 3: C++ // comment ===
PASS: True
Input: 'int x = 1; // comment\n'
Result: 'int x = 1; \n'
Expected: 'int x = 1; \n'

=== Test 4: C block comment ===
PASS: True
Input: 'int /* block */ x;\n'
Result: 'int x;\n'
Expected: 'int x;\n'

=== Test 5: Unknown lang # (NOT comment) ===
PASS: True
Input: 'code # not a comment for unknown lang\n'
Result: 'code # not a comment for unknown lang\n'
Expected: 'code # not a comment for unknown lang\n'

=== Test 6: Backslash escape in string + comment ===
PASS: True
Input: 'print("hello\\"world") # comment\n'
Result: 'print() \n'

=====
Python # comment: PASS
String + # comment: PASS
C++ // comment: PASS
Block /* */ comment: PASS
Unknown lang #: PASS
Backslash escape: PASS

Bug NOT CONFIRMED: function correctly masks both string literals AND comments.
NOT CONFIRMED
```

## src/git-py

### INCR-MISMATCH-019 — src--git-py--frozen\_worktree

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: [src/git-py/frozen\\_worktree.py](#)。
- Reasoner 结果: [logic\\_verification\\_results/src/git-py/frozen\\_worktree.json](#)。

- 详细报告: `src--git-py--frozen_worktree.md`。
- Probe: `probe_src--git-py--frozen_worktree.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/git.py

frozen_worktree(proj_dir, exclude=("fm_agent",), copy_excluded=True) -> yields str

Pre-condition:
- proj_dir is a filesystem path; it may or may not be a git repository and may
 or may not contain commits
- exclude is an iterable of strings naming subdirectories of proj_dir to keep out
 of the git snapshot commit
- copy_excluded is a boolean

Post-condition:
- A new, unique temporary directory is created under the system tmpdir. Its name
 begins with "fm_agent_wt_" followed by the basename of proj_dir.
- When proj_dir is a git repository with a reachable HEAD commit:
 - A private git index (GIT_INDEX_FILE) is used so that proj_dir's real index
 and working tree are never modified.
 - The snapshot commit captures the full state of proj_dir at entry time:
 HEAD tree + all tracked modifications + all untracked files, with every
 path in exclude removed from the snapshot commit tree.
 - That commit becomes a detached git worktree checked out inside the tmpdir
 at a "snapshot" subdirectory. The yielded path is this snapshot subdirectory.
 - If copy_excluded is truthy: for each name in exclude, if the corresponding
 subdirectory exists in proj_dir and does not already exist at the same
 relative path in the snapshot worktree, that subdirectory is recursively
 copied (with symlinks preserved) into the snapshot worktree.
- When proj_dir is NOT a git repository or has no reachable HEAD:
 - A plain recursive directory copy is performed from proj_dir into the
 snapshot subdirectory, using copytree with each name in exclude passed as
 an ignore pattern and with symlinks preserved. The yielded path is the
 snapshot subdirectory.
 - If copy_excluded is truthy: excluded subdirectories are copied into the
 snapshot worktree under the same conditions as the git-path case.
- The absolute path of the snapshot worktree is printed to stdout along with
 platform-appropriate removal instructions referencing either "git worktree
 remove" (git path) or "rm -rf" (non-git path).
- The snapshot worktree and its parent temporary directory persist after the
 context manager exits; automatic cleanup is not performed.
- If any git or filesystem operation fails (e.g. git command returns non-zero,
 directory is not writable), the corresponding subprocess.CalledProcessError or
 OSError propagates to the caller.

[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- A new, unique temporary directory is created under the system tmpdir. Its name
 begins with "fm_agent_wt_" followed by the basename of proj_dir.
- When proj_dir is a git repository with a reachable HEAD commit:
 - A private git index (GIT_INDEX_FILE) is used so that proj_dir's real index
 and working tree are never modified.
 - The snapshot commit captures the full state of proj_dir at entry time:
 HEAD tree + all tracked modifications + all untracked files, with every
 path in exclude removed from the snapshot commit tree.
 - That commit becomes a detached git worktree checked out inside the tmpdir
 at a "snapshot" subdirectory. The yielded path is this snapshot subdirectory.
 - If copy_excluded is truthy: for each name in exclude, if the corresponding
 subdirectory exists in proj_dir and does not already exist at the same
 relative path in the snapshot worktree, that subdirectory is recursively
 copied (with symlinks preserved) into the snapshot worktree.
- When proj_dir is NOT a git repository or has no reachable HEAD:
 - A plain recursive directory copy is performed from proj_dir into the
 snapshot subdirectory, using copytree with each name in exclude passed as
 an ignore pattern and with symlinks preserved. The yielded path is the
 snapshot subdirectory.
 - If copy_excluded is truthy: excluded subdirectories are copied into the
 snapshot worktree under the same conditions as the git-path case.
- The absolute path of the snapshot worktree is printed to stdout along with
 platform-appropriate removal instructions referencing either "git worktree
 remove" (git path) or "rm -rf" (non-git path).
```

- The snapshot worktree and its parent temporary directory persist after the context manager exits; automatic cleanup is not performed.
- If any git or filesystem operation fails (e.g. git command returns non-zero, directory is not writable), the corresponding subprocess.CallProcessError or OSError propagates to the caller.

- 推导 actual behavior:

After the function body has executed up to and including the yield statement (suspending the generator), the following holds:

- A unique temporary directory base was created via `tempfile.mkdtemp` inside the system temporary directory; its absolute path is stored in variable ``base``.
- The variable ``wt`` holds the absolute path ``os.path.join(base, 'snapshot')``, which is a directory that now exists.

**\*\*If ``proj_dir`` is a git repository with at least one commit\*\*** (i.e., ``git -C proj_dir rev-parse --verify HEAD`` succeeded):

- \* A private git index file was created at ``os.path.join(base, 'index')``.
- \* The HEAD tree was read into that index (``git read-tree HEAD``).
- \* All workingtree changes (tracked edits and untracked files, respecting ``.gitignore``) were staged via ``git add -A``.
- \* For every name in ``exclude``, ``git rm -r --cached --quiet --ignore-unmatch -- <name>`` was executed, removing those entries from the private index if present.
- \* A tree object was written from the index (``git write-tree``).
- \* A new commit object was created with that tree, parent HEAD, and message ``'fm_agent snapshot'`` (``git commit-tree``).
- \* A detached worktree was added at ``wt`` referencing that commit (``git worktree add --detach <wt> <snap>``).
- \* ``wt`` is a valid git checkout containing the committed state plus uncommitted edits and untracked files, **\*\*excluding\*\*** any paths that are gitignored **\*\*or\*\*** whose names are listed in ``exclude``.

**\*\*Otherwise\*\*** (``proj_dir`` is not a git repo or has no commit):

- \* ``shutil.copytree(proj_dir, wt, ignore=shutil.ignore_patterns(*exclude), symlinks=True)`` executed successfully.
- \* ``wt`` is a plain directory copy of ``proj_dir``, preserving symlinks, but omitting any files or subdirectories whose names match the patterns in ``exclude``.
- \* An INFO level log message was emitted stating that a copy was performed.

**\*\*If ``copy_excluded`` is True:\*\***

- \* For each name in ``exclude``, if ``os.path.join(proj_dir, name)`` exists as a directory **\*\*and\*\*** ``os.path.join(wt, name)`` does **\*\*not\*\*** already exist, its contents are copied into the snapshot via ``shutil.copytree(src, dst, symlinks=True)``.
- \* Thus, the snapshot physically contains the original contents of those directories from ``proj_dir``, even though they were excluded from the git commit (or the initial copy).

- Two messages were printed to stdout: one indicating the snapshot path ``wt``, and another with removal instructions (using ``git worktree remove`` if git was used, otherwise ``rm -rf``).
- The generator is suspended at the yield point; the caller receives ``wt`` as the yielded value.
- No cleanup of the temporary base directory or snapshot is performed; they persist on disk after the yield.

**\*\*Formal summary:\*\***

Let ``ProjDir`` be the absolved path. Assume ``ProjDir`` exists.

- ``base = tempfile.mkdtemp(...)`` ``wt = base + '/snapshot'`` ``IsDir(wt)``
- ``(IsGitRepo(ProjDir) HasCommit(ProjDir, HEAD))``
  - ``wt`` is a git worktree with content = ``Committed(ProjDir) WorkingTreeChanges(ProjDir) \ (GitIgnored(ProjDir) exclude)``
- ``(IsGitRepo(ProjDir) HasCommit(ProjDir, HEAD))``
  - ``wt`` is a plain directory with content = ``Content(ProjDir) \ { p | patexclude: Match(pat, baseName(p)) }``
- ``copy_excluded`` ``nameexclude: (IsDir(ProjDir+'/' + name) Exists(wt+'/' + name))``
- ``Content(wt+'/' + name) = Content(ProjDir+'/' + name)``
- Output lines on stdout describe ``wt`` and removal method.
- The generator yields ``wt`` and suspends.

- Code evidence:

Line 45: `_git("add", "-A", env=env)`

- Trigger condition:

The code's use of ``git add -A`` skips files that match ``.gitignore`` patterns, so untracked gitignored files are not included in the snapshot. The specification requires capturing all

untracked files without exception for .gitignore.

## Bug validator

- Trigger summary: git add -A skips untracked files matching .gitignore patterns, so gitignored untracked files are omitted from the snapshot despite the spec requiring all untracked files to be captured.
- Probe stdout:

```
[Pipeline] Snapshot created at: /tmp/fm_agent_wt_testrepo_vqrce874/snapshot
[Pipeline] Snapshot is kept after the run. Remove with: git -C /tmp/bug_probe_nuyn9g_/testrepo
worktree remove --force /tmp/fm_agent_wt_testrepo_vqrce874/snapshot
CONFIRMED - gitignored untracked file 'test.secret' is missing from snapshot. present=False,
normal_untracked_present=True, tracked_present=True
```

## INCR-MISMATCH-020 — `src--git-py--frozen_worktree::_git`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/git-py/frozen_worktree::_git.py`。
- Reasoner 结果: `logic_verification_results/src/git-py/frozen_worktree::_git.json`。
- 详细报告: `src--git-py--frozen_worktree::_git.md`。
- Probe: `probe_src--git-py--frozen_worktree::_git.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/git.py

_git(*args, **kwargs) -> str

Pre-condition:
- proj_dir is a filesystem path in enclosing scope, pointing to a directory.
- *args are zero or more positional string arguments forming the git subcommand
 and its operands (e.g., "add", "-A").
- **kwargs are zero or more keyword arguments forwarded to the subprocess
 invocation, merged with defaults that enforce text-mode output capture and
 non-zero-exit error raising.

Post-condition:
- Executes a git command rooted at proj_dir (equivalent to "git -C proj_dir"
 followed by each positional argument in order) as a child process.
- Neither stdout nor stderr of the child process appears on the parent's
 standard output or standard error streams.
- If the child process terminates with a non-zero exit code, raises
 subprocess.CallledProcessError whose attributes record the invoked command,
 the return code, and the captured stdout and stderr strings.
- If the child process terminates with exit code zero, returns the captured
 stdout with every leading and trailing whitespace character (space, tab,
 newline, carriage return) removed.
- The child process inherits the parent process's environment, subject to
 modification by any env keyword argument passed in **kwargs.
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Executes a git command rooted at proj_dir (equivalent to "git -C proj_dir"
 followed by each positional argument in order) as a child process.
- Neither stdout nor stderr of the child process appears on the parent's
 standard output or standard error streams.
- If the child process terminates with a non-zero exit code, raises
 subprocess.CallledProcessError whose attributes record the invoked command,
```

- the return code, and the captured stdout and stderr strings.
- If the child process terminates with exit code zero, returns the captured stdout with every leading and trailing whitespace character (space, tab, newline, carriage return) removed.
- The child process inherits the parent process's environment, subject to modification by any env keyword argument passed in `**kwargs`.

- 推导 actual behavior:

After the call to `_git` with arguments `*args` and `**kwargs`, and given `proj_dir` a directory path in the enclosing scope:

- If `**kwargs` contains any key among `'check'`, `'capture_output'`, or `'text'`, the function call raises a `TypeError` because of repeated keyword arguments in the `subprocess.run` call; otherwise,
- The function constructs the command list `['git', '-C', proj_dir] + list(args)` and invokes `subprocess.run(cmd, check=True, capture_output=True, text=True, **kwargs)`.
  - If this call raises a `CalledProcessError` (because the git command exited nonzero), that exception propagates unmodified.
  - If it raises any other exception (e.g., `FileNotFoundError` if `git` is not found), that exception also propagates.
  - If the call completes normally, it returns a `CompletedProcess` object `cp` whose `stdout` is a string (because `capture_output=True` and `text=True`). The function then returns `cp.stdout.strip()`, a string with leading/trailing whitespace removed.

Formally:

```
Let `cmd` = ['git', '-C', proj_dir] + list(args).
Let `conflict` = {'check', 'capture_output', 'text'} ∩ keys(kwargs).
Then:
- If `conflict` ≠ ∅: the invocation raises `TypeError`.
- If `conflict` = ∅:
 - If `subprocess.run(cmd, check=True, capture_output=True, text=True, **kwargs)` raises exception `E` (where `E` may be `CalledProcessError` or any other), then _git raises `E`.
 - Otherwise, let cp be the returned CompletedProcess (cp must have cp.returncode == 0 because check=True would have raised otherwise). Then _git returns cp.stdout.strip().
```

- Code evidence:

```
Line 2: return subprocess.run(
Line 3: ["git", "-C", proj_dir, *args],
Line 4: check=True, capture_output=True, text=True, **kwargs,
Line 5:).stdout.strip()
```

- Trigger condition:

The code hardcodes `check=True`, `capture_output=True`, `text=True` but also passes `**kwargs` to `subprocess.run`. If `kwargs` contains any of `'check'`, `'capture_output'`, or `'text'`, the call to `subprocess.run` raises `TypeError` due to duplicate keyword argument, violating the specification which requires executing the git command for any valid positional args and env keyword argument. For instance, `_git('status', capture_output=True)` raises `TypeError`, whereas spec demands it to capture output and return stripped stdout or raise `CalledProcessError` on failure.

## Bug validator

- Trigger summary: `_git()` hardcodes `check=True`, `capture_output=True`, `text=True` but passes `**kwargs` to `subprocess.run`; passing any of these keys in `kwargs` raises `TypeError` due to duplicate keyword argument.
- Probe stdout:

```
CONFIRMED — TypeError raised with duplicate 'capture_output': <MagicMock name='run'
id='136370205191552'> got multiple values for keyword argument 'capture_output'
```

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_extracted_files_by_method.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_extracted_files_by_method.json`。
- 详细报告: `src--incremental_reasoner-py--_extracted_files_by_method.md`。
- Probe: `probe_src--incremental_reasoner-py--_extracted_files_by_method.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: fm_agent/extracted_functions/src/incremental_reasoner-py/_extracted_files_by_method.py

_extracted_files_by_method(func_dir)

Pre-condition:
- func_dir is a filesystem path (which may or may not be an existing directory)

Post-condition:
- Returns a mutable dict-like mapping from string keys (function names) to lists of
 absolute filesystem paths, each list containing one or more entries
- When func_dir is not an existing directory, returns an empty mapping (no keys present)
- When func_dir is an existing directory, every regular file reachable from func_dir
 by recursive descent is indexed under one or two keys, using the file's basename
 with its final dot-separated extension removed as the base identifier (the "stem"):
 - The absolute path of the file is always appended to the list for the key equal to
 the full stem
 - Additionally, when the stem contains at least one "::" delimiter, the substring
 after the last "::" (the bare method name) is used as a second key, and the same
 absolute path is appended to the list for that key as well
 - When the stem contains no "::" delimiter, only the stem itself is used as a key
- The order of absolute paths within each key's list reflects the order in which the
 corresponding files were encountered during traversal
- Accessing a key not present in the mapping returns an empty list (rather than raising
 an error), and mutating the returned list does not affect the mapping
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns a mutable dict-like mapping from string keys (function names) to lists of
 absolute filesystem paths, each list containing one or more entries
- When func_dir is not an existing directory, returns an empty mapping (no keys present)
- When func_dir is an existing directory, every regular file reachable from func_dir
 by recursive descent is indexed under one or two keys, using the file's basename
 with its final dot-separated extension removed as the base identifier (the "stem"):
 - The absolute path of the file is always appended to the list for the key equal to
 the full stem
 - Additionally, when the stem contains at least one "::" delimiter, the substring
 after the last "::" (the bare method name) is used as a second key, and the same
 absolute path is appended to the list for that key as well
 - When the stem contains no "::" delimiter, only the stem itself is used as a key
- The order of absolute paths within each key's list reflects the order in which the
 corresponding files were encountered during traversal
- Accessing a key not present in the mapping returns an empty list (rather than raising
 an error), and mutating the returned list does not affect the mapping
```

- 推导 actual behavior:

```
If func_dir is not an existing directory, the function returns an empty defaultdict(list)
(len(result) == 0 or equivalently for all keys k, result[k] == []). If func_dir is an existing
directory and no filesystem errors occur during os.walk, the function returns a
defaultdict(list) result where keys are stems or bare names extracted from the file names found,
and for each file fn in the recursive walk, its absolute path p is appended to result[stem] and,
if stem contains '::', also to result[bare] (where bare = stem.split('::')[-1] and bare !=
stem). Formally, let F be the multiset of (absolute_path, stem, bare) tuples obtained from
os.walk(func_dir) where stem = fn[:fn.rfind('.')] if '.' in fn else fn, and bare =
stem.split('::')[-1] if '::' in stem else None. Then result is a defaultdict(list) such that:
```

(1) for all (p, s, b) in F, p is an element of result[s] and, if b is not None and b != s, p is an element of result[b]; (2) for any key k, result[k] contains only such paths and in the order they were added by the walk; (3) result[k] == [] for all keys k not appearing in any (stem, bare) from F. If os.walk raises an exception (e.g., OSError, PermissionError), that exception is propagated and the function does not return normally.

- Code evidence:

```
Line 11: index = defaultdict(list)
Line 27: return index
```

- Trigger condition:

The function returns a collections.defaultdict(list), which automatically stores a new list for any missing key when \_\_getitem\_\_ is called. This means that if a caller accesses a missing key and mutates the returned list (e.g., appends an element), the mapping is permanently modified with that new key and element, contradicting the requirement 'mutating the returned list does not affect the mapping'. To satisfy the specification, the mapping would need to return a copy or a readonly view for missing keys, or avoid using a sideeffecting default factory.

#### Bug validator

- Trigger summary: Accessing a missing key on the returned defaultdict(list) and mutating the returned list permanently adds the key+value to the mapping, violating the spec that mutations to the returned list must not affect the mapping.
- Probe stdout:

```
BUG CONFIRMED (Test 1): accessing missing key 'nonexistent_key' and mutating returned list
modified the mapping. Initial key count: 0, Final key count: 1. Spec requires: mutating returned
list does NOT affect the mapping.
BUG CONFIRMED (Test 2): accessing missing key 'NonExistentFunction' and mutating returned list
modified the mapping with existing files. Keys before: ['Bar', 'Foo'], Keys after: ['Bar',
'Foo', 'NonExistentFunction']. Spec requires: mutating returned list does NOT affect the
mapping.
CONFIRMED
```

---

#### INCR-MISMATCH-022 — `src--incremental_reasoner-py--_reconcile_extracted_dir`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_reconcile_extracted_dir.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_reconcile_extracted_dir.json`。
- 详细报告: `src--incremental_reasoner-py--_reconcile_extracted_dir.md`。
- Probe: `probe_src--incremental_reasoner-py--_reconcile_extracted_dir.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/incremental_reasoner-py/_reconcile_extracted_dir.py

_reconcile_extracted_dir(proj_dir, abs_src) -> None

Pre-condition:
- proj_dir is an absolute path to the project root directory
- abs_src is an absolute path to a source file within proj_dir

Post-condition:
- Let (func_dir, ext) be the pair that maps abs_src to the extracted-functions
 directory and source extension via the same naming convention used by
 run_extraction. If func_dir is not an existing directory on disk, no
 filesystem changes occur and the function returns.
- Otherwise, the set of expected extracted-function files for abs_src is
```



determined.

- \* When `abs_src` exists on disk and its file extension maps to a language recognized by the project's language registry, the expected files are derived from the current function spans of `abs_src`. Each span's deduplicated identifier forms an expected filename: the identifier suffixed with `".<ext>"` when `ext` is non-empty, or the bare identifier when `ext` is empty. The span boundaries are computed with the same backend (codegraph when it indexes the file, otherwise regex) that `run_extraction` uses.
- \* When `abs_src` does not exist on disk, or when its extension is not recognized, the set of expected files is empty.
- Every file reachable by recursively walking `func_dir` whose absolute path does not match an expected file path is deleted. Expected files are preserved with their contents unchanged.
- After file deletion, every subdirectory of `func_dir` – excluding `func_dir` itself – that contains neither files nor subdirectories is removed.

[SPEC]

## Reasoner 差异

- SPEC claim:

- Let `(func_dir, ext)` be the pair that maps `abs_src` to the extracted-functions directory and source extension via the same naming convention used by `run_extraction`. If `func_dir` is not an existing directory on disk, no filesystem changes occur and the function returns.
- Otherwise, the set of expected extracted-function files for `abs_src` is determined:
  - \* When `abs_src` exists on disk and its file extension maps to a language recognized by the project's language registry, the expected files are derived from the current function spans of `abs_src`. Each span's deduplicated identifier forms an expected filename: the identifier suffixed with `".<ext>"` when `ext` is non-empty, or the bare identifier when `ext` is empty. The span boundaries are computed with the same backend (codegraph when it indexes the file, otherwise regex) that `run_extraction` uses.
  - \* When `abs_src` does not exist on disk, or when its extension is not recognized, the set of expected files is empty.
- Every file reachable by recursively walking `func_dir` whose absolute path does not match an expected file path is deleted. Expected files are preserved with their contents unchanged.
- After file deletion, every subdirectory of `func_dir` excluding `func_dir` itself that contains neither files nor subdirectories is removed.

- 推导 actual behavior:

If an exception is raised during ``os.remove`` or ``os.rmdir``, the function terminates with that exception; the filesystem is left partially modified (some deletions performed up to the point of failure). Under normal termination (no exception), the following holds: Let `(fd, ext) = _src_rel_to_func_dir(proj_dir, abs_src)`. If `fd` is not a directory before the call, the filesystem is unchanged. Otherwise, let `lang = EXT_TO_LANG.get(ext)`; let `Valid = if lang is None or abs_src is not a file in the prestate, else { os.path.abspath(os.path.join(fd, ident) + ('.' + ext if ext else '')) | ident { name | (name, _, _) _function_spans(abs_src, lang, proj_dir) } }`. Define `Keep = { p | pre.file(p) p Valid }`. After the call, for any path `p`: (files) if `p` is under `fd` then `post.file(p) p Keep`; else `post.file(p) pre.file(p)`. (directories) if `p` is under `fd` and `p != fd` then `post.dir(p) (pre.dir(p) q Keep such that p is a proper ancestor of q)`; if `p = fd` then `post.dir(p)` holds; if `p` is not under `fd` then `post.dir(p) pre.dir(p)`. (No empty subdirectory under `fd` except possibly `fd` itself remains; directories outside `fd` are untouched.)

- Code evidence:

Line 26: `os.remove(abs_path)`

- Trigger condition:

Specification B requires that every non-expected file under `func_dir` is deleted. When `os.remove` raises an exception (e.g., `PermissionError`), the function terminates immediately, leaving the non-expected file `'stale.py'` on disk. This violates the specification because the required deletion did not complete.

- Trigger summary: `os.remove()` raises `PermissionError` on a file in a non-writable subdirectory, terminating the function immediately and leaving the filesystem partially modified — one orphaned file deleted, another still present.
- Probe stdout:

CONFIRMED — `PermissionError` raised; `stale_undeletable.txt` remains on disk while `stale_deletable.txt` was already removed. Filesystem left in partial state (deletion incomplete).

## INCR-MISMATCH-023 — `src--incremental_reasoner-py--_remove_stale_extracted`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_remove_stale_extracted.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_remove_stale_extracted.json`。
- 详细报告: `src--incremental_reasoner-py--_remove_stale_extracted.md`。
- Probe: `probe_src--incremental_reasoner-py--_remove_stale_extracted.py`。

### 完整生成 SPEC

```
[SPEC]
_remove_stale_extracted(proj_dir, modified_functions) -> None

Pre-condition:
- proj_dir is an absolute path to the project root directory, under which the child directory
fm_agent/ exists with sub-directory extracted_functions/ and a phases.json file.
- modified_functions is a dict whose keys are absolute source-file paths (the values are not
used by this function).

Post-condition:
- For every absolute source-file path that is either a key in modified_functions or listed in
the "source_files" entries of all phases loaded from phases.json, the extracted-function tree
under fm_agent/extracted_functions/ associated with that source file is reconciled with the
current codegraph output. Any extracted function file or directory that no longer corresponds to
a current source function (including when the source file itself is absent) is deleted, and any
empty parent directories are pruned.
- Files and directories under fm_agent/extracted_functions/ that correspond to source files
not in the union of modified_functions keys and phases.json entries are unchanged.
[SPEC]
```

### Reasoner 差异

- SPEC claim:

```
- For every absolute source-file path that is either a key in modified_functions or listed in
the "source_files" entries of all phases loaded from phases.json, the extracted-function tree
under fm_agent/extracted_functions/ associated with that source file is reconciled with the
current codegraph output. Any extracted function file or directory that no longer corresponds to
a current source function (including when the source file itself is absent) is deleted, and any
empty parent directories are pruned.
- Files and directories under fm_agent/extracted_functions/ that correspond to source files
not in the union of modified_functions keys and phases.json entries are unchanged.
```

- 推导 actual behavior:

After execution, the extracted-functions subdirectory tree under `fm_agent/` has been updated such that for every source file path that belongs to the union `S = modified_functions.keys()` (if loading `phases.json` succeeded) the set of absolute paths derived from all `source_files` entries in modules within phases of `phases.json`, the extracted directory corresponding to that source file (if it exists) contains only files that are currently expected according to the function spans of that source file, any stale extracted files have been deleted, and empty subdirectories

have been pruned. For any source file path not in S, no changes have been made to its extracted directory. No other side effects occur. Formal logic: Let  $M = \text{dom}(\text{modified\_functions})$ . Let  $P = \{ \text{if an } \text{OSError}, \text{ ValueError}, \text{ or } \text{KeyError} \text{ was raised during loading of phases.json; otherwise } P = \{ \text{os.path.abspath(os.path.join(proj\_dir, rel))} \mid \text{phase phases\_data['phases']}, \text{ module phase['modules']}, \text{ rel module['source\_files']} \} \}$ . Then  $S = M \setminus P$ . For each  $\text{abs\_src} \in S$ , the effect of `_reconcile_extracted_dir(proj_dir, abs_src)` has been applied, meaning: the extracted directory for  $\text{abs\_src}$  (if non-existent) was left untouched; otherwise, its contents now equal the set of expected extracted file paths derived from the current source, and all stale files/directories have been removed. For  $\text{abs\_src} \in S$ , its extracted directory (if any) is unchanged. The function returns None.

- Code evidence:

```
Line 23: for abs_src in srcs:
Line 24: _reconcile_extracted_dir(proj_dir, abs_src)
```

- Trigger condition:

The code only calls `_reconcile_extracted_dir` for each individual source file. This may leave empty parent directories (like `a/`) that are shared by multiple source-specific directories when all children are removed. The specification explicitly requires pruning any empty parent directories, which the code does not guarantee.

#### Bug validator

- Trigger summary: `_remove_stale_extracted` calls `_reconcile_extracted_dir` per source file, which removes stale files but never removes the `func_dir` itself or prunes empty parent directories above it; when multiple deleted source files share a common parent, empty directories are left behind.
- Probe stdout:

CONFIRMED – empty func directories `a-py` and `b-py` left behind; parent `pkg/` not pruned. Spec requires pruning all empty parent directories.

---

#### INCR-MISMATCH-024 — `src--incremental_reasoner-py--_update_specs_for_intent`

- 人工审计: 推理误判。
- Validator: **not\_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_update_specs_for_intent.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_update_specs_for_intent.json`。
- 详细报告: `src--incremental_reasoner-py--_update_specs_for_intent.md`。
- Probe: `probe_src--incremental_reasoner-py--_update_specs_for_intent.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/incremental_reasoner.py

_update_specs_for_intent(proj_dir, work_dir, developer_intent, changed_functions,
relevant_rel_files, extra_call_edges=None) -> list[str]

Pre-condition:
- proj_dir is an existing directory path
- work_dir is a directory path containing an extracted_functions/ subdirectory with
 extracted function files and a spec_prompts/ subdirectory with domain context and
 batch-prompt metadata
- developer_intent is a non-empty string describing the developer's modification goal
- changed_functions is a dict mapping absolute source-file paths to dicts each containing
 at least the keys "added" and "modified", whose values are lists of function names
 (as they appear in the source)
- relevant_rel_files is a list of strings, each a relative path from the
 extracted_functions/ directory identifying an extracted-function file to seed from
- extra_call_edges, when not None, supplies supplemental caller-to-callee edges beyond
```

those discoverable by static analysis

Post-condition:

- Returns a list of extracted-function relative paths (relative to the extracted\_functions/ directory) whose [SPEC] block, [INFO] block, or both were

## Reasoner 差异

- SPEC claim:

- Returns a list of extracted-function relative paths (relative to the extracted\_functions/ directory) whose [SPEC] block, [INFO] block, or both were created or modified by this call; the list is sorted lexicographically
- Returns an empty list when the combined seed set all function names from the "added" and "modified" classifications in changed\_functions, plus all FQNs derived from relevant\_rel\_files is empty
- For every function whose FQN belongs to the seed set:
  - \* If the function's extracted file has no pre-existing [SPEC] block, a new behavioral [SPEC] block (and optionally an [INFO] callee-expectations block) is generated by an external process informed by the developer intent and caller context, then prepended to the file; the original function source code following the block is preserved unchanged
  - \* If a pre-existing [SPEC] block exists, it is re-evaluated against developer\_intent by an external process; if the evaluation determines the spec must change, the [SPEC] block is replaced (and the [INFO] block may optionally be replaced if it too must change); the source code is preserved identical to its input form
- Functions are processed in caller-before-callee topological rounds: within each round, only functions whose callers (per the call graph) are either already processed or absent from the current pending set are eligible; a function is deferred to a subsequent round if any of its callers remains pending
- When a function's [SPEC] is newly generated or updated:
  - \* Every caller of that function has its [INFO] entry for the function reconciled against the new [SPEC] by an external process; the caller file is overwritten only if the reconciliation produces an [INFO] block that differs from the existing one
  - \* Callees listed in the function's updated [INFO] block that match previously unchecked FQNs are added to the pending frontier and will be spec-checked in subsequent rounds
- Under cycle conditions where every pending function has at least one also-pending caller, exactly one pending function is selected to break the cycle and guarantee forward progress
- Every written file has its source-code portion (everything after the leading [SPEC] and [INFO] comment blocks) byte-for-byte identical to the source code that was read from that file before modification
- No file outside of work\_dir/extracted\_functions/ is created, deleted, or modified

- 推导 actual behavior:

If an exception is raised by any function called in lines 41-80 (e.g., \_file\_to\_fqn, \_topdown\_ordered\_fqns), the function terminates with that exception and, because no file writes occur in this block, the contents of all files under work\_dir/extracted\_functions/ remain identical to their contents at the start of line 41.

Otherwise (normal flow), let preSeed be the set of FQNs already in `seed` just before line 41, and let preFiles be the file state at that moment. Define added = { \_file\_to\_fqn(os.path.join(extracted\_dir, rel), work\_dir) | rel relevant\_rel\_files }. After line 80, two cases arise:

1. If preSeed added is empty, the function immediately returns []. The extracted function files are unchanged: p AFiles, Files(p) = preFiles(p). The function terminates normally.
2. If preSeed added is nonempty, execution continues past line 80. The variable `seed` holds preSeed added, `topdown` and `order\_index` are computed as by the respective calls, and the nested function `\_plan\_spec\_update` is defined but not called. No file writes have been performed, so p AFiles, Files(p) = preFiles(p). The function has not yet returned.

Formally, using the notation from the given precondition:

Let OldEF\_41 be the restriction of Files to paths under extracted\_dir at the start of line 41. Let seed\_41 be the value of the variable seed at that point. Let AddFQN = { \_file\_to\_fqn(os.path.join(extracted\_dir, rel), work\_dir) | rel relevant\_rel\_files }.

After the execution of lines 41-80 (no exception):

```
(seed_41 AddFQN =)
 result = [] fqn FQNs . NewEF_41(FQNMap[fqn]) = OldEF_41(FQNMap[fqn]) the function
returns.
(seed_41 AddFQN)
 seed = seed_41 AddFQN topdown = _topdown_ordered_fqns(work_dir,
```

```
extra_call_edges=extra_call_edges)
 order_index = { fqn i | (i, fqn) enumerate(topdown) }
 _plan_spec_update is defined fqn FQNs . NewEF_41(FQNMap[fqn]) =
OldEF_41(FQNMap[fqn]) the function has not returned.

If any called auxiliary function raises an exception E, the function terminates with E and fqn
FQNs . NewEF_41(FQNMap[fqn]) = OldEF_41(FQNMap[fqn]).
```

- Code evidence:

```
Line 41: seed.add(_file_to_fqn(os.path.join(extracted_dir, rel), work_dir))
```

- Trigger condition:

The seed set is only populated from `relevant\_rel\_files`; the `changed\_targets` (functions from 'added' and 'modified' classifications) are never added to the seed. Consequently, when `relevant\_rel\_files` is empty but `changed\_targets` is nonempty the code incorrectly returns an empty list, violating the requirement that all changed functions must be processed.

#### Bug validator

- Trigger summary: The seed set is populated from BOTH changed\_targets (via seed.update) and relevant\_rel\_files; the logic verifier missed the seed.update(changed\_targets.keys()) call.
- Probe stdout:

NOT CONFIRMED – seed.update(changed\_targets.keys()) found in \_update\_specs\_for\_intent; changed\_targets are added to the seed alongside relevant\_rel\_files.

---

#### INCR-MISMATCH-025 — `src--incremental_reasoner-py--collect_relevant_function_scope`

- 人工审计: 推理误判。
- Validator: **not confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/incremental_reasoner-py/collect_relevant_function_scope.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/collect_relevant_function_scope.json`。
- 详细报告: `src--incremental_reasoner-py--collect_relevant_function_scope.md`。
- Probe: `probe_src--incremental_reasoner-py--collect_relevant_function_scope.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: fm_agent/extracted_functions/src/incremental_reasoner-
py/collect_relevant_function_scope.py

collect_relevant_function_scope(proj_dir, developer_intent, changed_functions, range=None) ->
list[str]

Pre-condition:
- proj_dir is a path to a project directory whose fm_agent/ subdirectory contains phases.json
 (with a "phases" list of phase objects, each containing a "modules" list) and
 extracted_functions/
- developer_intent is a non-empty string describing the modification goal
- changed_functions is a dict mapping absolute source file paths to dicts with string-list
 values
 under at least the keys "added", "modified", and "removed"
- range is None or a non-negative integer

Post-condition:
- Returns a list of paths, each relative to the extracted_functions/ directory, ordered by
```

descending relevance to developer\_intent; paths with equal relevance are ordered lexicographically

- Every returned path refers to an existing regular file under extracted\_functions/
- When range is not None, the returned list has length  $\leq$  range
- Returns an empty list when phases.json defines no modules, or when no module is selected by the relevance assessment
- A module is selected when EITHER its natural-language description (as recorded in phases.json)
  - is assessed as relevant to the developer intent, OR the module contains at least one source file
    - whose path, relativized against proj\_dir, matches a key in changed\_functions
- Within each selected module, a source file is included only when its content is assessed as relevant to the developer intent, EXCEPT that every source file present in changed\_functions is included unconditionally
- When the per-module file-relevance assessment cannot be obtained, every source file in that module is included
- Within each included source file, the set of extracted functions whose relevance scores (computed from heuristic signals derived from developer\_intent) rank within the top of that file
  - are included
- When per-file function ranking is unavailable for an included source file, every extracted function from that file is included
- Multiple extracted-function files mapping to the same source-level function are deduplicated,
  - keeping only the occurrence with the highest relevance score

[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns a list of paths, each relative to the extracted\_functions/ directory, ordered by descending relevance to developer\_intent; paths with equal relevance are ordered lexicographically
- Every returned path refers to an existing regular file under extracted\_functions/
- When range is not None, the returned list has length range
- Returns an empty list when phases.json defines no modules, or when no module is selected by the relevance assessment
- A module is selected when EITHER its natural-language description (as recorded in phases.json)
  - is assessed as relevant to the developer intent, OR the module contains at least one source file
    - whose path, relativized against proj\_dir, matches a key in changed\_functions
- Within each selected module, a source file is included only when its content is assessed as relevant to the developer intent, EXCEPT that every source file present in changed\_functions is included unconditionally
- When the per-module file-relevance assessment cannot be obtained, every source file in that module is included
- Within each included source file, the set of extracted functions whose relevance scores (computed from heuristic signals derived from developer\_intent) rank within the top of that file
  - are included
- When per-file function ranking is unavailable for an included source file, every extracted function from that file is included
- Multiple extracted-function files mapping to the same source-level function are deduplicated,
  - keeping only the occurrence with the highest relevance score

- 推导 actual behavior:

Let L be the list returned by collect\_relevant\_function\_scope. The function does not modify any of its arguments or the filesystem (it only reads files and writes logs). Under the given preconditions (proj\_dir contains fm\_agent/phases.json and fm\_agent/extracted\_functions, developer\_intent is nonempty, changed\_functions has the required structure, range is None or a nonnegative integer) the following holds:

If the flattened module list derived from phases.json is empty (no modules in any phase), L = [].

Otherwise, the function performs three passes:

1. Module selection: an LLM chooses a subset of modules relevant to developer\_intent.
2. File selection: for each chosen module, opencode selects a subset of its source files.
3. Function selection: for each selected file, rank\_functions\_in\_file ranks the functions and returns a list of dicts ordered by descending score. Through \_extracted\_files\_by\_method each relevant function is mapped to one or more extractedfunction file paths relative to proj\_dir/fm\_agent/extracted\_functions. The union of these paths, ordered by the original descending scores (preserving perfile order and combining files by descending modulethenfile

relevance), forms a sequence S.

If S is empty (no modules, files or functions were selected), L = [].

If range is None, L = S.

If range is a nonnegative integer, L = S[:range] (the first at most range paths).

Thus: L is a list of strings, each string is a relative path into the extracted\_functions directory, and L is sorted by decreasing relevance to developer\_intent. The list is empty exactly when the selection process produces no results.

- Code evidence:

```
Line 1: def collect_relevant_function_scope(proj_dir, developer_intent, changed_functions,
range=None):
Line 40: # Pass 1: module selection. The module descriptions are already parsed from
phases.json
```

- Trigger condition:

Specification requires that a module is selected when it contains at least one source file whose relative path matches a key in changed\_functions, even if its description is not assessed as relevant. The code performs only an LLM-based relevance assessment on module descriptions and does not incorporate the changed\_functions criterion. In the counterexample, the module with an irrelevant description contains a file present in changed\_functions, so it must be selected, but the code omits it, yielding an empty list instead of the required nonempty result.

#### Bug validator

- Trigger summary: Bug claim asserts changed\_functions criterion is not incorporated; code inspection at lines 1158-1162 shows the or any() clause IS present, and the probe confirms it selects modules with changed files even when LLM assessment returns empty.
- Probe stdout:

```
NOT CONFIRMED - changed_functions criterion IS incorporated: selected modules=['util_module'],
expected=['util_module']
```

---

#### INCR-MISMATCH-026 — `src--incremental_reasoner-py--run_incremental_pipeline`

- 人工审计: 推理误判。
- Validator: **not\_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/incremental_reasoner-py/run_incremental_pipeline.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/run_incremental_pipeline.json`。
- 详细报告: `src--incremental_reasoner-py--run_incremental_pipeline.md`。
- Probe: `probe_src--incremental_reasoner-py--run_incremental_pipeline.py`。

#### 完整生成 SPEC

[SPEC]

Unit: `src/incremental_reasoner-py/run_incremental_pipeline.py`

```
run_incremental_pipeline(proj_dir, intent_file_path, old_commit_id,
 domain_knowledge_files=None, submodules=None,
 one_phase=False, extra_call_edges_path=None)
-> list[str] | None
```

Pre-condition:

- proj\_dir is a path to an existing directory containing source code under version control; fm\_agent/ is a writable subdirectory within it.
- intent\_file\_path is a string; when non-empty and pointing to a regular file, its content describes the developer's modification goal.
- old\_commit\_id is a string identifying a prior commit in proj\_dir's git repository.



- `domain_knowledge_files`, when not `None`, is a list of paths to Markdown files providing project-specific domain context.
- `submodules`, when not `None`, is a list of subdirectory paths within `proj_dir` to scope analysis to.
- `one_phase` is a `bool` (default `False`) controlling whether all source files are placed into a single phase.
- `extra_call_edges_path`, when not `None`, is a path to a JSON file defining supplemental call-graph edges.

Post-condition:

- If `proj_dir` has no previous full-run baseline (`fm_agent/phases.json` absent or `fm_agent/extracted_functions/` incomplete given submodules), delegates the entire pipeline to a full run via `run_pipeline()` with the same arguments and returns `None`.
- If `intent_file_path` does not refer to an existing regular file, or the file content is empty after whitespace stripping, logs an error and returns `None` without modifying any project or `fm_agent/` file.
- Before producing any new output, removes all files under `fm_agent/logic_verification_results/` and `fm_agent/bug_validation/`, and removes incremental scope-selection and spec-update artifacts prefixed with `"select_relevant_"`, `"relevant_"`, and `"spec_update_"` from `fm_agent/`.
- Regenerates `fm_agent/phases.json` from the current working tree.
- Re-extracts every function from the current code, then restores the captured [SPEC] and [INFO] blocks from the prior run onto each function

## Reasoner 差异

- SPEC claim:

- If `proj_dir` has no previous full-run baseline (`fm_agent/phases.json` absent or `fm_agent/extracted_functions/` incomplete given submodules), delegates the entire pipeline to a full run via `run_pipeline()` with the same arguments and returns `None`.
- If `intent_file_path` does not refer to an existing regular file, or the file content is empty after whitespace stripping, logs an error and returns `None` without modifying any project or `fm_agent/` file.
- Before producing any new output, removes all files under `fm_agent/logic_verification_results/` and `fm_agent/bug_validation/`, and removes incremental scope-selection and spec-update artifacts prefixed with `"select_relevant_"`, `"relevant_"`, and `"spec_update_"` from `fm_agent/`.
- Regenerates `fm_agent/phases.json` from the current working tree.
- Re-extracts every function from the current code, then restores the captured [SPEC] and [INFO] blocks from the prior run onto each function whose body is identical between `old_commit_id` and the current working tree.
- Produces a mapping from each changed source-file path to the sets of function names added, modified, or removed since `old_commit_id`; deletes extracted-function files for removed functions.
- Produces a ranked list of extracted-function relative paths whose implementations are judged relevant to the developer intent.
- For every function that is either changed (added or modified) or appears in the relevance-ranked list, re-evaluates whether its [SPEC] and/or [INFO] blocks need updating to reflect the current code and intent; when a callee's [SPEC] changes, propagates the update to every caller's [INFO] block. Writes the set of files whose specs were modified to `fm_agent/incremental_updated_specs.json`.
- Runs verification on the affected subset: every changed function, every function with an updated spec, and every function that calls a callee whose spec was updated. Returns a sorted list of extracted-function relative paths for which the reasoner reported a spec-to-code mismatch (MISMATCH verdict) and bug validation subsequently confirmed the violation. Returns an empty list when no such violations are confirmed.
- Does not modify any file under `proj_dir` outside of `fm_agent/`.

- 推导 actual behavior:

```
Normal termination paths:
- If `check_last_run_existence(proj_dir, submodules)` returns `False`: the function emits a warning log message ("No previous full run detected..."), invokes `run_pipeline(proj_dir, domain_knowledge_files=..., submodules=..., ...)` (which performs the full pipeline and produces artifacts under `fm_agent/`), and then returns `None` immediately.
- If `check_last_run_existence` returns `True` but the intent file at `intent_file_path` does not exist or is empty after stripping: the function logs an error ("Intent file ... does not exist" or "is empty") and returns `None` immediately.
- Otherwise (last run exists and intent file is valid): the function logs that a previous run was found, logs "[Stage 2/10] Loading developer intent...", reads and binds the nonempty developer intent to `developer_intent`, logs "intent loaded (%d chars)", then removes the stale
```



directories ``output_dir`` and ``<work_dir>/bug_validation`` if they exist (logging one line per removed directory). Execution continues normally beyond line 80 with ``developer_intent`` set and the stale artifacts removed; no value is returned yet.

Additional logging side effects that are **\*\*always\*\*** performed by the code block (unless an exception prevented reaching them):

- A separator line of 70 ``='`` characters is emitted (line 41).
- The message "[Stage 1/10] Checking for a previous full run to compare against..." is emitted.
- In the ``True`` branch of ``check_last_run_existence``: " -> previous full run found; proceeding with incremental analysis." is emitted.

**\*\*Exception paths:\*\***

- If ``check_last_run_existence`` raises an exception, it propagates to the caller immediately; any log messages emitted up to that point (the separator and the "[Stage 1/10] ..." message) persist.
- If the fileopen or read on the intent file raises an exception, it propagates; the "[Stage 2/10] ..." log and the preceding stage1 logs remain.
- If ``run_pipeline`` raises, the exception propagates and all side effects performed by ``run_pipeline`` up to the failure point, together with the preceding logs and the ``run_pipeline`` call itself, are persisted; the function does not reach the return.
- Other operations (``shutil.rmtree``, ``os.path.isdir``, ``logging.info``, etc.) are assumed not to raise under the given preconditions.

**\*\*Formal logic:\*\***

Let  $\backslash( \text{PRE} \backslash )$  denote the precondition state just before line 41, which includes:

- ``work_dir``, ``input_dir``, ``output_dir``, ``extra_call_edges``, ``staged_knowledge`` are bound as previously described.
- Incremental logging is configured.
- The following log records have already been emitted (due to earlier code): if ``staged_knowledge`` is truthy, a line with the number of Markdown files; a separator line of 70 ``='`` characters; "INCREMENTAL PIPELINE START"; project directory; intent file path; base commit ID; and optionally submodule scope.
- No exception has occurred so far.

Let  $\backslash( \text{POST} \backslash )$  be the state after the block (lines 41-80) finishes.  $\backslash( \text{POST} \backslash )$  satisfies:

```
\[
\begin{aligned}
& \& \text{PRE} \& \backslash \text{and} \backslash \backslash \\
& \& (\& \text{\text{normal_exit}} \& \backslash \text{implies} \backslash \backslash \\
& \& \& \text{\text{exists}} \backslash, \& \text{\text{separator_log}}, \& \text{\text{stage1_log}}, \& \text{\text{result_log}}, \& \text{\text{etc.}} \& \backslash \\
& \& \text{\text{emitted by this block}} \& \backslash \backslash \\
& \& \& \text{\text{and}} \backslash \& ; (\& \text{\text{check_path}} \& = \& \text{\text{false}} \& \backslash \text{implies} \backslash \backslash \\
& \& \& \text{\text{run_pipeline_called}} \& (\text{proj_dir}, \& \text{\text{dots}}) \& \text{\text{and}} \& \text{\text{return_value}} \& = \& \text{\text{None}} \& \backslash \backslash \\
& \& \& \text{\text{full_pipeline_side_effects}} \& (\text{proj_dir}) \& \backslash \backslash \\
& \& \& \text{\text{log_record_exists}} \& (\text{\text{warning, "No previous full run..."}}) \& \backslash \backslash \\
& \& \& \text{\text{and}} \backslash \& \& ; (\& \text{\text{check_path}} \& = \& \text{\text{true}} \& \backslash \text{implies} \backslash \backslash \\
& \& \& \text{\text{intent_file_invalid}} \& \backslash \text{implies} \backslash \backslash \\
& \& \& \text{\text{log_record_exists}} \& (\text{\text{error, "Intent file ..."}}) \& \text{\text{and}} \& \text{\text{return_value}} \& = \& \text{\text{None}} \& \backslash \backslash \\
& \& \& \text{\text{and}} \backslash \& \& ; (\& \text{\text{intent_file_valid}} \& \backslash \text{implies} \backslash \backslash \\
& \& \& \text{\text{developer_intent}} \& = \& \text{\text{non_empty_content}} \& \backslash \backslash \\
& \& \& \text{\text{stale_dirs_removed}} \& (\text{output_dir}, \text{work_dir} \& \text{\text{bug_validation}}) \& \backslash \backslash \\
& \& \& \text{\text{no_return}} \backslash \& ; \& \text{\text{(execution continues)}} \& \backslash \backslash \\
& \& \& \text{\text{and}} \backslash \& \& ; (\& \text{\text{exception_exit}} \& \backslash \text{implies} \backslash \backslash \\
& \& \& \text{\text{propagated_exception}} \& \text{\text{and}} \& \text{\text{partial_effects_preserved_up_to_failure}} \& \backslash \backslash \\
& \& \& \text{\text{and}} \backslash \& \& ; \\
& \& \text{\text{end}} \& \text{\text{aligned}} \& \backslash \backslash \\
& \& \backslash \backslash
\end{aligned}
```

where:

- $\backslash( \& \text{\text{check\_path}} \& \backslash )$  is the return value of ``check_last_run_existence`` (``True``/``False``).
- $\backslash( \& \text{\text{intent\_file\_invalid}} \& \backslash )$  means the file does not exist or is empty after stripping.
- $\backslash( \& \text{\text{stale\_dirs\_removed}} \& \text{\text{ldots}} \backslash )$  means if a directory existed, it was recursively removed and a log line was emitted.

#### • Code evidence:

```
Line 77: for stale_dir in (output_dir, os.path.join(work_dir, "bug_validation")):
Line 78: if os.path.isdir(stale_dir):
Line 79: shutil.rmtree(stale_dir, ignore_errors=True)
Line 80: logging.info(" -> removed stale results dir %s.", stale_dir)
```

#### • Trigger condition:

The specification demands removal of incremental scope-selection and spec-update artifacts (files prefixed with 'select\_relevant\_', 'relevant\_', and 'spec\_update\_') before producing new output. The code only removes the two directories, omitting the required prefixbased file deletion, so a valid run where those files exist leaves the system in a state that violates the precondition for the subsequent incremental steps.

#### Bug validator

- Trigger summary: The code\_evidence only cited the directory-removal lines and missed the immediately-following artifact-removal code (lines 231-243) that deletes files matching select\_relevant\_, relevant\_, and spec\_update\_\* globs via glob.glob + os.remove — the spec is satisfied.
- Probe stdout:

NOT CONFIRMED – code includes required prefixed-file removal: all 6 glob patterns present in source

## src/languages/c-py

### INCR-MISMATCH-027 — src--languages--c-py--batch\_extract

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: src/languages/c-py/batch\_extract.py。
- Reasoner 结果: logic\_verification\_results/src/languages/c-py/batch\_extract.json。
- 详细报告: src--languages--c-py--batch\_extract.md。
- Probe: probe\_src--languages--c-py--batch\_extract.py。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/c-py/batch_extract.py

batch_extract(proj_dir) -> dict

Pre-condition:
- proj_dir is a path to an existing directory containing C source files

Post-condition:
- Returns a dictionary mapping absolute file paths to lists of
 (function_name, function_body) tuples for every C function extracted
 from the project using codegraph analysis
- Returns an empty dictionary {} when codegraph initialization fails
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- Returns a dictionary mapping absolute file paths to lists of
 (function_name, function_body) tuples for every C function extracted
 from the project using codegraph analysis
- Returns an empty dictionary {} when codegraph initialization fails
```

- 推导 actual behavior:

The function returns a dictionary. If `CodeGraphExtractor.from_proj_dir(proj_dir)` returns a valid extractor object `C` (i.e., not `None`), the result equals `C.get_functions_by_file('c', proj_dir)`: a dict mapping absolute file paths (strings) to lists of `(func_name, body)` tuples for all C files under `proj_dir` that could be successfully read; unreadable files are omitted. If initialization fails (`C` is `None`), the result is an empty dict `{}`. Formally: let `C = CodeGraphExtractor.from_proj_dir(proj_dir)`. Then `result = (C.get_functions_by_file('c', proj_dir) if C is not None else {})`.

- Code evidence:

```
Line 4: return cg.get_functions_by_file("c", proj_dir) if cg else {}
```

- Trigger condition:

The code uses the truthiness of `cg` (if `cg`) to check whether initialization succeeded, but the specification of `from_proj_dir` only guarantees it returns an initialized `CodeGraphExtractor` or `None`; it does not guarantee the returned object is truthy. An initialized extractor could evaluate to `False` (e.g., if it defines `__len__` to return `0`), causing the function to erroneously return `{}` and violate the requirement to return the extracted functions when initialization succeeds.

**Bug validator**

- Trigger summary: `batch_extract` uses truthiness check (if `cg`) instead of explicit `None` check, so a non-`None` but falsy `CodeGraphExtractor` incorrectly returns `{}` instead of extracted functions.
- Probe stdout:

```
CONFIRMED – actual: {} | expected: {'/fake/path.c': [('main', 'int main(void) {}\\n')]}
```

**INCR-MISMATCH-028 — `src--languages--c-py--function_spans`**

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/c-py/function_spans.py`。
- Reasoner 结果: `logic_verification_results/src/languages/c-py/function_spans.json`。
- 详细报告: `src--languages--c-py--function_spans.md`。
- Probe: `probe_src--languages--c-py--function_spans.py`。

**完整生成 SPEC**

```
[SPEC]
Unit: src/languages/c-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list[tuple[str, int, int]] | None

Pre-condition:
- proj_dir is a path to an existing project directory
- filepath is a path to a C source file with a ".c" extension

Post-condition:
- Returns None when a codegraph instance cannot be initialized from proj_dir
- Otherwise returns a list of (function_name, start_idx, end_idx) tuples for every
 function defined in the C source file at filepath, where start_idx and end_idx
 are 0-indexed inclusive line numbers
[SPEC]
```

**Reasoner 差异**

- SPEC claim:

- Returns None when a codegraph instance cannot be initialized from proj\_dir
- Otherwise returns a list of (function\_name, start\_idx, end\_idx) tuples for every function defined in the C source file at filepath, where start\_idx and end\_idx are 0-indexed inclusive line numbers

- 推导 actual behavior:

After execution, the function returns either `None` or a list of `(name, start\_idx, end\_idx)` tuples. Formally, let `cg = CodeGraphExtractor.from\_proj\_dir(proj\_dir)`. If `cg` is `None`, the return value is `None`. Otherwise, the return value is `cg.get\_function\_spans("c", filepath)`. Consequently, the overall return value is `None` if and only if the code graph is unavailable for `proj\_dir` or the call to `get\_function\_spans` returns `None` (e.g., language key not recognized, file not indexed, no definitions, or path resolution failure). If the return value is not `None`, it is a Python list `L` where each element is a tuple `(name: str, start\_idx: int, end\_idx: int)` such that  $0 \leq \text{start\_idx} \leq \text{end\_idx}$ , the tuples represent all function/method definitions detected in the C source file pointed to by `filepath`, `name` is a class-qualified identifier, and `L` is sorted by ascending `start\_idx`. No side effects are visible to the caller beyond the value returned.

- Code evidence:

```
Line 8: return cg.get_function_spans("c", filepath) if cg else None
```

- Trigger condition:

The specification states that None is returned only when a codegraph instance cannot be initialized from proj\_dir; otherwise a list (possibly empty) of function tuples must be returned. The code, however, also returns None when get\_function\_spans returns None for reasons other than initialization failure (e.g., the file has no definitions). This violates the specification because for valid proj\_dir and a C file with no functions, the code returns None instead of an empty list.

#### Bug validator

- Trigger summary: For a valid proj\_dir and a C file with no functions, function\_spans returns None instead of an empty list because get\_function\_spans returns None for files with no definitions and the code passes this through.
- Probe stdout:

```
CONFIRMED — actual: None | expected: []
```

## src/languages/codegraph-py

### INCR-MISMATCH-029 — src--languages--codegraph-py--

#### CodeGraphExtractor::get\_functions\_by\_file

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: src/languages/codegraph-py/CodeGraphExtractor::get\_functions\_by\_file.py。
- Reasoner 结果: logic\_verification\_results/src/languages/codegraph-py/CodeGraphExtractor::get\_functions\_by\_file.json。
- 详细报告: src--languages--codegraph-py--CodeGraphExtractor::get\_functions\_by\_file.md。
- Probe: probe\_src--languages--codegraph-py--CodeGraphExtractor::get\_functions\_by\_file.py。

```
[SPEC]
Unit: src/languages/codegraph.py

CodeGraphExtractor.get_functions_by_file(lang_key: str, proj_dir: str = None) -> dict

Pre-condition:
- lang_key is a string
- proj_dir is a string path to a directory, or None
- The receiver has an initialized codegraph database accessible for reading

Post-condition:
- Returns a dict whose keys are absolute filesystem paths (str) and whose
 values are lists of (str, str) tuples
- Each tuple consists of a function identifier and the full source text of
 the corresponding function body
- Each function body ends with a newline character ("\n")
- For a given file, tuples are ordered by ascending line number of the
 function definition within the source file
- Function identifiers are class-qualified; when multiple functions in the
 same file share the same identifier, the first occurrence retains the
 bare name and every subsequent occurrence appends a numeric suffix
 starting from 1
- When lang_key is not a recognized language identifier, the returned dict
 is empty
- Source files that cannot be opened for reading are omitted from the
 result; no error is raised
- When proj_dir is provided, file paths stored in the database are resolved
 relative to proj_dir to produce absolute keys

[SPEC]
```

## Reasoner 差异

- SPEC claim:

- Returns a dict whose keys are absolute filesystem paths (str) and whose values are lists of (str, str) tuples
- Each tuple consists of a function identifier and the full source text of the corresponding function body
- Each function body ends with a newline character ("\n")
- For a given file, tuples are ordered by ascending line number of the function definition within the source file
- Function identifiers are class-qualified; when multiple functions in the same file share the same identifier, the first occurrence retains the bare name and every subsequent occurrence appends a numeric suffix starting from 1
- When lang\_key is not a recognized language identifier, the returned dict is empty
- Source files that cannot be opened for reading are omitted from the result; no error is raised
- When proj\_dir is provided, file paths stored in the database are resolved relative to proj\_dir to produce absolute keys

- 推导 actual behavior:

The function returns a dictionary `result` such that: if `\_CG\_LANG.get(lang\_key)` is falsy (None or empty list), `result` is the empty dictionary. Otherwise, let `cg\_langs = \_CG\_LANG[lang\_key]`. The function opens a read-only connection to the database at `self.\_db`, queries the `nodes` table for rows where `kind` is 'function' or 'method' and `language` is in `cg\_langs`, ordered by `file\_path` then `start\_line`. Each row is a tuple `(name, qualified\_name, file\_path, start\_line, end\_line)`. Rows are grouped by `file\_path` preserving query order. For each distinct `file\_path`:

- Compute `abs\_path = os.path.join(proj\_dir, file\_path)` if `proj\_dir` is not None, else `file\_path`.
- If opening `abs\_path` for reading raises `OSError`, that file is skipped (no entry in `result`).
- Otherwise, read all lines from the file into `all\_lines`.
- For each function entry in that file, processed in order of `start\_line`:
  1. Compute `ident = \_extraction\_ident(name, qualified\_name)`, a deterministic filesystem-safe string.
  2. Count occurrences of `ident` within the file (starting at 0). The first occurrence keeps the bare `ident`; subsequent ones become `{ident}\_{count}` where `count` is the number of previous occurrences.
  3. Extract body lines: slice `all\_lines[start\_line-1 : end\_line]` (1-indexed, inclusive). Join

them into a single string. If the resulting string does not end with a newline character, append one.

4. Append the tuple `(deduped\_ident, body)` to a list for the file.

- `result[abs\_path]` is set to that list, preserving extraction order.

The function returns `result` after closing the database connection. No database modifications occur; the post-condition covers normal termination (no unhandled exceptions from the database or filesystem).

Formally:

```
...
Let cg_langs = _CG_LANG.get(lang_key).
If cg_langs is None or cg_langs == []:
 result = {}
Else:
 conn = sqlite3.connect(self._db); cur = conn.cursor()
 cur.execute(query, cg_langs) // query selects (name, qualified_name, file_path, start_line,
end_line)
 rows = cur.fetchall()
 conn.close()
 Partition rows into by_file: dict mapping file_path list of (name, qualified_name,
start_line, end_line) in order of appearance.
 result = {}
 For each file_path in keys(by_file) in the order they appear in rows:
 abs_path = os.path.join(proj_dir, file_path) if proj_dir else file_path
 If open(abs_path) raises OSError: continue
 else:
 all_lines = list of lines from file
 ident_counts = {}
 file_funcs = []
 For (name, qualified_name, start_line, end_line) in by_file[file_path]:
 ident = _extraction_ident(name, qualified_name)
 cnt = ident_counts.get(ident, 0)
 ident_counts[ident] = cnt + 1
 deduped_ident = ident if cnt == 0 else f"{ident}_{cnt}"
 body = "".join(all_lines[start_line-1 : end_line])
 if not body.endswith("\n"): body += "\n"
 file_funcs.append((deduped_ident, body))
 result[abs_path] = file_funcs
 return result
...
```

- Code evidence:

```
Line 32: abs_path = os.path.join(proj_dir, file_path) if proj_dir else file_path
```

- Trigger condition:

Specification B requires that when `proj_dir` is provided, the returned dict keys are absolute filesystem paths. The code uses `os.path.join(proj_dir, file_path)` but does not ensure the result is absolute; passing a relative `proj_dir` produces relative keys, violating the specification.

#### Bug validator

- Trigger summary: Passing a relative `proj_dir` to `get_functions_by_file` produces relative filesystem path keys in the returned dict instead of absolute paths as the specification requires.
- Probe stdout:

```
CONFIRMED — spec requires absolute paths as dict keys, but passing a relative
proj_dir='.././probe_cg_bq8naau0' produced relative key:
['.././probe_cg_bq8naau0/test_module.py'] instead of expected absolute key
'/tmp/probe_cg_bq8naau0/test_module.py'
```

---

#### INCR-MISMATCH-030 — `src--languages--codegraph-py--_bare_function_name`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/codegraph-py/_bare_function_name.py`。

- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_bare_function_name.json`。
- 详细报告: `src--languages--codegraph-py--_bare_function_name.md`。
- Probe: `probe_src--languages--codegraph-py--_bare_function_name.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/languages/codegraph.py

_bare_function_name(name: str) -> str

Pre-condition:
- name is a string that may be a raw function name, a scope-qualified name,
 a decorated function signature (function-pointer or pointer-return syntax),
 an operator overload name, or an empty string

Post-condition:
- Returns a string containing the bare function identifier extracted from name,
 following these rules in order:

1. Strips leading/trailing whitespace. If the result is empty, returns "".

2. Determines a "tail" string:
- Initially tail = name (after stripping).
- If tail contains "::", tail is set to the substring after the last "::",
 with leading whitespace removed.
- Else if tail contains ".", tail is set to the substring after the last ".",
 with leading whitespace removed.

3. Operator overload detection (applied to tail):
If tail starts with "operator":
- Let rest = tail[len("operator"):].lstrip()
- If rest starts with "[", returns "operator[]".
- If rest starts with "(", returns "operator()".
- If rest matches the pattern "new" optionally followed by whitespace
 and "[" whitespace "]", returns "operator new[]" if brackets are present,
 otherwise "operator new".
- If rest matches the pattern "delete" optionally followed by whitespace
 and "[" whitespace "]", returns "operator delete[]" if brackets are present,
 otherwise "operator delete".
- Otherwise, collects consecutive characters from rest that are in the set
 + - * / % & | ^ ~ ! = < > , and returns "operator" + the collected symbols.

4. If no operator result was produced, attempts the following regex matches on
 the original stripped name (before tail modification):
a. `^(?:^|::|\.)(\w+)$` - returns the rightmost identifier component
 (sequence of word characters) preceded by start-of-string, "::", or ".".
b. `(\s*\s*(\w+)\s*\s*)` - returns the identifier inside a
 function-pointer expression like "(*func)(...)".
c. `^\s*(\w+)$` - returns the identifier after a leading "*" (pointer
 return syntax).
d. `^(\w+)$` - returns the leading sequence of word characters.

5. If none of the above matches, returns the stripped name unchanged.

- Because the extraction patterns use \w+, template parameter brackets (<...>)
 and parenthesized parameter/argument lists are implicitly excluded from the
 returned identifier, except for operator names where they are explicitly
 included as part of the operator representation.
```

[SPEC]

## Reasoner 差异

- SPEC claim:

```
- Returns a string containing the bare function identifier extracted from name,
 following these rules in order:

1. Strips leading/trailing whitespace. If the result is empty, returns "".

2. Determines a "tail" string:
- Initially tail = name (after stripping).
- If tail contains "::", tail is set to the substring after the last "::",
 with leading whitespace removed.
- Else if tail contains ".", tail is set to the substring after the last ".",
```

with leading whitespace removed.

3. Operator overload detection (applied to tail):  
If tail starts with "operator":
    - Let rest = tail[len("operator")].rstrip()
    - If rest starts with "[", returns "operator[".
    - If rest starts with "(", returns "operator(").
    - If rest matches the pattern "new" optionally followed by whitespace and "[" whitespace "]", returns "operator new[]" if brackets are present, otherwise "operator new".
    - If rest matches the pattern "delete" optionally followed by whitespace and "[" whitespace "]", returns "operator delete[]" if brackets are present, otherwise "operator delete".
    - Otherwise, collects consecutive characters from rest that are in the set + - \* / % & | ^ ~ ! = < > , and returns "operator" + the collected symbols.
  4. If no operator result was produced, attempts the following regex matches on the original stripped name (before tail modification):
    - a. ``(?:^|::|\.)\w+$`` returns the rightmost identifier component (sequence of word characters) preceded by start-of-string, "::", or ".".
    - b. ``\s*\s*\s*(\w+)\s*\s*`` returns the identifier inside a function-pointer expression like `("*func)(...)"`.
    - c. ``\*\s*(\w+)`` returns the identifier after a leading "\*" (pointer return syntax).
    - d. ``^(\w+)`` returns the leading sequence of word characters.
  5. If none of the above matches, returns the stripped name unchanged.
- Because the extraction patterns use `\w+`, template parameter brackets (`<...>`) and parenthesized parameter/argument lists are implicitly excluded from the returned identifier, except for operator names where they are explicitly included as part of the operator representation.

- 推导 actual behavior:

The function returns a string `r` that is the bare function identifier extracted from the input name. Let `s = name.strip()`. If `s` is empty, `r = ''`. Otherwise, define `tail = s` if neither `::` nor `.` appear in `s`; else `tail = (s.rsplit(':', 1)[1] if '::' in s else s.rsplit('.', 1)[1]).rstrip()`. If tail starts with 'operator', then: let `rest = tail[8:].rstrip()`; if rest starts with '[' then `r = 'operator['`; else if rest starts with '(' then `r = 'operator('`; else if `re.fullmatch(r'new(?:\s*\[\s*\])?', rest)` then `r = 'operator new[]'` if '[' in rest else 'operator new'; else if `re.fullmatch(r'delete(?:\s*\[\s*\])?', rest)` then `r = 'operator delete[]'` if '[' in rest else 'operator delete'; else let `sym` be the longest prefix of rest consisting only of characters from the set `"+-*/%&|^~!=<>,"`; if `sym` is nonempty then `r = 'operator' + sym`; else fall through. If no return yet, then if `re.search(r'(?:^|::|\.)\w+$', s)` then `r =` the captured word; else if `re.match(r'\s*\s*\s*(\w+)\s*\s*', s)` then `r =` the captured identifier; else if `re.match(r'\*\s*(\w+)', s)` then `r =` the captured identifier; else if `re.match(r'^(\w+)', s)` then `r =` the captured word; else `r = s`. The result is always a string without leading/trailing whitespace, representing either a simple identifier or an operator name (e.g., `'operator=='`, `'operator new[]'`). In all cases, the function terminates without raising exceptions.

- Code evidence:

```
Line 41: if symbol:
Line 42: return "operator" + "".join(symbol)
```

- Trigger condition:

When tail starts with 'operator' but rest contains no consecutive operator symbols (e.g., 'Foo'), the specification requires returning 'operator' (the result of collecting zero symbols and concatenating). The code only returns if symbol is non-empty; otherwise it falls through to the regex patterns on the original name, which return 'operatorFoo' instead of 'operator'.

## Bug validator

- Trigger summary: When tail starts with 'operator' but rest contains no consecutive operator symbols (e.g. 'operatorFoo'), the code falls through to regex matching and returns 'operatorFoo' instead of the spec-required 'operator'.
- Probe stdout:



CONFIRMED — actual: 'operatorFoo' | expected: 'operator'

## INCR-MISMATCH-031 — `src--languages--codegraph-py--_codegraph_cmd`

- 人工审计：契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/codegraph-py/_codegraph_cmd.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_codegraph_cmd.json`。
- 详细报告: `src--languages--codegraph-py--_codegraph_cmd.md`。
- Probe: `probe_src--languages--codegraph-py--_codegraph_cmd.py`。

### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/codegraph.py

_codegraph_cmd() -> str

Pre-condition:
- settings.codegraph.bin_dir is a string specifying a directory path, potentially beginning
 with a tilde (~) representing the current user's home directory.

Post-condition:
- Returns a string suitable for use as an executable command name.
- When a file named "codegraph" exists within the directory obtained by expanding any leading
 tilde in the configured bin_dir to the user's home directory and that file has the execute
 permission bit set for the effective user of the current process, returns the absolute
 filesystem path to that file.
- When that file does not exist or lacks the execute permission bit, returns the bare string
 "codegraph", deferring resolution to the directories named by the PATH environment variable of
 the calling process.
- Never raises an exception.
[SPEC]
```

### Reasoner 差异

- SPEC claim:

```
- Returns a string suitable for use as an executable command name.
- When a file named "codegraph" exists within the directory obtained by expanding any leading
 tilde in the configured bin_dir to the user's home directory and that file has the execute
 permission bit set for the effective user of the current process, returns the absolute
 filesystem path to that file.
- When that file does not exist or lacks the execute permission bit, returns the bare string
 "codegraph", deferring resolution to the directories named by the PATH environment variable of
 the calling process.
- Never raises an exception.
```

- 推导 actual behavior:

```
The function returns the absolute path of the 'codegraph' executable under the tilde-expanded
`settings.codegraph.bin_dir` directory if that file exists and has the executable permission
(os.X_OK); otherwise it returns the bare command string 'codegraph'. Formally: let expanded =
os.path.expanduser(settings.codegraph.bin_dir), let path = os.path.join(expanded, 'codegraph');
then the return value r satisfies r = path if os.access(path, os.X_OK) else r = 'codegraph'.
```

- Code evidence:

```
Line 13: local = os.path.join(bin_dir, "codegraph")
Line 14: return local if os.access(local, os.X_OK) else "codegraph"
```

- Trigger condition:

The code does not convert the joined path to an absolute path before returning it, so when the configured `bin_dir` is relative the returned path is relative, violating the specification's requirement to return an absolute filesystem path.

#### Bug validator

- Trigger summary: When `settings.codegraph.bin_dir` is a relative path and an executable 'codegraph' file exists at that path, `_codegraph_cmd()` returns the relative path instead of an absolute one.
- Probe stdout:

CONFIRMED — actual (relative): `'../../probe_codegraph_cmd_yy10d4pr/codegraph'` | expected (absolute): `'/tmp/probe_codegraph_cmd_yy10d4pr/codegraph'`

### INCR-MISMATCH-032 — `src--languages--codegraph-py--_extraction_ident`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/codegraph-py/_extraction_ident.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_extraction_ident.json`。
- 详细报告: `src--languages--codegraph-py--_extraction_ident.md`。
- Probe: `probe_src--languages--codegraph-py--_extraction_ident.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/codegraph.py

_extraction_ident(name: str, qualified_name: str) -> str

Pre-condition:
- name and qualified_name are strings obtained from the codegraph database for a single function or method node

Post-condition:
- Returns a string composed of one or more components joined by the literal "::"
- When qualified_name is non-empty and its suffix equals name, the leading components of the returned string (all except the last) correspond, in order, to the scope qualifiers extracted from the prefix of qualified_name that precedes name
- When qualified_name is empty or its suffix does not equal name, the returned string consists of exactly one component
- The final component of the returned string is derived from name
- No component of the returned string contains any character that would be a directory separator in any filesystem
- No component of the returned string contains signature syntax, pointer syntax, or template syntax that may have been present in the raw database column values
- The same (name, qualified_name) pair always produces the same returned string
- The separator character ("." vs "::") used in qualified_name does not affect the set or order of scope components in the returned string
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- Returns a string composed of one or more components joined by the literal "::"
- When qualified_name is non-empty and its suffix equals name, the leading components of the returned string (all except the last) correspond, in order, to the scope qualifiers extracted from the prefix of qualified_name that precedes name
- When qualified_name is empty or its suffix does not equal name, the returned string consists of exactly one component
- The final component of the returned string is derived from name
```

- No component of the returned string contains any character that would be a directory separator in any filesystem
- No component of the returned string contains signature syntax, pointer syntax, or template syntax that may have been present in the raw database column values
- The same (name, qualified\_name) pair always produces the same returned string
- The separator character ( "." vs "::" ) used in qualified\_name does not affect the set or order of scope components in the returned string

- 推导 actual behavior:

The function returns a string `r` computed as `r = '::'.join(canonicalize(_bare_function_name(p)) for p in _qualified_parts(name, qualified_name))`. Let `Q = _qualified_parts(name, qualified_name)`; then `Q` is a list of at least one non-empty string, with the last element equal to `name`. For each element `q` in `Q`, define `s = canonicalize(_bare_function_name(q))`. Each `s` is a string containing no characters that are invalid in filesystem path components; `s` may be empty if `_bare_function_name` returns an empty string (e.g., when `q` consists entirely of whitespace). The returned string `r` is the concatenation of the resulting strings `s1, s2, ..., sn` interleaved with `::`. Thus `r` has the form `s0 + '::' + s1 + '::' + ... + s_{n-1}`. When `qualified_name` is non-empty and has `name` as a suffix, the scope qualifier components from `qualified_name`'s prefix are included as the first elements of `Q`; otherwise `Q = [name]`. The return value is deterministic and depends only on `name` and `qualified_name`.

- Code evidence:

```
Line 15: return "::".join(
Line 16: canonicalize(_bare_function_name(p))
Line 17: for p in _qualified_parts(name, qualified_name)
Line 18:)
```

- Trigger condition:

The code does not filter out empty strings that can result from `_bare_function_name`. For the input where `qualified_name` has a whitespace-only scope qualifier ( `' ::MyClass::func'` ), `_bare_function_name` returns an empty string for that qualifier, causing the final joined string to be `::MyClass::func`. The specification requires the returned string to be composed of one or more (nonempty) components joined by `::`, but the result starts with `::` due to the empty leading component, violating that requirement.

#### Bug validator

- Trigger summary: Whitespace-only scope qualifier components (e.g., `'foo:: ::func'`) produce empty strings from `_bare_function_name`, which pass through `canonicalize` and create empty `::`-separated components in the result, violating the spec's non-empty component requirement.
- Probe stdout:

```
CONFIRMED - Empty components found in _extraction_ident output
[whitespace-only middle qualifier] name='func', qualified_name='foo:: ::func'
 actual: 'foo:::func'
 split by '::': ['foo', '', 'func'] (contains empty!)
[space+tab qualifier] name='baz', qualified_name='X:: :: baz'
 actual: 'X:::baz'
 split by '::': ['X', '', '', 'baz'] (contains empty!)
```

---

#### INCR-MISMATCH-033 — `src--languages--codegraph-py--_node_fqn_map`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/codegraph-py/_node_fqn_map.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_node_fqn_map.json`。
- 详细报告: `src--languages--codegraph-py--_node_fqn_map.md`。
- Probe: `probe_src--languages--codegraph-py--_node_fqn_map.py`。

[SPEC]

Unit: src/languages/codegraph-py/\_node\_fqn\_map.py

`_node_fqn_map(cur, cg_langs) -> dict`

Pre-condition:

- cur is a database cursor connected to a codegraph database containing a nodes table with columns id, name, file\_path, start\_line, kind, and language
- cg\_langs is a non-empty sequence of language key strings

Post-condition:

- Returns a dict mapping each node's id to its fully-qualified function name (FQN)
- The mapping includes exactly the rows from the nodes table whose kind is either 'function' or 'method' and whose language is one of the given cg\_langs values, ordered by (file\_path ASC, start\_line ASC)
- Each FQN is derived from the node's file\_path and a canonicalized, deduplicated function name in the canonical convention where path components are joined by "::" and the source file extension in the parent directory component is replaced by a hyphen
- Function name canonicalization strips angle-bracket template parameters and normalizes operator-overload names to safe identifier forms
- When N > 1 nodes share the same file\_path and canonicalized name, the first such node in the result ordering receives the canonicalized name without a suffix, and each subsequent node (k-th, 1-indexed) receives the canonicalized name suffixed with \_k
- Returns an empty dict when the query matches no rows
- The deduplication rule and ordering correspond to those used by get\_functions\_by\_file, ensuring that the FQN assigned to a node here matches the FQN the extracted function file receives for the same node

[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns a dict mapping each node's id to its fully-qualified function name (FQN)
- The mapping includes exactly the rows from the nodes table whose kind is either 'function' or 'method' and whose language is one of the given cg\_langs values, ordered by (file\_path ASC, start\_line ASC)
- Each FQN is derived from the node's file\_path and a canonicalized, deduplicated function name in the canonical convention where path components are joined by "::" and the source file extension in the parent directory component is replaced by a hyphen
- Function name canonicalization strips angle-bracket template parameters and normalizes operator-overload names to safe identifier forms
- When N > 1 nodes share the same file\_path and canonicalized name, the first such node in the result ordering receives the canonicalized name without a suffix, and each subsequent node (k-th, 1-indexed) receives the canonicalized name suffixed with \_k
- Returns an empty dict when the query matches no rows
- The deduplication rule and ordering correspond to those used by get\_functions\_by\_file, ensuring that the FQN assigned to a node here matches the FQN the extracted function file receives for the same node

- 推导 actual behavior:

If no exception occurs: The function returns a dictionary `result` such that: Let `Q` be the ordered list of rows from `cur.execute` of `SELECT id, name, qualified\_name, file\_path, start\_line FROM nodes WHERE kind IN ('function','method') AND language IN (?,...?)` with parameters `cg\_langs`, sorted by `file\_path, start\_line` ascending. For each row `(id, name, qualified\_name, file\_path, \_)` in `Q` in order, let `ident = \_extraction\_id(name, qualified\_name)` and `key = (file\_path, ident)`. Define a counter function `occ(key, i) = |{ j < i | key\_j = key }|` (the 0based occurrence index). Then `deduped = ident if occ(key, i) = 0 else f"{ident}\_{occ(key,i)}"`. Then `result[id] = \_fqn\_for(file\_path, deduped)`. After iterating all rows, `result` contains exactly `{r.id for r in Q}` as keys and no other entries. The cursor `cur` has been completely fetched (no remaining rows from that query). No other mutable state is modified. If an exception is raised (e.g., SQL error, fetch error, or exceptions from helper functions), the exception propagates; no explicit rollback or cleanup is performed, and the state of `cur` and any partially built `result` is lost to the caller.

- Code evidence:

```
Line 26: deduped = ident if c == 0 else f"{ident}_{c}"
```

- Trigger condition:

The specification requires the k-th occurrence (1-indexed) to be suffixed with `_k`, whereas the code uses a zero-based counter that produces `_1` for the second occurrence, `_2` for the third, etc., violating the defined deduplication rule.

#### Bug validator

- Trigger summary: 0-based counter 'c' used directly as suffix; spec requires 1-indexed `_k` for k-th duplicate — second occurrence gets `_1` instead of `_2`.
- Probe stdout:

```
CONFIRMED — off-by-one deduplication suffix reproduced.
Expected (spec-correct): {1: 'src::util-c::helper', 2: 'src::util-c::helper_2', 3: 'src::util-c::helper_3'}
Actual (buggy): {1: 'src::util-c::helper', 2: 'src::util-c::helper_1', 3: 'src::util-c::helper_2'}
Buggy pattern matches: True
```

---

#### INCR-MISMATCH-034 — `src--languages--codegraph-py--_qualified_parts`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/codegraph-py/_qualified_parts.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_qualified_parts.json`。
- 详细报告: `src--languages--codegraph-py--_qualified_parts.md`。
- Probe: `probe_src--languages--codegraph-py--_qualified_parts.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/codegraph.py

_qualified_parts(name: str, qualified_name: str) -> list

Pre-condition:
- name and qualified_name are strings

Post-condition:
- Returns a list of non-empty strings
- The last element of the returned list equals name
- When qualified_name is non-empty and has name as a suffix, the elements before the last are the scope qualifier components extracted from the prefix of qualified_name that precedes name, split on "::" or "."
- When qualified_name is empty or does not have name as a suffix, the returned list is [name]
- The character used as the scope separator in qualified_name ("." or "::") does not affect the set or order of components in the returned list
- The same (name, qualified_name) pair always produces the same returned list
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- Returns a list of non-empty strings
- The last element of the returned list equals name
```

- When `qualified_name` is non-empty and has `name` as a suffix, the elements before the last are the scope qualifier components extracted from the prefix of `qualified_name` that precedes `name`, split on `::` or `.`
- When `qualified_name` is empty or does not have `name` as a suffix, the returned list is `[name]`
- The character used as the scope separator in `qualified_name` (`.` or `::`) does not affect the set or order of components in the returned list
- The same `(name, qualified_name)` pair always produces the same returned list

- 推导 actual behavior:

The function returns a list `L` with the following behavior. Let `q = qualified_name.strip()`. If `q` is the empty string or does not end with `name`, `L = [name]`. Otherwise, let `scope = q[:-len(name)].rstrip(':.')`. If `scope` is the empty string, `L = [name]`. Otherwise, `L = [p for p in re.split(r'::|\.', scope) if p != ''] + [name]`. No exceptions are raised because inputs are strings and operations are safe for strings.

- Code evidence:

```
Line 14: q = (qualified_name or "").strip()
```

- Trigger condition:

The specification requires extracting the scope prefix from the original `qualified_name` (without stripping), but the code strips whitespace via Line 14 before processing. For `qualified_name='bar::foo'`, the specification would yield `['bar', 'foo']` because the prefix `'bar::'` split on `::` gives `'bar'`, while the code strips to `'bar::foo'` and returns `['bar', 'foo']`, violating the requirement that the returned list reflects the scope components of the original `qualified_name` prefix.

#### Bug validator

- Trigger summary: Function strips whitespace from `qualified_name` before extracting scope components, so leading whitespace in scope prefix is lost: `qualified_name=' bar::foo'` with `name='foo'` returns `['bar', 'foo']` instead of spec-expected `['bar', 'foo']`.
- Probe stdout:

```
CONFIRMED — actual: ['bar', 'foo'] | expected: ['bar', 'foo']
```

---

#### INCR-MISMATCH-035 — `src--languages--codegraph-py--_warn_on_codegraph_version_mismatch`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/codegraph-py/_warn_on_codegraph_version_mismatch.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_warn_on_codegraph_version_mismatch.json`。
- 详细报告: `src--languages--codegraph-py--_warn_on_codegraph_version_mismatch.md`。
- Probe: `probe_src--languages--codegraph-py--_warn_on_codegraph_version_mismatch.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/codegraph-py/_warn_on_codegraph_version_mismatch.py

_warn_on_codegraph_version_mismatch(cmd: str) -> None
```

Pre-condition:

- cmd is a non-empty string identifying an executable command on the system PATH.
- settings.codegraph.version is a string (may be empty or whitespace-only).

Post-condition:

- Returns None; never raises an exception.
- The function has no externally observable side effect unless all of the following conditions are met:
  - (a) the configured codegraph version, after stripping leading and trailing whitespace and removing any leading "v" prefix, is non-empty;
  - (b) executing the command referred to by cmd with the argument "--version" succeeds as a subprocess and produces non-empty output after stripping leading and trailing whitespace from its captured stdout;
  - (c) that output does not equal the configured version after each has been stripped of whitespace and any leading "v" prefix.
- When all conditions (a), (b), and (c) are met: a log record at WARNING severity is emitted whose message identifies both the version string obtained from the command output and the configured version string from fm-agent.toml.

[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns None; never raises an exception.
- The function has no externally observable side effect unless all of the following conditions are met:
  - (a) the configured codegraph version, after stripping leading and trailing whitespace and removing any leading "v" prefix, is non-empty;
  - (b) executing the command referred to by cmd with the argument "--version" succeeds as a subprocess and produces non-empty output after stripping leading and trailing whitespace from its captured stdout;
  - (c) that output does not equal the configured version after each has been stripped of whitespace and any leading "v" prefix.
- When all conditions (a), (b), and (c) are met: a log record at WARNING severity is emitted whose message identifies both the version string obtained from the command output and the configured version string from fm-agent.toml.

- 推导 actual behavior:

The function completes without raising any exception. Let  $V = \text{settings.codegraph.version.strip().removeprefix('v')}$ . If  $V$  is empty, no warning is logged. Otherwise, the function attempts to run `subprocess.run([cmd, '--version'], capture_output=True, text=True, timeout=10)`. If this raises `OSError` or `subprocess.SubprocessError`, no warning is logged. If it succeeds, let `output` = the stripped stdout of the completed process. If `output` is truthy and `output != V`, then a WARNING log record is emitted via `logging.warning` with the format string `'codegraph %r does not match the pinned %r (fm-agent.toml [codegraph].version); re-run install.sh to update.'` and arguments `(output, V)`. Otherwise, no warning is logged. No other side effects occur.

Formally: Let  $V = \text{strip(removeprefix(settings.codegraph.version, 'v'))}$ . The post-condition is:  
 $(V='') \rightarrow (\text{subprocess.run}([cmd, '--version'], \text{capture\_output}=\text{True}, \text{text}=\text{True}, \text{timeout}=10) \text{ raises } \text{OSError} \text{ or } \text{SubprocessError}) \rightarrow (\text{let } stdout = \text{strip(result.stdout)}; (stdout \neq V)) \rightarrow \text{no WARNING log with the specified message is emitted};$   
 $(V \neq '' \wedge \text{subprocess succeeds}) \rightarrow (\text{let } s = \text{strip(result.stdout)}; s \neq V) \rightarrow \text{a WARNING log with the specified message and arguments } (s, V) \text{ is emitted}.$

- Code evidence:

Line 6: `want = settings.codegraph.version.strip().removeprefix("v");` Line 15: `if got and got != want:`

- Trigger condition:

The code removes the leading 'v' only from the configured version, not from the command output. The specification requires that both strings be stripped and have any leading 'v' removed before

comparison. When both strings match after this normalization (e.g., configured 'v1.0' and output 'v1.0'), the code incorrectly emits a warning, violating condition (c) of the specification.

## Bug validator

- Trigger summary: The code removes leading 'v' only from the configured version but not from the command output, causing a false WARNING when both versions are semantically identical (e.g., configured 'v1.0' and output 'v1.0').
- Probe stdout:

CONFIRMED — WARNING emitted despite versions matching after normalization: "codegraph 'v1.0' does not match the pinned '1.0' (fm-agent.toml [codegraph].version); re-run install.sh to update."

## INCR-MISMATCH-036 — `src--languages--codegraph-py--try_codegraph_init`

- 人工审计: 实现缺陷候选。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/codegraph-py/try_codegraph_init.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/try_codegraph_init.json`。
- 详细报告: `src--languages--codegraph-py--try_codegraph_init.md`。
- Probe: `probe_src--languages--codegraph-py--try_codegraph_init.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/languages/codegraph-py/try_codegraph_init.py

try_codegraph_init(proj_dir: str, force: bool = True) -> None

Pre-condition:
- proj_dir is a non-empty string representing a directory path on the filesystem.
- force is True or False.

Post-condition:
- Returns None; never raises an exception.
- When the `codegraph` executable is not found on the system PATH: returns immediately without creating, modifying, or removing any files under proj_dir.
- When proj_dir/.codegraph/codegraph.db exists AND force is False: returns immediately; the existing index file and its parent directory are preserved.
- Otherwise (force is True, or proj_dir/.codegraph/codegraph.db does not exist):
 - If a proj_dir/.codegraph/ directory exists, it is removed prior to rebuilding (recursively, with errors ignored).
 - `codegraph init` is executed with proj_dir as its working directory.
 - If `codegraph init` exits with code 0: proj_dir/.codegraph/codegraph.db exists after return and reflects the file tree of proj_dir at the time `codegraph init` was invoked.
 - If `codegraph init` exits with a non-zero code: a warning is logged whose message includes the first 300 characters of stderr; the function returns and the contents of proj_dir/.codegraph/ are unspecified.

[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns None; never raises an exception.
- When the `codegraph` executable is not found on the system PATH: returns immediately without creating, modifying, or removing any files under proj_dir.
- When proj_dir/.codegraph/codegraph.db exists AND force is False: returns immediately; the existing index file and its parent directory are preserved.
- Otherwise (force is True, or proj_dir/.codegraph/codegraph.db does not exist):
 - If a proj_dir/.codegraph/ directory exists, it is removed prior to rebuilding (recursively, with errors ignored).
```



- ``codegraph init`` is executed with `proj_dir` as its working directory.
- If ``codegraph init`` exits with code 0: `proj_dir/.codegraph/codegraph.db` exists after return and reflects the file tree of `proj_dir` at the time ``codegraph init`` was invoked.
- If ``codegraph init`` exits with a non-zero code: a warning is logged whose message includes the first 300 characters of `stderr`; the function returns and the contents of `proj_dir/.codegraph/` are unspecified.

- 推导 actual behavior:

After the function returns, the following post-conditions hold regarding the project directory ``proj_dir``:

1. The existence of the codegraph index database file ``db = os.path.join(proj_dir, '.codegraph', 'codegraph.db')`` satisfies:

```
db_post_exists ((db_pre_exists force = False) (cmd_available (force db_pre_exists)
exit_code = 0))
```

where

- `db_pre_exists` = `os.path.exists(db)` before the call,
- `db_post_exists` = `os.path.exists(db)` after the call,
- `cmd_available` = the ``codegraph`` command identified by ``_codegraph_cmd()`` is present on PATH and does not raise ``FileNotFoundError``,
- `exit_code` = return code of ``subprocess.run([cmd, 'init'], ...)`` if it runs (0 on success).

2. If ``force`` was True and ``db_pre_exists`` was True, the entire ``.codegraph`` directory was removed via ``shutil.rmtree`` before the command attempt.
3. The function never raises an exception; it handles missing executables and non-zero exit codes gracefully.
4. Informational messages are printed or logged as per lines 25,27,37,39-42.

- Code evidence:

```
Line 19: if os.path.exists(db_path):
Line 20: if not force:
Line 21: return
Line 24: shutil.rmtree(codegraph_dir, ignore_errors=True)
```

- Trigger condition:

The code removes the existing `.codegraph` directory (line 24) before checking whether the `'codegraph'` executable exists (line 31). When the executable is missing, a `FileNotFoundError` is caught (line 34-35) and the function returns, but the removal has already occurred. This violates the specification requirement that when the executable is not found, the function must return immediately without creating, modifying, or removing any files under `proj_dir`.

#### Bug validator

- Trigger summary: When `force=True` and `.codegraph/codegraph.db` exists but the codegraph executable is not found on PATH, `shutil.rmtree()` removes the `.codegraph` directory before the executable check, violating the spec requirement of no file modifications when the executable is missing.
- Probe stdout:

```
[Pipeline] Rebuilding codegraph index for current working tree...
CONFIRMED - .codegraph directory was removed before checking for codegraph executable. Spec
violation: the spec requires that when the codegraph executable is not found, the function
returns immediately without creating, modifying, or removing any files under proj_dir.
```

## src/languages/cpp-py

- 人工审计: **推理**误判。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: **src/languages/cpp-py/function\_spans.py**。
- Reasoner 结果: **logic\_verification\_results/src/languages/cpp-py/function\_spans.json**。
- 详细报告: **src--languages--cpp-py--function\_spans.md**。
- Probe: **probe\_src--languages--cpp-py--function\_spans.py**。

完整生成 SPEC

```
[SPEC]
Unit: src/languages/cpp-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list | None

Pre-condition:
- proj_dir is a valid path to a project directory on the filesystem
- filepath is a string identifying a C++ source file within the project

Post-condition:
- Returns None when a codegraph backend is unavailable or does not index the file, signaling the caller to fall back to regex-based extraction
- Otherwise returns a list of (name, start_idx, end_idx) tuples, each identifying one function in the file by its name and the range of source lines it occupies
- In every returned tuple, start_idx and end_idx are 0-indexed inclusive line indices
- The returned list covers every function that the codegraph backend detects in the file
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns None when a codegraph backend is unavailable or does not index the file, signaling the caller to fall back to regex-based extraction
- Otherwise returns a list of (name, start_idx, end_idx) tuples, each identifying one function in the file by its name and the range of source lines it occupies
- In every returned tuple, start_idx and end_idx are 0-indexed inclusive line indices
- The returned list covers every function that the codegraph backend detects in the file
```

- 推导 actual behavior:

```
After execution, the function either propagates any exception raised by
`CodeGraphExtractor.from_proj_dir(proj_dir)` or returns a value. If no exception occurs, the
return value is determined as follows: let `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`. If
`cg` is falsy (e.g., `None`, `False`), the function returns `None`. Otherwise, let `spans =
cg.get_function_spans('cpp', filepath)`. If `spans` is `None`, the function returns `None`;
otherwise it returns the list `spans`, which is a nonempty list of 3tuples `(name: str, start:
int, end: int)`. Each tuple describes a function definition found in the given C++ source file,
with `start` and `end` being 0indexed inclusive line numbers. The function does not modify any
external state. Formally, let `R` denote the outcome (exception or return value), `E_from` the
event that `CodeGraphExtractor.from_proj_dir(proj_dir)` raises an exception. Then: (E_from R =
that exception) (E_from cg is falsy R = None) (E_from cg is truthy (spans =
cg.get_function_spans('cpp', filepath) (spans = None R = None) (spans None R = spans spans
is a list of tuples each of the form (string, int, int) with semantics as stated))).
```

- Code evidence:

```
Line 7: cg = CodeGraphExtractor.from_proj_dir(proj_dir)
Line 8: return cg.get_function_spans("cpp", filepath) if cg else None
```

- Trigger condition:

```
The function does not handle exceptions from CodeGraphExtractor.from_proj_dir. When proj_dir is
an invalid directory, from_proj_dir may raise an exception (e.g., FileNotFoundError) instead of
returning a falsy value. The specification requires returning None when the codegraph backend is
```

unavailable, so the caller can fall back to regex extraction. By propagating the exception, the code violates this specification for any invalid proj\_dir.

#### Bug validator

- Trigger summary: Passing None as proj\_dir causes CodeGraphExtractor.from\_proj\_dir() to raise TypeError via os.path.abspath(None), which propagates instead of returning None as the spec requires.
- Probe stdout:

CONFIRMED — exception propagated: TypeError: expected str, bytes or os.PathLike object, not NoneType | expected: None

## src/languages/erlang-py

### INCR-MISMATCH-038 — src--languages--erlang-py--ElpClient::\_handle\_server\_message

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::_handle_server_message.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::_handle_server_message.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::_handle_server_message.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::_handle_server_message.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang.py

ElpClient._handle_server_message(self, message: dict)

Pre-condition:
- self is an ElpClient whose __enter__ has been called (ELP subprocess
 is running and the JSON-RPC message reader thread is active)
- message is a dict representing a parsed JSON-RPC message received
 from the ELP server

Post-condition:
- When message.method is "elp/status", updates self._status to the
 value of message.params.status; if the params dict is absent or lacks
 a "status" key, self._status is unchanged
- When message lacks an "id" field, or message.method is absent or
 falsy, no response is sent (the message is treated as a notification)
- When message carries both a non-empty "method" and an "id" (a server
 request), sends a JSON-RPC response with jsonrpc "2.0" and the same
 id; the result value satisfies the protocol-defined expectation for
 that method:
 - For workspace configuration queries: result is a list whose length
 equals the number of requested configuration items, each element
 being null
 - For workspace folder queries: result is a singleton list containing
 the workspace-folder descriptor with the project root URI and
 directory name
 - For workspace edit requests: result indicates the edit was declined
 (applied is false)
 - For any other method the client does not handle: result is null

[SPEC]
```

#### Reasoner 差异

- SPEC claim:

- When message.method is "elp/status", updates self.\_status to the value of message.params.status; if the params dict is absent or lacks a "status" key, self.\_status is unchanged
- When message lacks an "id" field, or message.method is absent or falsy, no response is sent (the message is treated as a notification)
- When message carries both a non-empty "method" and an "id" (a server request), sends a JSON-RPC response with jsonrpc "2.0" and the same id; the result value satisfies the protocol-defined expectation for that method:
  - For workspace configuration queries: result is a list whose length equals the number of requested configuration items, each element being null
  - For workspace folder queries: result is a singleton list containing the workspace-folder descriptor with the project root URI and directory name
  - For workspace edit requests: result indicates the edit was declined (applied is false)
  - For any other method the client does not handle: result is null

#### • 推导 actual behavior:

After execution of `_handle_server_message`: if the incoming message's method field equals 'elp/status', the client's `_status` attribute is set to the value of `params.get('status')` (which may be None). If the message contains an 'id' and the method field is truthy (non-empty string), a JSON-RPC 2.0 response with the same id and a computed result is sent to the ELP server via the stdin pipe, using the transport protocol framing; the server will receive it as its next input. The result sent is: for 'workspace/configuration', a list of None of length equal to `len(params.get('items', []))`; for 'workspace/workspaceFolders', `[{'uri': self.root_uri, 'name': os.path.basename(self.proj_dir)}]`; for 'workspace/applyEdit', `{'applied': False}`; for any other method with an id, None. No response is sent if the message is a notification (no 'id') or the method is falsy. The ELP subprocess remains running and the message reader thread continues to be active. Formal logic: Let pre-state satisfy `IsRunning(pre) ThreadActive(pre) pre_message = message`. Then the post-state satisfies: `IsRunning(post) ThreadActive(post) (self._status = pre_self._status (message.method = 'elp/status' self._status = message.params.get('status')) ((('id' message is_truthy(message.method)) sent_response : sent_response.jsonrpc = '2.0' sent_response.id = message.id WrittenToStdin(sent_response) sent_response.result = RESULT(message.method, params, self)) with RESULT = method, params, self. case method of 'workspace/configuration' [None | _ params.get('items', [])], 'workspace/workspaceFolders' [{'uri': self.root_uri, 'name': os.path.basename(self.proj_dir)}], 'workspace/applyEdit' {'applied': False}, other None.`

#### • Code evidence:

```
Line 4: if params is None:
Line 5: params = {}
Line 7: self._status = params.get("status")
```

#### • Trigger condition:

The code unconditionally sets `self._status` to `params.get('status')`, which evaluates to None when the 'status' key is absent or `params` is missing. The specification requires `self._status` to remain unchanged in that scenario.

### Bug validator

- Trigger summary: `params.get('status')` returns None when 'status' key is absent, overwriting `_status` instead of leaving it unchanged
- Probe stdout:

CONFIRMED — actual: None | expected: 'INITIAL\_VALUE'

INCR-MISMATCH-039 — `src--languages--erlang-py--ElpClient::_next_message`

- 人工审计: 推理误判。

- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::_next_message.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::_next_message.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::_next_message.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::_next_message.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang.py

ElpClient._next_message(self, deadline: float)

Pre-condition:
- deadline is a point in time expressed as a monotonic clock value
- self is an ElpClient whose message reader thread is active and
 writing parsed JSON-RPC messages into an internal queue

Post-condition:
- When a message is available before the monotonic clock reaches
 deadline, returns the next pending parsed JSON-RPC response or
 notification message from the server as a dict whose shape conforms
 to the JSON-RPC 2.0 specification
- Raises TimeoutError when no message is available before the
 monotonic clock reaches deadline, including when the deadline has
 already passed at call entry
- Raises RuntimeError when the message reader thread terminated with
 an unrecoverable exception; the original exception from the reader
 thread is chained as the cause of the RuntimeError
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- When a message is available before the monotonic clock reaches
 deadline, returns the next pending parsed JSON-RPC response or
 notification message from the server as a dict whose shape conforms
 to the JSON-RPC 2.0 specification
- Raises TimeoutError when no message is available before the
 monotonic clock reaches deadline, including when the deadline has
 already passed at call entry
- Raises RuntimeError when the message reader thread terminated with
 an unrecoverable exception; the original exception from the reader
 thread is chained as the cause of the RuntimeError
```

- 推导 actual behavior:

After the method executes, exactly one of the following outcomes occurs:

1. Normal return: The method returns a message `msg`. `msg` is not an instance of `BaseException`. It was removed from `self.\_messages` before `deadline` (i.e., at some instant `t` with `t <= deadline`). The internal queue after the call contains every element that was enqueued by the reader thread during the call except `msg`. The monotonic clock at the return point satisfies `time.monotonic() <= deadline`.
2. TimeoutError raised: A `TimeoutError` is raised. No element was removed from `self.\_messages` before `deadline`. The queue after the call is unchanged apart from any new elements added by the reader thread. At the moment the exception is raised, `time.monotonic() >= deadline`.
3. RuntimeError raised: A `RuntimeError` is raised, chained from a `BaseException` `e`. `e` was removed from `self.\_messages` before `deadline`. The queue after the call contains every other element enqueued by the reader thread during the call. The removal occurs at some instant `t` with `t <= deadline`.

The active message reader thread continues to run and may keep enqueueing parsed JSON-RPC messages or exceptions into `self.\_messages`.

Formally, let:

- `Q\_pre` be the multiset of elements in `self.\_messages` at method entry.
- `Q\_post` be the multiset at the point the method ends (after return or just before exception

```
propagation).
- `T_pre` = `time.monotonic()` at entry, `T_post` at exit.
- `added` be the multiset of elements the reader thread enqueued during the interval `[T_pre, T_post]`.
- `deadline` the given deadline.
```

The postcondition is the disjunction:

```
- `(return msg) [msg BaseException msg (Q_pre added) Q_post = (Q_pre added) \ {msg}
T_post deadline]`

- `(raise TimeoutError) [ Q_post = Q_pre added    T_post deadline ]`

- `(raise RuntimeError from e) [e BaseException e (Q_pre added) Q_post = (Q_pre
added) \ {e} T_post deadline]`
```

- Code evidence:

Line 11: return message

- Trigger condition:

The specification requires the method to return a dict that conforms to the JSON-RPC 2.0 specification. The code, however, returns any non-BaseException object from the queue without checking that it is a dict, thus violating the specification for any input where the queue contains a non-dict non-exception value.

#### Bug validator

- Trigger summary: `_next_message` returns any non-BaseException queue item without checking it is a dict, so a non-dict JSON value (like a list) is returned in violation of the spec.
- Probe stdout:

CONFIRMED — actual: [1, 2, 3] (type: list) | expected: dict conforming to JSON-RPC 2.0

---

#### INCR-MISMATCH-040 — `src--languages--erlang-py--ElpClient::_send`

- 人工审计: **推理误判**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::_send.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::_send.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::_send.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::_send.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang.py

ElpClient._send(self, message: dict)

Pre-condition:
- self._proc is not None and self._proc.stdin is not None and open for
 writing
- message is a dict representing a JSON-serializable JSON-RPC message

Post-condition:
- The JSON-serialized form of message is transmitted to the ELP
 subprocess's standard input using LSP transport protocol framing
- The transmitted frame consists of a Content-Length header whose value
 is the length in bytes of the UTF-8 encoded JSON payload, followed by
 a CRLF blank line, then the UTF-8 encoded JSON payload itself
- The header portion is ASCII-encoded; the payload is compact JSON with
```

```
no whitespace between keys, values, colons, or commas, and all non-ASCII
characters are preserved in their original form
- Transmission is atomic with respect to other concurrent _send calls on
 the same client instance
- On return, the complete frame has been delivered to the subprocess's
 input stream (the write has been flushed to the OS pipe)
- Raises RuntimeError when the ELP subprocess is not running or its
 standard input is unavailable
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- The JSON-serialized form of message is transmitted to the ELP
 subprocess's standard input using LSP transport protocol framing
- The transmitted frame consists of a Content-Length header whose value
 is the length in bytes of the UTF-8 encoded JSON payload, followed by
 a CRLF blank line, then the UTF-8 encoded JSON payload itself
- The header portion is ASCII-encoded; the payload is compact JSON with
 no whitespace between keys, values, colons, or commas, and all non-ASCII
 characters are preserved in their original form
- Transmission is atomic with respect to other concurrent _send calls on
 the same client instance
- On return, the complete frame has been delivered to the subprocess's
 input stream (the write has been flushed to the OS pipe)
- Raises RuntimeError when the ELP subprocess is not running or its
 standard input is unavailable
```

- 推导 actual behavior:

After execution under the given pre-condition, the method completes without raising an exception. The dict message is serialized to a JSON string (UTF-8 encoded), framed with a 'Content-Length' header, written atomically to self.\_proc.stdin under self.\_write\_lock, and then flushed. The flush ensures all buffered data is pushed to the subprocess's console stream (unconditionally) and, if the log stream is open, to the log stream as well. After the method returns, self.\_proc.stdin remains open and writable, self.\_write\_lock is released, and the entire frame has been successfully transmitted to the subprocess. Formally, let  $\text{pre\_state}$  be the pre-state and  $\text{post\_state}$  be the post-state. Then  $\text{pre\_state} \rightarrow \text{post\_state}$  satisfies: (1) The sequence of bytes written to the console stream of self.\_proc.stdin is `.console_stream frame`, where `frame = b'Content-Length: ' + len(payload_encoded).to_bytes() + b'\r\n\r\n' + payload_encoded` and `payload_encoded = json.dumps(message, ...).encode('utf-8')`. (2) If the optional log stream associated with self.\_proc.stdin was open in  $\text{pre\_state}$ , then its content is `.log_stream frame`; otherwise it is unchanged. (3) The internal buffer of self.\_proc.stdin is empty. (4) self.\_write\_lock is in the unlocked state. (5) self.\_proc.stdin remains non-None and writable.

- Code evidence:

```
Line 2: if self._proc is None or self._proc.stdin is None:
```

- Trigger condition:

The specification requires raising RuntimeError when the subprocess's standard input is unavailable. The code only checks for None, but stdin can be unavailable while still being a non-None object (e.g., closed or broken pipe), causing a different exception to propagate instead of RuntimeError.

## Bug validator

- Trigger summary: stdin is a non-None object but closed/unavailable; the None check passes so RuntimeError is not raised, and write/flush throws ValueError instead
- Probe stdout:

CONFIRMED – code raised ValueError("write to closed file") but spec requires RuntimeError when stdin is unavailable. \_proc.stdin is non-None (mock: <MagicMock name='mock.stdin')

## INCR-MISMATCH-041 — `src--languages--erlang-py--ElpClient::_wait_for_response`

- 人工审计: **SPEC** 错误。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::_wait_for_response.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::_wait_for_response.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::_wait_for_response.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::_wait_for_response.py`。

### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang.py

ElpClient._wait_for_response(self, request_id: int, deadline: float) -> Any

Pre-condition:
- self is an ElpClient whose underlying JSON-RPC communication channel
 is open and operational
- A request with the given integer request_id was previously transmitted
 over the same channel
- deadline is a monotonic time value representing an absolute time point
 in the same clock domain as the underlying channel's timeout mechanism

Post-condition:
- Blocks the caller, consuming messages from the underlying channel in
 order until a JSON-RPC response carrying a matching id field arrives
- A message is considered a matching response when its "id" field equals
 request_id and the message lacks a "method" field, distinguishing
 server responses from server-initiated requests and notifications
- Messages received from the server that are not the matching response
 are forwarded for server-initiated handling and do not cause the
 function to return
- When the underlying channel closes or no message arrives before
 deadline: raises TimeoutError
- When the matching response contains an "error" field:
 • If the error is a dict whose "code" field equals the transient
 content-modified error code: raises _ContentModifiedError carrying
 the error details
 • Otherwise: raises RuntimeError whose message includes a
 description of the error
- When the matching response contains no "error" field: returns the
 value of the "result" field from that response

[SPEC]
```

### Reasoner 差异

- SPEC claim:

```
- Blocks the caller, consuming messages from the underlying channel in
 order until a JSON-RPC response carrying a matching id field arrives
- A message is considered a matching response when its "id" field equals
 request_id and the message lacks a "method" field, distinguishing
 server responses from server-initiated requests and notifications
- Messages received from the server that are not the matching response
 are forwarded for server-initiated handling and do not cause the
 function to return
- When the underlying channel closes or no message arrives before
 deadline: raises TimeoutError
- When the matching response contains an "error" field:
 If the error is a dict whose "code" field equals the transient
 content-modified error code: raises _ContentModifiedError carrying
 the error details
 Otherwise: raises RuntimeError whose message includes a
 description of the error
- When the matching response contains no "error" field: returns the
 value of the "result" field from that response
```



- 推导 actual behavior:

If a JSON-RPC response message `m` with `m['id'] == request_id` and `'method' not in m` is received before the absolute monotonic time deadline, then the method either returns `m['result']` if `m` contains no `'error'` field or `'error'` is null, raises `_ContentModifiedError(m['error'])` if `m['error']` is a dict with key `'code'` equal to `_CONTENT_MODIFIED_ERROR`, or raises `RuntimeError('ELP request failed: {error}')` for any other non-null `'error'`. If no such matching response is received before deadline, `TimeoutError` is raised. During execution, every message received from the server that does not satisfy the matching condition is passed to `self._handle_server_message`, which processes it according to registered handlers, mutating the client state appropriately, and is guaranteed not to raise an exception under normal operation. The underlying communication channel remains open and operational unless one of those handlers causes a fatal exception, in which case the exception propagates uncaught. Formally, let the incoming message stream be an ordered sequence `M` of parsed message dictionaries. Let `P` be the longest prefix of `M` that does not contain a message with `'id' == request_id` and lacking `'method'`, and that is fully consumed before deadline. The execution processes all messages in `P` via `_handle_server_message`, producing intermediate state `Si`. If the first message after `P` is `m*` that satisfies the matching condition and arrives deadline, then the outcome is `return(m*.get('result'))` or raise exception as above, with the client state advanced accordingly. If no such `m*` exists before the deadline, the outcome is raise `TimeoutError`, and the state after processing `P` is the final state. In all cases, no other side effects on the `ElpClient` or its channel occur.

- Code evidence:

Line 6: `if error:`

- Trigger condition:

The code checks truthiness of the `'error'` value with `'if error:'`, which is `False` for `None` (and other false-like values). The specification requires that any matching response containing an `'error'` field raises an exception (`RuntimeError` if not the specific content-modified error). Thus, for a response with `error=null`, the code incorrectly returns the `'result'` instead of raising `RuntimeError`.

**Bug validator**

- Trigger summary: Response with `error=null` causes truthiness check to fail, returning result instead of raising `RuntimeError`
- Probe stdout:

CONFIRMED – actual returned: `'some_value'` | expected: `RuntimeError` (per spec)

**INCR-MISMATCH-042 — `src--languages--erlang-py--ElpClient::initialize`**

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::initialize.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::initialize.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::initialize.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::initialize.py`。

**完整生成 SPEC**

[SPEC]  
Unit: `src/languages/erlang.py`  
  
`ElpClient.initialize(self, bootstrap_path: str, bootstrap_source: str | None = None)`  
  
Pre-condition:  
- self is an `ElpClient` whose `__enter__` has been called (ELP subprocess is

- running, stdin/stdout pipes are open, and the JSON-RPC message reader thread is active)
- bootstrap\_path is a non-empty string identifying a filesystem path; if bootstrap\_source is None, bootstrap\_path must resolve to an existing, readable text file
- bootstrap\_source, when provided, is a string containing the document content to use instead of reading from bootstrap\_path

Post-condition:

- Completes the LSP initialization handshake: sends the "initialize" request with client capabilities, then sends the "initialized" notification
- Opens the document at bootstrap\_path on the server with the text content matching bootstrap\_source (or the file contents of bootstrap\_path when bootstrap\_source is None)
- Blocks until the server's reported status indicates it has reached a running state, or raises TimeoutError when that does not occur within self.timeout seconds measured from the call entry
- Returns the "serverInfo" sub-dict from the server's "initialize" response, or None when the response is missing, is not a dict, or does not contain a "serverInfo" key
- Raises RuntimeError when the ELP subprocess is not running (stdin unavailable) or the JSON-RPC channel encounters an unrecoverable error
- Raises TimeoutError when the server fails to reach the running state within the deadline or the subprocess stops producing messages

[SPEC]

## Reasoner 差异

- SPEC claim:

- Completes the LSP initialization handshake: sends the "initialize" request with client capabilities, then sends the "initialized" notification
- Opens the document at bootstrap\_path on the server with the text content matching bootstrap\_source (or the file contents of bootstrap\_path when bootstrap\_source is None)
- Blocks until the server's reported status indicates it has reached a running state, or raises TimeoutError when that does not occur within self.timeout seconds measured from the call entry
- Returns the "serverInfo" sub-dict from the server's "initialize" response, or None when the response is missing, is not a dict, or does not contain a "serverInfo" key
- Raises RuntimeError when the ELP subprocess is not running (stdin unavailable) or the JSON-RPC channel encounters an unrecoverable error
- Raises TimeoutError when the server fails to reach the running state within the deadline or the subprocess stops producing messages

- 推导 actual behavior:

If the method returns normally, then the return value is (R.get('serverInfo') if type(R) == dict else None) where R is the 'result' field of the JSON-RPC 'initialize' response; the 'initialize' request, the 'initialized' notification, and the 'textDocument/didOpen' notification for bootstrap\_path were all successfully sent; and the while loop processed server messages until str(self.\_status).lower() == 'running', so self.\_status indicates 'running'; all server requests received before that status were appropriately responded to, and any 'elp/status' notifications updated self.\_status. The deadline for message reception was fixed at loop entry (time.monotonic() + self.timeout). If the method raises an exception, it is either TimeoutError (from request or \_next\_message exceeding self.timeout), RuntimeError (from request error, retry exhaustion, notify failure, or reader thread exception), or IOError (from open\_document when bootstrap\_source is None and the file cannot be read), and the client state may be partially modified (e.g., 'initialize' sent but status not 'running'). Formal: (return(server\_info) exception) (R: R = request('initialize',...).result server\_info = (R.get('serverInfo') if dict(R) else None) sent('initialized') sent(didOpen(bootstrap\_path,...)) (str(self.\_status).lower() = 'running') m received\_before('running'): handled(m)). If exception e raised, then e {TimeoutError, RuntimeError, IOError} (some\_side\_effects none).

- Code evidence:

Line 26: deadline = time.monotonic() + self.timeout

- Trigger condition:

The specification requires that the timeout deadline be measured from the call entry, but the code sets it after the initialize request and open\_document, potentially allowing the method to succeed when it should time out.

#### Bug validator

- Trigger summary: Deadline computed after request() instead of at call entry; request delay extends effective timeout window beyond spec limit.
- Probe stdout:

CONFIRMED — actual deadline: 6546.858 > expected: 6546.257 (delta: 0.601s, spec requires deadline from call entry)

---

#### INCR-MISMATCH-043 — `src--languages--erlang-py--ElpClient::open_document`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::open_document.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::open_document.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::open_document.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::open_document.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang.py

ElpClient.open_document(self, path: str, source: str | None = None)

Pre-condition:
- self is an ElpClient whose underlying JSON-RPC communication channel
 is open and writable
- path is a non-empty string identifying a filesystem path
- When source is None, path must resolve to an existing, readable
 text file

Post-condition:
- Transmits a "textDocument/didOpen" notification to the ELP server
 whose textDocument field is a dict containing:
 - uri: the absolute file:// URI representing the path argument
 - languageId: "erlang"
 - version: 1
 - text: source when source is provided; otherwise the UTF-8 text
 content of the file at path
- When source is None and the file at path cannot be read, the
 underlying IOError propagates to the caller
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- Transmits a "textDocument/didOpen" notification to the ELP server
 whose textDocument field is a dict containing:
 - uri: the absolute file:// URI representing the path argument
 - languageId: "erlang"
 - version: 1
 - text: source when source is provided; otherwise the UTF-8 text
 content of the file at path
- When source is None and the file at path cannot be read, the
 underlying IOError propagates to the caller
```

- 推导 actual behavior:

```
The ElpClient's underlying JSON-RPC communication channel remains open and writable. A JSON-RPC 2.0 notification message with method "textDocument/didOpen" has been transmitted. The message params contain a single key "textDocument" whose value is a dictionary with keys: "uri" set to the URI of the resolved absolute path of `path` (i.e., `Path(path).resolve().as_uri()`), "languageId" set to "erlang", "version" set to 1, and "text" set to the string content provided by `source` if `source` was not None, or to the entire contents of the file at the resolved path read as UTF-8 text with replacement for decoding errors if `source` was None. The local variable `document` holds the resolved absolute `Path` object, and the local variable `source` holds the text content that was sent (either the original argument or the file contents). No return value is produced. Formal: (ch = self.communication_channel . is_open(ch) writable(ch)) (msg = notification(method: "textDocument/didOpen", params: { "textDocument": { uri: document.as_uri(), languageId: "erlang", version: 1, text: source_text } }) . transmitted(ch, msg)) where document = resolve(Path(path)) source_text = (if source_arg None then source_arg else read_text(document, encoding="utf-8", errors="replace")) source_arg = original `source` binding on entry.
```

- Code evidence:

```
Line 2: document = Path(path).resolve()
```

- Trigger condition:

The code uses `Path(path).resolve()` which resolves symbolic links, so the resulting URI does not represent the original path argument as required by the specification. The specification requires the absolute `file://` URI to represent the path argument, not its resolved target.

#### Bug validator

- Trigger summary: `Path.resolve()` follows symlinks, so `open_document` sends the target URI instead of the path argument URI when path is a symbolic link.
- Probe stdout:

```
CONFIRMED — actual URI: 'file:///tmp/tmpn3s6sqex/real.erl' | expected URI: 'file:///tmp/tmpn3s6sqex/link.erl'
```

---

#### INCR-MISMATCH-044 — `src--languages--erlang-py--ElpClient::request`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::request.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::request.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::request.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::request.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang.py

ElpClient.request(self, method: str, params: dict | list | None = None) -> Any

Pre-condition:
- self is an ElpClient whose underlying JSON-RPC communication channel is open and operational
- method is a non-empty string
- params, when not None, is a JSON-serializable dict or list

Post-condition:
- Transmits a JSON-RPC 2.0 request to the server with the given method
```

and params (where None params is treated as an empty object), tagged with a unique integer identifier that is strictly increasing across successive calls on the same client instance

- Blocks the caller until the server returns a response matching that identifier or until the total elapsed time since entry reaches self.timeout seconds, whichever occurs first
- On success: returns the value of the "result" field from the matching response
- When the server indicates a transient ContentModified error: re-issues the request up to a fixed maximum number of total attempts, bounded in total duration by self.timeout seconds from entry; when all attempts are exhausted without success, raises RuntimeError identifying the failing method
- When no matching response arrives before self.timeout seconds elapse from entry: raises TimeoutError
- When the server responds with an error whose semantics are not covered by the retry policy: raises RuntimeError

[SPEC]

## Reasoner 差异

- SPEC claim:

- Transmits a JSON-RPC 2.0 request to the server with the given method and params (where None params is treated as an empty object), tagged with a unique integer identifier that is strictly increasing across successive calls on the same client instance
- Blocks the caller until the server returns a response matching that identifier or until the total elapsed time since entry reaches self.timeout seconds, whichever occurs first
- On success: returns the value of the "result" field from the matching response
- When the server indicates a transient ContentModified error: re-issues the request up to a fixed maximum number of total attempts, bounded in total duration by self.timeout seconds from entry; when all attempts are exhausted without success, raises RuntimeError identifying the failing method
- When no matching response arrives before self.timeout seconds elapse from entry: raises TimeoutError
- When the server responds with an error whose semantics are not covered by the retry policy: raises RuntimeError

- 推导 actual behavior:

After execution, if `_MAX_CONTENT_MODIFIED_RETRIES > 0`, exactly one of the following outcomes occurs: (i) The method returns a value `V` such that `V` equals the 'result' field of the JSON-RPC response with matching request id, and no transient content-modified error was raised on the final attempt. (ii) A `TimeoutError` is raised, because either a `_wait_for_response` call timed out (deadline exceeded or channel closed) on any attempt, or the deadline expired during retry handling after catching a `_ContentModifiedError`. (iii) A `RuntimeError` is raised, either because a non-transient server error was received on any attempt (from `_wait_for_response`), or because all `_MAX_CONTENT_MODIFIED_RETRIES` attempts resulted in transient content-modified errors and the retry limit was reached. (iv) Any other exception raised by `_send` or `_wait_for_response` (e.g., connection errors) propagates without being caught. If `_MAX_CONTENT_MODIFIED_RETRIES == 0`, an `AssertionError` with message 'unreachable' is raised. Formally: ( `v`. return `v` `response_of_final_attempt.result = v` no `_ContentModifiedError` on final attempt) (raise `TimeoutError` ( `attempt._wait_for_response` raised `TimeoutError` on attempt (caught `_ContentModifiedError` `deadline time.monotonic() - 0`))) (raise `RuntimeError` (( `attempt._wait_for_response` raised `RuntimeError` not subclass of `_ContentModifiedError`) ( `attempts._ContentModifiedError` raised))) (raise `AssertionError` `_MAX_CONTENT_MODIFIED_RETRIES == 0`). The communication channel state remains open under normal returns; on exception its state is undefined.

- Code evidence:

```
Line 4: for attempt in range(_MAX_CONTENT_MODIFIED_RETRIES):
Line 26: raise AssertionError("unreachable")
```

- Trigger condition:

The specification only permits returning a result, raising `TimeoutError`, or raising `RuntimeError`. When `_MAX_CONTENT_MODIFIED_RETRIES` `0`, the code raises an `AssertionError`, which is not among the allowed outcomes.

#### Bug validator

- Trigger summary: Setting `_MAX_CONTENT_MODIFIED_RETRIES` to `0` causes `range(0)` to produce zero iterations, falling through to raise `AssertionError("unreachable")` which violates the spec's allowed outcomes (`return`, `TimeoutError`, `RuntimeError`).
- Probe stdout:

CONFIRMED – `AssertionError` raised (spec violation): `unreachable`

### INCR-MISMATCH-045 — `src--languages--erlang-py--_SourceIndex::build`

- 人工审计: 实现缺陷候选。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/erlang-py/_SourceIndex::build.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_SourceIndex::build.json`。
- 详细报告: `src--languages--erlang-py--_SourceIndex::build.md`。
- Probe: `probe_src--languages--erlang-py--_SourceIndex::build.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang-py/_SourceIndex.py

_SourceIndex.build(cls, source: str) -> _SourceIndex

Pre-condition:
- source is a string

Post-condition:
- Returns a _SourceIndex instance whose content is derived solely from source
- The returned index represents source as an ordered sequence of lines,
 where line boundaries correspond to newline character positions in source
- For each line, the byte offset of its first character within source is
 computable from the returned index
- The total number of bytes across all lines in the returned index equals
 the length of source
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- Returns a _SourceIndex instance whose content is derived solely from source
- The returned index represents source as an ordered sequence of lines,
 where line boundaries correspond to newline character positions in source
- For each line, the byte offset of its first character within source is
 computable from the returned index
- The total number of bytes across all lines in the returned index equals
 the length of source
```

- 推导 actual behavior:

The method returns an instance of ``cls`` (expected to be ``_SourceIndex`` or a subclass) with three attributes: ``source`` equals the original input string ``source``, ``lines`` is the list of strings obtained by calling ``source.splitlines(keepends=True)``, and ``line_offsets`` is a list of integers where each element represents the character offset of the start of the corresponding line in the

original source. Formally, let `r` be the returned object. Then `r.source == source`, `r.lines == source.splitlines(keepends=True)`, `len(r.line_offsets) == len(r.lines)`, and for all `i`, `0 ≤ i < len(r.lines)`, `r.line_offsets[i] == ∑j=0i-1 len(r.lines[j])` (with the sum defined as 0 when `i=0`). No exceptions are raised and no other side effects occur.

- Code evidence:

```
Line 2: lines = source.splitlines(keepends=True)
```

- Trigger condition:

The specification requires line boundaries to correspond only to newline character positions. For `source='hello\vworld'`, the vertical tab (`\v`) is not a standard newline character, but `splitlines(keepends=True)` splits on it, creating two lines (`'hello\v'` and `'world'`) and computing line offsets accordingly. This violates the requirement that the returned index represents source as an ordered sequence of lines where line boundaries correspond to newline positions.

#### Bug validator

- Trigger summary: For `source='hello\vworld'`, `splitlines(keepends=True)` incorrectly splits on vertical tab (`\v`), which is not a newline character, producing 2 lines instead of 1.
- Probe stdout:

```
CONFIRMED — actual lines: 2 | expected lines: 1
lines: ['hello\x0b', 'world']
line_offsets: [0, 6]
source: 'hello\x0bworld'
```

---

#### INCR-MISMATCH-046 — `src--languages--erlang-py--_analyze_project_uncached`

- 人工审计: 推理误判。
- Validator: **not\_confirmed**; 尝试次数: 3。
- 原始函数与 SPEC: `src/languages/erlang-py/_analyze_project_uncached.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_analyze_project_uncached.json`。
- 详细报告: `src--languages--erlang-py--_analyze_project_uncached.md`。
- Probe: `probe_src--languages--erlang-py--_analyze_project_uncached.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang-py/_analyze_project_uncached.py

_analyze_project_uncached(proj_dir: str) -> ErlangAnalysis

Pre-condition:
- proj_dir is a non-empty string representing a filesystem path

Post-condition:
- Returns an ErlangAnalysis object whose .functions attribute is a dict
 mapping each .erl file absolute path to a list of (function_id, source_text)
 tuples, where function_id is a canonical string identifier and source_text
 is the source code of that function
- Returns an ErlangAnalysis whose .edges attribute is a dict mapping
 (function_id, caller_module) tuples to sets of callee function_ids
- Returns an ErlangAnalysis whose .spans attribute is a dict mapping each
 .erl file absolute path to a list of (function_id, start_line, end_line)
 tuples, where start_line and end_line are 1-based inclusive line numbers
- Returns an ErlangAnalysis whose .server_info attribute is populated from
 the ELP server initialization response
- When no .erl files exist under the directory tree rooted at proj_dir after
 resolution to an absolute path, returns an ErlangAnalysis with all three
 dict attributes empty
- Raises an exception when the ELP backend process cannot be started, the LSP
```

communication channel fails, or the project at proj\_dir cannot be analyzed  
[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns an ErlangAnalysis object whose .functions attribute is a dict mapping each .erl file absolute path to a list of (function\_id, source\_text) tuples, where function\_id is a canonical string identifier and source\_text is the source code of that function
- Returns an ErlangAnalysis whose .edges attribute is a dict mapping (function\_id, caller\_module) tuples to sets of callee function\_ids
- Returns an ErlangAnalysis whose .spans attribute is a dict mapping each .erl file absolute path to a list of (function\_id, start\_line, end\_line) tuples, where start\_line and end\_line are 1-based inclusive line numbers
- Returns an ErlangAnalysis whose .server\_info attribute is populated from the ELP server initialization response
- When no .erl files exist under the directory tree rooted at proj\_dir after resolution to an absolute path, returns an ErlangAnalysis with all three dict attributes empty
- Raises an exception when the ELP backend process cannot be started, the LSP communication channel fails, or the project at proj\_dir cannot be analyzed

- 推导 actual behavior:

The function returns an ErlangAnalysis object when it terminates normally; otherwise a TimeoutError or RuntimeError is raised and no value is returned.

**\*\*Normal termination (no exception)\*\***

1. If `proj\_dir` contains no `.erl` files (recursively), the function returns `ErlangAnalysis(functions={}, edges={})`. No ElpClient is started, no files are read.
2. If there is at least one `.erl` file,
  - All `.erl` files under `os.path.abspath(proj\_dir)` are collected into `files`; their contents are read into the `sources` dict (UTF-8, errors replaced).
  - An ElpClient is created and entered, starting the language-server subprocess; after the `with` block the subprocess is stopped and the client closed.
  - The server is initialised with the first file and its source; all other files are opened via `open\_document`.
  - For each file:
    - \* A source index is built.
    - \* The caller module name is determined.
    - \* Document symbols are requested from the server.
    - \* Function symbols (kind == FUNCTION\_KIND) with a valid range are processed. Malformed or duplicate symbols (by canonical function ID within the same file) are silently skipped. A warning is logged for malformed ones.
    - \* The canonical function ID is obtained from the symbol's URI and name. Valid symbols produce tuples added to the `functions` dictionary: `functions[function\_id]` becomes a nonempty list of `(caller\_module, source\_text\_of\_function)` where the source text is extracted from the original file using the range of the function definition.
    - \* `edges` and `spans` are populated similarly. `spans` maps a file path to `[(function\_name, start\_line, end\_line)]` for every recognised function, and `edges` captures callercallee relationships.
  - After all files have been processed, the function returns `ErlangAnalysis(functions=functions, edges=edges, spans=spans)`.
  - All file reads and server communications succeeded; no unhandled ValueError occurs because the `\_function\_id` ValueError is caught and ignored.

**\*\*Exception case\*\***

An unhandled `TimeoutError` or `RuntimeError` may be raised during server initialisation or symbol requests. In that case the `with` block still ensures the ElpClient is cleaned up (server subprocess terminated), but the function does not return a value.

**\*\*Formal logic\*\***

Let P be the input `proj\_dir`,  $F = \{ f \mid \_erlang\_files(os.path.abspath(P)).f \}$ , and R the returned value when no exception occurs.

- R ErlangAnalysis.
- If  $F = \emptyset$  then  $R.functions = \{\}$   $R.edges = \{\}$ .
- If  $F \neq \emptyset$  then:
  - $functions, edges, spans$  such that  $R.functions = functions$   $R.edges = edges$   $R.spans = spans$
  - $fid$  keys(functions) (functions[fid] is a list L L [] (cm, src) L, cm =  $\_caller\_module(path)$  for the file containing fid, and src = source index substring for that function's range).
  - $path$  F,  $spans[path] =$  a list of tuples (name, l1, l2) corresponding to defined functions in that file.
  - (edges reflects static call relations extracted from the LSP symbols).



The server subprocess was running during the with block and has been terminated before the return.

- Code evidence:

```
Line 5: return ErlangAnalysis(functions={}, edges={})
```

- Trigger condition:

When no .erl files exist the specification requires the returned ErlangAnalysis to have all three dict attributes (functions, edges, spans) empty; the code returns an object without a spans attribute, violating the specification.

#### Bug validator

- Trigger summary: When no .erl files exist, ErlangAnalysis(functions={}, edges={}) still has spans={} via dataclass field(default\_factory=dict) — false positive from logic verifier analyzing extracted function without seeing class defaults.
- Probe stdout:

```
WARNING:root:ELP Erlang analysis unavailable for /tmp/tmp8yhcvxf: simulated failure
NOT CONFIRMED - ErlangAnalysis(functions={}, edges={}) asdict shows all three: functions=True,
edges=True, spans=True. Fallback path also correct: functions=True, edges=True, spans=True.
DataClass field(default_factory=dict) always provides spans={}.
```

---

#### INCR-MISMATCH-047 — `src--languages--erlang-py--_elp_argv`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/_elp_argv.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_elp_argv.json`。
- 详细报告: `src--languages--erlang-py--_elp_argv.md`。
- Probe: `probe_src--languages--erlang-py--_elp_argv.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang-py/_elp_argv.py

_elp_argv() -> list[str]

Pre-condition:
- The process environment is available for reading

Post-condition:
- Returns a non-empty list of strings representing the argument vector used to
 launch the Erlang Language Platform server subprocess
- The last element of the returned list is the string "server"
- The returned value is deterministic across calls: given an unchanged value of
 the ELP_COMMAND environment variable and the same operating-system platform
 (POSIX vs non-POSIX), repeated calls return the same list
- When the ELP_COMMAND environment variable changes, the returned list reflects
 the new command
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

- Returns a non-empty list of strings representing the argument vector used to launch the Erlang Language Platform server subprocess
- The last element of the returned list is the string "server"
- The returned value is deterministic across calls: given an unchanged value of the ELP\_COMMAND environment variable and the same operating-system platform (POSIX vs non-POSIX), repeated calls return the same list
- When the ELP\_COMMAND environment variable changes, the returned list reflects the new command

- 推导 actual behavior:

The function returns a list of strings, specifically the result of `(shlex.split(settings.erlang.command.strip() or 'elp', posix=(os.name != 'nt')) or ['elp']) + ['server']`. In natural language: the return value `r` is a nonempty list whose last element is 'server'. The prefix list is obtained by taking the stripped value of `settings.erlang.command`, defaulting to 'elp' if empty, splitting it into tokens via `shlex.split` (with `posix` mode true when `os.name` is not 'nt'), and using `['elp']` if that split yields an empty list. Formally: `r = _elp_argv() r = (let c = settings.erlang.command.strip() in let cmd = c if c != '' else 'elp' in let parts = shlex.split(cmd, posix=(os.name != 'nt')) in (parts if parts else ['elp'])) + ['server']`.

- Code evidence:

```
Line 2: command = settings.erlang.command.strip() or "elp"
```

- Trigger condition:

When `ELP_COMMAND` is not set, `settings.erlang.command` may be `None`. Calling `.strip()` on `None` raises `AttributeError`, so the function does not return a list of strings as required by the specification.

#### Bug validator

- Trigger summary: When `settings.erlang.command` is `None`, calling `.strip()` raises `AttributeError` instead of returning a valid `argv` list as required by the specification.
- Probe stdout:

```
CONFIRMED - AttributeError raised when command is None: 'NoneType' object has no attribute 'strip'
```

---

#### INCR-MISMATCH-048 — `src--languages--erlang-py--_source_for_range`

- 人工审计: 推理误判。
- Validator: **not\_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/_source_for_range.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_source_for_range.json`。
- 详细报告: `src--languages--erlang-py--_source_for_range.md`。
- Probe: `probe_src--languages--erlang-py--_source_for_range.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang-py/_source_for_range.py

_source_for_range(source, lsp_range) -> str

Pre-condition:
- source is a string containing source code text
```

- `lsp_range` is a dict with keys 'start' and 'end', each being a dict with keys 'line' (int, 0-based) and 'character' (int, 0-based)

Post-condition:

- Returns the substring of source that spans from the start position (inclusive) to the end position (exclusive)  
[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns the substring of source that spans from the start position (inclusive) to the end position (exclusive)

- 推导 actual behavior:

If `lsp_range['start']` does not follow `lsp_range['end']` in source order, the function returns the substring of `source` that begins at the byte offset corresponding to the inclusive start position (line `lsp_range['start']['line']`, character `lsp_range['start']['character']`) and ends at the byte offset corresponding to the exclusive end position (line `lsp_range['end']['line']`, character `lsp_range['end']['character']`). If the start position does follow the end position, the behavior is undefined: the function may raise an exception (e.g., an `AssertionError` or `ValueError`) or return an arbitrary result.

- Code evidence:

Line 3: `return _SourceIndex.build(source).source_for_range(lsp_range)`

- Trigger condition:

The specification requires returning the substring from start to end for any input, but the code's behavior is undefined when start follows end. For the counterexample, start character 2 > end character 1. The code may raise an exception or return an arbitrary result, while the specification implies returning an empty substring (the span from start to end exclusive when start > end is empty). Thus the code violates the specification.

## Bug validator

- Trigger summary: LSP range with start character 2 > end character 1 on same line; Python slice `source[start:end]` returns empty string, matching spec-implied empty span.
- Probe stdout:

NOT CONFIRMED — actual matched expected: ''

---

## INCR-MISMATCH-049 — `src--languages--erlang-py--_timeout_seconds`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/_timeout_seconds.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_timeout_seconds.json`。
- 详细报告: `src--languages--erlang-py--_timeout_seconds.md`。
- Probe: `probe_src--languages--erlang-py--_timeout_seconds.py`。

完整生成 SPEC

```
[SPEC]
Unit: src/languages/erlang-py/_timeout_seconds.py

_timeout_seconds() -> int

Pre-condition:
- None (the function requires no arguments)

Post-condition:
- Returns an integer ≥ 1 representing the maximum number of seconds to wait for
 a single LSP operation
- When the environment variable `ELP_TIMEOUT_SECONDS` is set to a string
 representing a decimal integer n, the returned value is n when $n \geq 1$, and 1
 when $n < 1$
- When `ELP_TIMEOUT_SECONDS` is not set, or is set to a string that is not a
 valid decimal integer representation, the returned value is the built-in
 default constant `_DEFAULT_TIMEOUT_SECONDS`
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns an integer 1 representing the maximum number of seconds to wait for
 a single LSP operation
- When the environment variable `ELP_TIMEOUT_SECONDS` is set to a string
 representing a decimal integer n, the returned value is n when $n \geq 1$, and 1
 when $n < 1$
- When `ELP_TIMEOUT_SECONDS` is not set, or is set to a string that is not a
 valid decimal integer representation, the returned value is the built-in
 default constant `_DEFAULT_TIMEOUT_SECONDS`
```

- 推导 actual behavior:

The function returns an integer, which is always at least 1. Formally: `isinstance(result, int)`  
`result >= 1`.

- Code evidence:

```
Line 1: def _timeout_seconds() -> int:
Line 4: return max(1, settings.erlang.timeout_s)
```

- Trigger condition:

The specification (B) mandates that when `ELP_TIMEOUT_SECONDS` is missing or invalid, the function must return `_DEFAULT_TIMEOUT_SECONDS`. The code never reads the environment variable and never references `_DEFAULT_TIMEOUT_SECONDS`; it only returns `max(1, settings.erlang.timeout_s)`. Even if the config loader normally sets the default, the function's own logic does not enforce the specified fallback, so there exist states (e.g., `settings.erlang.timeout_s` `_DEFAULT_TIMEOUT_SECONDS`) where the output violates B.

## Bug validator

- Trigger summary: `settings.erlang.timeout_s` diverged from `_DEFAULT_TIMEOUT_SECONDS` (30 vs 180) with `ELP_TIMEOUT_SECONDS` unset; function returns settings value instead of spec-required default constant
- Probe stdout:

CONFIRMED — actual: 30 | expected: 180

## INCR-MISMATCH-050 — src--languages--go-py--batch\_extract

- 人工审计：契约待确认。
- Validator： **confirmed**；尝试次数： 1。
- 原始函数与 SPEC： `src/languages/go-py/batch_extract.py`。
- Reasoner 结果： `logic_verification_results/src/languages/go-py/batch_extract.json`。
- 详细报告： `src--languages--go-py--batch_extract.md`。
- Probe： `probe_src--languages--go-py--batch_extract.py`。

### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/go-py/batch_extract.py

batch_extract(proj_dir) -> Dict[str, List[Tuple[str, str]]]

Pre-condition:
- proj_dir is a non-empty string path to a project root containing Go source files

Post-condition:
- Returns a dict whose keys are absolute file paths (strings) of Go source files
 and whose values are lists of (func_name, body) tuples for all function
 definitions extracted from each file
- func_name is a string containing the canonicalized function identifier; body is
 a string containing the full source text of the function definition
- Returns an empty dict {} when no codegraph backend is available for Go
- Only .go source files within proj_dir are processed
[SPEC]
```

### Reasoner 差异

- SPEC claim:

```
- Returns a dict whose keys are absolute file paths (strings) of Go source files
 and whose values are lists of (func_name, body) tuples for all function
 definitions extracted from each file
- func_name is a string containing the canonicalized function identifier; body is
 a string containing the full source text of the function definition
- Returns an empty dict {} when no codegraph backend is available for Go
- Only .go source files within proj_dir are processed
```

- 推导 actual behavior:

```
If CodeGraphExtractor.from_proj_dir(proj_dir) returns None, the function returns an empty
dictionary {}. Otherwise, let cg be the returned instance; the function returns
cg.get_functions_by_file('go', proj_dir), which is a dict mapping absolute file paths to lists
of (func_name, body) tuples with each body ending with newline, tuples ordered by ascending line
number, duplicate names disambiguated with numeric suffixes, unreadable source files silently
skipped, and an empty dict if 'go' is not a recognized language. Formally: ret = ({ } if cg is
None else cg.get_functions_by_file('go', proj_dir)).
```

- Code evidence:

```
Line 4: return cg.get_functions_by_file("go", proj_dir) if cg else {}
```

- Trigger condition:

The code's behavior does not guarantee that only .go files within proj\_dir are processed; get\_functions\_by\_file may return file paths outside proj\_dir, violating the specification.

#### Bug validator

- Trigger summary: When the codegraph database contains `file_path` entries with parent-directory traversal (`../`), `get_functions_by_file` resolves them via `os.path.join` without validating that the path remains within `proj_dir`, causing files outside `proj_dir` to appear in the returned dict.
- Probe stdout:

CONFIRMED – spec requires only files within proj\_dir, but returned keys include paths outside proj\_dir: ['tmp/probe\_batch\_go\_p07ik0lo/project/../outside/evil.go']. Inside keys: ['tmp/probe\_batch\_go\_p07ik0lo/project/pkg/handler.go']

---

#### INCR-MISMATCH-051 — `src--languages--go-py--function_spans`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/go-py/function_spans.py`。
- Reasoner 结果: `logic_verification_results/src/languages/go-py/function_spans.json`。
- 详细报告: `src--languages--go-py--function_spans.md`。
- Probe: `probe_src--languages--go-py--function_spans.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/go-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list | None

Pre-condition:
- proj_dir is a non-empty string referencing a project directory.
- filepath is a string identifying a Go source file within that project.

Post-condition:
- Returns a list of (function_name, start_line, end_line) tuples for every
 top-level function definition found in the file at filepath.
- start_line and end_line are 0-indexed and inclusive.
- The returned list is ordered by function occurrence within the file.
- Returns None when the codegraph backend is unavailable or does not index
 the file at filepath.
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- Returns a list of (function_name, start_line, end_line) tuples for every
 top-level function definition found in the file at filepath.
- start_line and end_line are 0-indexed and inclusive.
- The returned list is ordered by function occurrence within the file.
- Returns None when the codegraph backend is unavailable or does not index
 the file at filepath.
```

- 推导 actual behavior:

Natural: If `CodeGraphExtractor.from_proj_dir(proj_dir)` returns a falsy value (e.g., `None` because the codegraph could not be loaded), the function immediately returns `None`. Otherwise, it calls the obtained instance's `get_function_spans("go", filepath)` and returns its result. That result is `None` when the codegraph does not index the given file or the file contains no function

definitions, or when the preconditions of `get_function_spans` are violated (e.g., `filepath` is not an absolute path, or the language key is not recognised). If the internal preconditions hold, the return value is a nonNone list of 3tuples (`name`, `start_idx`, `end_idx`) where `name` is a classqualified function/method identifier, `start_idx` and `end_idx` are 0indexed inclusive line numbers converted from the backend, and the list is sorted by ascending `start_idx`. No exceptions are intentionally raised; all error conditions are signalled through the None return value. Formal: Let `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`. If not `cg`: return None. Else: `result = cg.get_function_spans("go", filepath)`. The final return value satisfies: (`result = None`) ( (`lang_key "go"` is supported `filepath` is an absolute path inside the project root) (`result` is a list `L` `i : L[i] = (n_i, s_i, e_i)` `n_i` String `s_i, e_i` 0 `s_i e_i` (`j < i : s_j s_i`) each `n_i` is a classqualified function/method name in the file indexed by `filepath`) ). If those additional conditions are not met, `result` may be None or have an unspecified structure reflecting the internal implementation.

- Code evidence:

```
Line 8: return cg.get_function_spans("go", filepath) if cg else None
```

- Trigger condition:

The code hardcodes the language key 'go', causing it to return None for non-Go files that are nevertheless indexed by the codegraph and contain top-level function definitions, violating the specification that requires returning the list of definitions for any such file.

#### Bug validator

- Trigger summary: `function_spans` hardcodes the language key 'go' in the call to `get_function_spans`, causing it to return None for non-Go files that are indexed by the codegraph and contain function definitions.
- Probe stdout:

```
CONFIRMED — actual: None | expected: [('my_func', 0, 5)]
The hardcoded 'go' language key caused get_function_spans to miss the python-language node,
returning None instead of the indexed function spans.
```

## src/languages/javascript-py

### INCR-MISMATCH-052 — src--languages--javascript-py--function\_spans

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/javascript-py/function_spans.py`。
- Reasoner 结果: `logic_verification_results/src/languages/javascript-py/function_spans.json`。
- 详细报告: `src--languages--javascript-py--function_spans.md`。
- Probe: `probe_src--languages--javascript-py--function_spans.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/javascript-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list[tuple] | None

Pre-condition:
- proj_dir is a path to an existing project directory.
- filepath is a path to a JavaScript source file within the project.

Post-condition:
- Returns None when the codegraph backend is unavailable for the project, or when the
 backend exists but does not index the given file.
```

```
- Otherwise returns a list of (name, start_idx, end_idx) tuples, one per function defined in the file.
- start_idx and end_idx are 0-indexed inclusive line numbers.
- The list is ordered by appearance (ascending start_idx).
- The list is empty when no functions are defined in the file.
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns None when the codegraph backend is unavailable for the project, or when the backend exists but does not index the given file.
- Otherwise returns a list of (name, start_idx, end_idx) tuples, one per function defined in the file.
- start_idx and end_idx are 0-indexed inclusive line numbers.
- The list is ordered by appearance (ascending start_idx).
- The list is empty when no functions are defined in the file.
```

- 推导 actual behavior:

```
If CodeGraphExtractor.from_proj_dir(proj_dir) returns None, function_spans returns None. Otherwise, let cg be that returned CodeGraphExtractor; if cg.get_function_spans("javascript", filepath) returns None, then function_spans returns None; otherwise, it returns the list of (name, start_idx, end_idx) tuples with 0-indexed inclusive line numbers. Formally: let R = function_spans(proj_dir, filepath), C = CodeGraphExtractor.from_proj_dir(proj_dir). Then (C = None R = None) ((C None C.get_function_spans("javascript", filepath) = None) R = None) ((C None C.get_function_spans("javascript", filepath) None) R = C.get_function_spans("javascript", filepath)).
```

- Code evidence:

```
Line 8: return cg.get_function_spans("javascript", filepath) if cg else None
```

- Trigger condition:

The specification requires the returned list to be ordered by ascending start\_idx. The code simply returns the raw list from cg.get\_function\_spans without sorting. The post-condition of get\_function\_spans does not guarantee ordering, so a valid input where the codegraph backend returns an unordered list leads to a specification violation.

## Bug validator

- Trigger summary: function\_spans returns the raw list from cg.get\_function\_spans without sorting by start\_idx; when the backend returns unordered spans the spec's ascending-start\_idx guarantee is violated.
- Probe stdout:

```
CONFIRMED — actual: [('func_c', 40, 52), ('func_a', 5, 18), ('func_b', 22, 35)] | expected: [('func_a', 5, 18), ('func_b', 22, 35), ('func_c', 40, 52)]
```

# src/languages/rust-py

## INCR-MISMATCH-053 — src--languages--rust-py--batch\_extract

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: [src/languages/rust-py/batch\\_extract.py](#)。



- Reasoner 结果: `logic_verification_results/src/languages/rust-py/batch_extract.json`。
- 详细报告: `src--languages--rust-py--batch_extract.md`。
- Probe: `probe_src--languages--rust-py--batch_extract.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/languages/rust-py/batch_extract.py

batch_extract(proj_dir) -> dict

Pre-condition:
- proj_dir is a string representing a filesystem path to a project directory

Post-condition:
- Returns a dictionary where each key is an absolute filesystem path (str) to a Rust source
 file located within or under the project directory
- Each value is a non-empty list of (str, str) tuples: the first element is a function name
 declared in that file, and the second element is the complete source text of the function
 body
- A source file containing N detected functions produces N entries in its value list
- Returns an empty dictionary when no Rust codegraph backend is available for the given
 project
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns a dictionary where each key is an absolute filesystem path (str) to a Rust source
 file located within or under the project directory
- Each value is a non-empty list of (str, str) tuples: the first element is a function name
 declared in that file, and the second element is the complete source text of the function
 body
- A source file containing N detected functions produces N entries in its value list
- Returns an empty dictionary when no Rust codegraph backend is available for the given
 project
```

- 推导 actual behavior:

The function returns a dictionary mapping absolute file paths of Rust source files in the project directory `proj\_dir` to lists of their contained function definitions as `(function\_name: str, function\_body: str)` tuples. If `CodeGraphExtractor.from\_proj\_dir(proj\_dir)` fails (returns `None`), the function returns an empty dictionary `{}`. If the extractor initializes successfully, the dictionary may still be empty if no Rust sources are found or all such files are unreadable. Formally: `result = batch\_extract(proj\_dir) result dict ( (CodeGraphExtractor.from\_proj\_dir(proj\_dir) = None result = {}) (CodeGraphExtractor.from\_proj\_dir(proj\_dir) None (k keys(result), k is an absolute path of a readable Rust file in proj\_dir result[k] is a list of (name, body) pairs for functions in that file (f in projects Rust sources, if f is readable then entry in result with key abs(f) and value that list, else f is omitted)))`.

- Code evidence:

```
Line 4: return cg.get_functions_by_file("rust", proj_dir) if cg else {}
```

- Trigger condition:

The specification requires each value in the returned dictionary to be a non-empty list of `(function\_name, function\_body)` tuples. The code directly returns the result of `get\_functions\_by\_file`, which can include entries mapping a readable file to an empty list when no functions are detected. This violates the non-empty requirement.

- Trigger summary: `batch_extract` passes through empty-list values from `get_functions_by_file` without filtering, violating the spec's requirement that each value be a non-empty list of tuples.
- Probe stdout:

```
CONFIRMED – batch_extract does not filter empty-list values. File '/fake/proj/src/empty_mod.rs'
maps to [] but spec requires Each value must be a non-empty list of (function_name,
function_body) tuples
```

## INCR-MISMATCH-054 — `src--languages--rust-py--function_spans`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/rust-py/function_spans.py`。
- Reasoner 结果: `logic_verification_results/src/languages/rust-py/function_spans.json`。
- 详细报告: `src--languages--rust-py--function_spans.md`。
- Probe: `probe_src--languages--rust-py--function_spans.py`。

### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/rust.py

function_spans(proj_dir: str, filepath: str) -> list[tuple[str, int, int]] | None

Pre-condition:
- proj_dir is a filesystem path to a project directory
- filepath is a path to a single Rust source file within the project

Post-condition:
- If codegraph is available and indexes filepath: returns a list of
 (function_name, start_line, end_line) tuples, one per top-level function
 declared in the file. Each start_line and end_line is a 0-indexed
 inclusive line number bounding the function's source span.
- If filepath contains no top-level function declarations: returns an
 empty list.
- If codegraph is unavailable or does not index filepath: returns None.
 A None return signals the caller to fall back to regex-based extraction.
[SPEC]
```

### Reasoner 差异

- SPEC claim:

```
- If codegraph is available and indexes filepath: returns a list of
 (function_name, start_line, end_line) tuples, one per top-level function
 declared in the file. Each start_line and end_line is a 0-indexed
 inclusive line number bounding the function's source span.
- If filepath contains no top-level function declarations: returns an
 empty list.
- If codegraph is unavailable or does not index filepath: returns None.
 A None return signals the caller to fall back to regex-based extraction.
```

- 推导 actual behavior:

```
If the call `CodeGraphExtractor.from_proj_dir(proj_dir)` raises an exception, `function_spans`
raises that exception. Otherwise, let `cg` be the returned value. If `cg` is `None`, the
function returns `None`. If `cg` is a `CodeGraphExtractor` instance, then upon evaluating
`cg.get_function_spans('rust', filepath)`: if that call raises an exception, `function_spans`
raises that exception; else the function returns the result, which is either `None` (when the
language key 'rust' is not recognized by the backend or the database contains no entries for
`filepath`) or a list of `(name, start_idx, end_idx)` tuples for each function and method
definition found in `filepath`, with 0-indexed inclusive line indices, ordered by ascending
```

`start\_idx`. The function does not modify any externally observable state beyond any internal state initialized during `from\_proj\_dir` and the read-only access to the codegraph backend.

- Code evidence:

```
Line 8: return cg.get_function_spans("rust", filepath) if cg else None
```

- Trigger condition:

The code returns the unfiltered list from `cg.get_function_spans`, which includes all function and method definitions (as documented). The specification requires only top-level function declarations; method definitions inside `impl` blocks must be excluded. This input contains both a top-level function and a method, causing the code to produce an output that includes the method, violating the requirement.

#### Bug validator

- Trigger summary: `function_spans` delegates to `get_function_spans` which returns both 'function' and 'method' kinds, but the spec requires only top-level function declarations; methods inside `impl` blocks leak through unfiltered.
- Probe stdout:

```
CONFIRMED — function_spans returned method 'Foo::bar' (inside an impl block) in addition to top-level function 'top_level'. Spec requires only top-level functions. Full result: [('top_level', 0, 2), ('Foo::bar', 4, 6)]
```

## src/languages/typescript-py

### INCR-MISMATCH-055 — src--languages--typescript-py--batch\_extract

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/typescript-py/batch_extract.py`。
- Reasoner 结果: `logic_verification_results/src/languages/typescript-py/batch_extract.json`。
- 详细报告: `src--languages--typescript-py--batch_extract.md`。
- Probe: `probe_src--languages--typescript-py--batch_extract.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/languages/typescript.py

batch_extract(proj_dir: str) -> dict[str, list[tuple[str, str]]]

Pre-condition:
- proj_dir is a filesystem path to a project directory

Post-condition:
- If codegraph is available: returns a dict whose keys are absolute file
 paths to TypeScript source files within proj_dir, and whose values are
 lists of (function_name, function_body) tuples for every top-level
 function declared in the corresponding file.
- Each function_name is the identifier of the function declaration.
- Each function_body is the full source text of the function definition.
- If proj_dir contains no TypeScript files with top-level functions:
 returns an empty dict.
- If codegraph is unavailable: returns an empty dict {}.

[SPEC]
```

- SPEC claim:

```
- If codegraph is available: returns a dict whose keys are absolute file
 paths to TypeScript source files within proj_dir, and whose values are
 lists of (function_name, function_body) tuples for every top-level
 function declared in the corresponding file.
- Each function_name is the identifier of the function declaration.
- Each function_body is the full source text of the function definition.
- If proj_dir contains no TypeScript files with top-level functions:
 returns an empty dict.
- If codegraph is unavailable: returns an empty dict {}.
```

- 推导 actual behavior:

The function returns a dictionary. Let `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`. If `cg` is `None`, the result is the empty dictionary `{}`. Otherwise, the result is `cg.get_functions_by_file("typescript", proj_dir)`. Formally: `result = {}` (`cg` `None` `result = cg.get_functions_by_file("typescript", proj_dir)`). The values of `result`, if any, satisfy: for every key `k` (`str`), `result[k]` is a list of tuples; each tuple (`name: str`, `body: str`) represents a TypeScript function. The result may be empty for any of the following reasons: `CodeGraphExtractor` initialization failed, the language "typescript" is not recognized, or no TypeScript source files with extractable functions were found under `proj_dir`.

- Code evidence:

```
Line 4: return cg.get_functions_by_file("typescript", proj_dir) if cg else {}
```

- Trigger condition:

The specification (B) states values must contain only toplevel function definitions. The implementation delegates to ``get_functions_by_file``, whose documented postcondition does not restrict results to toplevel functions. Consequently, a project directory with a file containing nested functions will produce an output that includes those nested declarations, failing requirement B.

Bug validator

- Trigger summary: `batch_extract` delegates to `get_functions_by_file` which returns ALL functions including nested ones, but the spec requires only top-level function definitions.
- Probe stdout:

```
CONFIRMED – batch_extract returns nested functions (violates spec).
Actual names: ['exportData', 'formatItem']
Expected names: ['exportData']
The nested function 'formatItem' should not appear per spec claim.
```

---

## src/llm\_client-py

### INCR-MISMATCH-056 — src--llm\_client-py--\_inject\_targets

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/llm_client-py/_inject_targets.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_inject_targets.json`。
- 详细报告: `src--llm_client-py--_inject_targets.md`。

- Probe: `probe_src--llm_client-py--_inject_targets.py`.

## 完整生成 SPEC

```
[SPEC]
Unit: src/llm_client.py

_inject_targets() -> list[str]

Pre-condition:
- None (no arguments required)

Post-condition:
- Returns a list of non-empty strings representing the configured targets for
 user-id metadata injection into request bodies
- The returned values are determined by the INJECT_HOST environment variable:
 when set, the value is parsed as a comma-separated list, each segment is
 stripped of leading and trailing whitespace, and any resulting empty segments
 are discarded
- When INJECT_HOST is unset or empty, returns an empty list
- The relative order of elements in the returned list matches the order of
 their corresponding segments in INJECT_HOST
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns a list of non-empty strings representing the configured targets for
 user-id metadata injection into request bodies
- The returned values are determined by the INJECT_HOST environment variable:
 when set, the value is parsed as a comma-separated list, each segment is
 stripped of leading and trailing whitespace, and any resulting empty segments
 are discarded
- When INJECT_HOST is unset or empty, returns an empty list
- The relative order of elements in the returned list matches the order of
 their corresponding segments in INJECT_HOST
```

- 推导 actual behavior:

Natural language: The function `_inject_targets()` accesses the global or module-level object ``settings.inject.hosts``. If that attribute chain exists, it returns a list of non-empty strings. The value of ``settings.inject.hosts`` is first passed through the expression ``(settings.inject.hosts or '')``, so if it is `None`, an empty string, or any other falsy value, an empty string is used instead. That resulting string is split on commas, each part is stripped of leading and trailing whitespace, and any part that becomes empty after stripping is discarded. The returned list contains the stripped non-empty parts. If the attribute chain does not exist (i.e., ``settings``, ``settings.inject``, or ``settings.inject.hosts`` is not defined), an `AttributeError` is raised. Formal logic: Let `S = settings.inject.hosts` if `settings`, `settings.inject`, and `settings.inject.hosts` exist; otherwise `S` is undefined. If `S` is defined, then `result = COMPREHENSION[ s.strip() | for each s in (S or '').split(',') if s.strip() != '' ]`. If `S` is undefined, the function raises `AttributeError`. In the success case, `e` result, `e` is a string and `len(e) > 0`.

- Code evidence:

```
Line 2: return [s.strip() for s in (settings.inject.hosts or "").split(",") if s.strip()]
```

- Trigger condition:

The function reads the host list from `settings.inject.hosts`, but the specification requires reading from the `INJECT_HOST` environment variable. When `INJECT_HOST` is set but `settings.inject.hosts` is absent or contains different data, the function either raises an error or returns an incorrect list, violating the specification.

## Bug validator

- Trigger summary: INJECT\_HOST env var is set to comma-separated hosts but settings.inject.hosts is empty; function returns [] instead of the parsed env var values
- Probe stdout:

```
CONFIRMED – reads from settings.inject.hosts instead of INJECT_HOST env var | actual: [] |
expected: ['alpha', 'beta', 'gamma']
```

## INCR-MISMATCH-057 — `src--llm_client-py--_messages_to_anthropic`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/llm_client-py/_messages_to_anthropic.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_messages_to_anthropic.json`。
- 详细报告: `src--llm_client-py--_messages_to_anthropic.md`。
- Probe: `probe_src--llm_client-py--_messages_to_anthropic.py`。

### 完整生成 SPEC

```
[SPEC]
Unit: src/llm_client.py

_messages_to_anthropic(messages) -> (str, list)

Pre-condition:
- messages is a list of dictionaries, each optionally containing keys
 "role" and "content".

Post-condition:
- Returns a pair (system_text, anthropic_messages) where:
- anthropic_messages is a list of dicts, each with exactly the keys
 "role" and "content", containing every input message whose role is
 "user" or "assistant", in their original relative order.
- For an input message whose "content" value is a string, it passes
 through unchanged. When "content" is a list of dicts (content blocks),
 it is replaced with a single string formed by joining the "text" value
 of each dict in the list with newline separators. A dict in the list
 without a "text" key contributes an empty string at that position.
- system_text is the empty string when no input message has role
 "system". When exactly one system-role message is present,
 system_text is its (possibly flattened) content string verbatim
 (with no whitespace stripping). When more than one system-role
 message is present, system_text is the result of concatenating
 their (possibly flattened) content strings in order, joining them
 with "\n\n", and then stripping leading and trailing whitespace
 from the entire concatenated result.
- Messages whose role is neither "system", "user", nor "assistant" are
 excluded from both outputs.
[SPEC]
```

### Reasoner 差异

- SPEC claim:

```
- Returns a pair (system_text, anthropic_messages) where:
- anthropic_messages is a list of dicts, each with exactly the keys
 "role" and "content", containing every input message whose role is
 "user" or "assistant", in their original relative order.
- For an input message whose "content" value is a string, it passes
 through unchanged. When "content" is a list of dicts (content blocks),
 it is replaced with a single string formed by joining the "text" value
 of each dict in the list with newline separators. A dict in the list
 without a "text" key contributes an empty string at that position.
- system_text is the empty string when no input message has role
 "system". When exactly one system-role message is present,
 system_text is its (possibly flattened) content string verbatim
 (with no whitespace stripping). When more than one system-role
 message is present, system_text is the result of concatenating
```

their (possibly flattened) content strings in order, joining them with "\n\n", and then stripping leading and trailing whitespace from the entire concatenated result.

- Messages whose role is neither "system", "user", nor "assistant" are excluded from both outputs.

- 推导 actual behavior:

The function returns a tuple (system\_text, out). Let processed\_content(m) be: if m.get('content', '') is a string, use it; otherwise (content is a list), flatten it to a string by joining c.get('text', '') for each c in content that is a dict, separated by newlines. system\_text is built from all messages m in input where m.get('role') == 'system': if there are no such messages, system\_text is ''; if exactly one, system\_text is processed\_content(m) (without additional stripping); if more than one, system\_text is the result of joining all processed\_content of those messages in order with the separator '\n\n' and then stripping leading and trailing whitespace. out is a list of dictionaries {'role': m['role'], 'content': processed\_content(m)}, preserving the relative order of those messages in the input, for each message where m.get('role') in {'user', 'assistant'}. The function has no side effects.

- Code evidence:

```
Line 13: system_text = (system_text + "\n\n" + content).strip() if system_text else content
```

- Trigger condition:

When there are multiple system messages, the code strips whitespace after each concatenation, causing interior whitespace to be lost. The specification requires concatenating all contents first with '\n\n' and then stripping only the final result. With the input `[{'role': 'system', 'content': ' a '}, {'role': 'system', 'content': ' b '}]`, the code produces 'a\n\n b' while the spec requires 'a\n\n b ', which differ.

#### Bug validator

- Trigger summary: With 3+ system messages, the code's iterative .strip() on line 92 prematurely removes trailing whitespace from intermediate messages' content, while the spec requires concatenating all contents first and stripping only the final result.
- Probe stdout:

```
CONFIRMED – bug reproduced. Failures:
[three systems (key test)]
 actual: 'a\n\n b\n\nc'
 expected: 'a\n\n b \n\nc'
[four systems]
 actual: 'x \n\n y\n\n z\n\nw'
 expected: 'x \n\n y\n\n z \n\nw'
```

---

#### INCR-MISMATCH-058 — `src--llm_client-py--_metadata_body`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/llm_client-py/_metadata_body.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_metadata_body.json`。
- 详细报告: `src--llm_client-py--_metadata_body.md`。
- Probe: `probe_src--llm_client-py--_metadata_body.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/llm_client.py
```

```
_metadata_body() -> dict
```

Pre-condition:

- None (takes no arguments)

Post-condition:

- Returns a dictionary with exactly one top-level key "metadata", whose value is a nested dictionary containing a "user\_id" key
  - The value associated with "user\_id" is a stable, consistent string that identifies the current environment across calls within the same installation
- [SPEC]

## Reasoner 差异

- SPEC claim:

- Returns a dictionary with exactly one top-level key "metadata", whose value is a nested dictionary containing a "user\_id" key

- The value associated with "user\_id" is a stable, consistent string that identifies the current environment across calls within the same installation

- 推导 actual behavior:

Returns: a dictionary object `result`. Post-condition: `isinstance(result, dict) and ('metadata' in result) and (len(result) == 1) and isinstance(result['metadata'], dict) and ('user\_id' in result['metadata']) and (len(result['metadata']) == 1) and isinstance(result['metadata']['user\_id'], str) and (len(result['metadata']['user\_id']) > 0)`. Natural language: The function always returns a dictionary with the exact structure `{"metadata": {"user\_id": s}}`, where `s` is a non-empty string (the result of calling `\_stable\_user\_id()`).

- Code evidence:

```
Line 2: return {"metadata": {"user_id": _stable_user_id()}}
```

- Trigger condition:

The function relies on `\_stable\_user\_id()`, which returns `settings.inject.id` when truthy. Because that setting can mutate between calls, the output user\_id is not guaranteed to be stable, conflicting with the specification.

## Bug validator

- Trigger summary: Mutating settings.inject.id between calls to \_metadata\_body() causes the returned user\_id to change, violating the specification's stability guarantee.
- Probe stdout:

CONFIRMED – user\_id changed across calls after settings mutation.

First call (settings.inject.id=''): user\_id='stable-user-or-session-id-xxxxxxx123'

Second call (settings.inject.id='mutated-user-id-abc123'): user\_id='mutated-user-id-abc123'

Spec requires: user\_id is a stable, consistent string that identifies the current environment across calls within the same installation.

Bug: settings.inject.id is mutable, so \_stable\_user\_id() (called by \_metadata\_body) can return different values between calls.

---

## INCR-MISMATCH-059 — `src--llm_client-py--_retry_create`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/llm_client-py/_retry_create.py`。



- Reasoner 结果: `logic_verification_results/src/llm_client-py/_retry_create.json`。
- 详细报告: `src--llm_client-py--_retry_create.md`。
- Probe: `probe_src--llm_client-py--_retry_create.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/llm_client-py/_retry_create.py

_retry_create(client, model, messages) -> (str, dict)

Pre-condition:
- client can send conversation messages for the given model and return text responses
- model is a non-empty string
- messages is a list of message dicts with "role" and "content" keys

Post-condition:
- Returns a tuple of (response_text, usage_metadata_dict) from a successful LLM call
- response_text is the text content returned by the LLM
- usage_metadata_dict maps token-usage keys to numeric counts from the LLM response, or is an empty dict when no usage data is reported
- When the CLI backend is active, the LLM interaction is delegated to an external agent
- For direct client calls, Anthropic-family models use a dedicated native Anthropic endpoint; all other models use the standard chat-completions endpoint
- Recoverable errors (rate limiting, server unavailability, and other transient HTTP/middleware failures) are retried with increasing delay bounded by a per-category maximum retry count
- Non-recoverable errors (provider-rejected malformed requests) are propagated immediately without retry
- Raises RuntimeError when a recoverable error exhausts its retry budget
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns a tuple of (response_text, usage_metadata_dict) from a successful LLM call
- response_text is the text content returned by the LLM
- usage_metadata_dict maps token-usage keys to numeric counts from the LLM response, or is an empty dict when no usage data is reported
- When the CLI backend is active, the LLM interaction is delegated to an external agent
- For direct client calls, Anthropic-family models use a dedicated native Anthropic endpoint; all other models use the standard chat-completions endpoint
- Recoverable errors (rate limiting, server unavailability, and other transient HTTP/middleware failures) are retried with increasing delay bounded by a per-category maximum retry count
- Non-recoverable errors (provider-rejected malformed requests) are propagated immediately without retry
- Raises RuntimeError when a recoverable error exhausts its retry budget
```

- 推导 actual behavior:

```
If `is_cli_backend_enabled()` is true, the function returns the result of
`run_agent_for_messages(model, messages)` and does not raise any exception handled within this
function. Otherwise, the function repeatedly attempts to obtain a completion for the given model
and messages, retrying on rate-limit (HTTP 429 or RateLimitError) up to _MAX_RATE_LIMIT_RETRIES
times with exponential backoff and on transient failures (HTTP 5xx, other exceptions) up to
_MAX_LLM_RETRIES times with exponential backoff. If a completion is successful before retries
are exhausted, the function returns a tuple (text, usage) where text is a string of the
assistant's response content, and usage is a dictionary of token usage details (or an empty dict
if no usage is available). If a BadRequestError or an HTTPError with status 400 occurs, the
function raises that exception immediately. If rate-limit retries are exhausted, a RuntimeError
is raised. If transient retries are exhausted, a RuntimeError is raised. Any other exception
that occurs after the last allowed retry is wrapped in a RuntimeError. Formally, for every
execution E of `_retry_create(client, model, messages)`: E either terminates with a return value
R or raises exception X. (E returns R) [(is_cli_backend_enabled() R =
run_agent_for_messages(model, messages)) (is_cli_backend_enabled() t: str, u: dict . R = (t,
u) valid_response(t, u))]. (E raises X) [X {BadRequestError, HTTPError(400), RuntimeError}]
where RuntimeError indicates either rate-limit or transient retry exhaustion. No other outcomes
are possible, and the function respects the described retry limits and sleep intervals.
```

- Code evidence:

```
Line 60: except Exception as exc:
```

- Trigger condition:

The code catches all Exception instances and treats them as transient errors to retry. A TypeError caused by an invalid messages type is not a recoverable transient error; the specification requires that only recoverable errors (rate limiting, server unavailability, etc.) be retried. Non-recoverable errors like malformed input should be propagated immediately without retry, but the code retries them and eventually raises a RuntimeError.

#### Bug validator

- Trigger summary: Passing input that causes a TypeError inside client.chat.completions.create() triggers the catch-all except Exception handler, which retries it 5 times as a transient error instead of propagating it immediately.
- Probe stdout:

```
WARNING:root:LLM error (TypeError: invalid messages type: expected list of dicts, got str),
sleeping 7.8s (attempt 1)
WARNING:root:LLM error (TypeError: invalid messages type: expected list of dicts, got str),
sleeping 12.9s (attempt 2)
WARNING:root:LLM error (TypeError: invalid messages type: expected list of dicts, got str),
sleeping 22.5s (attempt 3)
WARNING:root:LLM error (TypeError: invalid messages type: expected list of dicts, got str),
sleeping 41.7s (attempt 4)
CONFIRMED – TypeError was retried (5x with 5 retry budget) instead of propagating immediately;
final result was RuntimeError: LLM request failed after 5 retries: invalid messages type:
expected list of dicts, got str
```

---

#### INCR-MISMATCH-060 — `src--llm_client-py--_stable_user_id`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/llm_client-py/_stable_user_id.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_stable_user_id.json`。
- 详细报告: `src--llm_client-py--_stable_user_id.md`。
- Probe: `probe_src--llm_client-py--_stable_user_id.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/llm_client.py

_stable_user_id() -> str

Pre-condition:
- None (takes no arguments)

Post-condition:
- Returns the value of `settings.inject.id` when that value is truthy
- Returns the predefined static default `_DEFAULT_INJECT_USER_ID` when `settings.inject.id` is
falsy (empty or None)
- The returned string is non-empty in all cases
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

- Returns the value of ``settings.inject.id`` when that value is truthy
- Returns the predefined static default ``_DEFAULT_INJECT_USER_ID`` when ``settings.inject.id`` is falsy (empty or None)
- The returned string is non-empty in all cases

- 推导 actual behavior:

The function returns the value of `settings.inject.id` if it is truthy (as per Python bool conversion), otherwise returns `_DEFAULT_INJECT_USER_ID`. No external state is modified. Formally: let `ret` be the return value. Then `ret = settings.inject.id` if `bool(settings.inject.id)` else `_DEFAULT_INJECT_USER_ID`, and all module-level objects remain unchanged.

- Code evidence:

```
Line 2: return settings.inject.id or _DEFAULT_INJECT_USER_ID
```

- Trigger condition:

The specification states that the returned string is non-empty in all cases, implying the function must always return a string. However, when `settings.inject.id` is a truthy non-string (e.g., an integer 5), the code returns that non-string value, violating the requirement that the return value be a string.

#### Bug validator

- Trigger summary: When `settings.inject.id` is a truthy non-string (e.g., integer 5), Python's `or` returns the non-string value as-is instead of a string, violating the spec requirement that the return value always be a non-empty string.
- Probe stdout:

```
CONFIRMED – actual type: int, value: 5 | expected type: str
```

## src/opencode\_trace-py

### INCR-MISMATCH-061 — src--opencode\_trace-py--\_opencode\_provider\_config

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/opencode_trace-py/_opencode_provider_config.py`。
- Reasoner 结果: `logic_verification_results/src/opencode_trace-py/_opencode_provider_config.json`。
- 详细报告: `src--opencode_trace-py--_opencode_provider_config.md`。
- Probe: `probe_src--opencode_trace-py--_opencode_provider_config.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/opencode_trace.py

_opencode_provider_config() -> dict | None

Pre-condition:
- The global settings object has been loaded and its .llm attribute provides
 access to the LLM configuration fields api_key, base_url, name, provider,
 and api_style.

Post-condition:
```

- Returns None when any of `api_key`, `base_url`, `name`, or `provider` is falsy (i.e., an empty string, None, or otherwise evaluates to False).
- When all of `api_key`, `base_url`, `name`, and `provider` are truthy, returns a dict that, when included in an OpenCode configuration, defines a valid provider.
- The returned dict nests under a "provider" key, keyed by the value of the provider setting, and contains an npm adapter package name, a base URL, a model name, and an API key reference.
- The npm adapter is chosen based on the `api_style` setting: the Anthropic SDK package when `api_style` is "anthropic", and an OpenAI-compatible SDK package otherwise.
- The API key is specified as an environment-variable reference (`{env:LLM_API_KEY}`) rather than a literal key value.
- The function has no side effects: it does not mutate any global state, perform I/O, or modify any passed-in arguments.
- No exceptions are raised under normal operation.

[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns None when any of `api_key`, `base_url`, `name`, or `provider` is falsy (i.e., an empty string, None, or otherwise evaluates to False).
- When all of `api_key`, `base_url`, `name`, and `provider` are truthy, returns a dict that, when included in an OpenCode configuration, defines a valid provider.
- The returned dict nests under a "provider" key, keyed by the value of the provider setting, and contains an npm adapter package name, a base URL, a model name, and an API key reference.
- The npm adapter is chosen based on the `api_style` setting: the Anthropic SDK package when `api_style` is "anthropic", and an OpenAI-compatible SDK package otherwise.
- The API key is specified as an environment-variable reference (`{env:LLM_API_KEY}`) rather than a literal key value.
- The function has no side effects: it does not mutate any global state, perform I/O, or modify any passed-in arguments.
- No exceptions are raised under normal operation.

- 推导 actual behavior:

The function `_opencode_provider_config()` does not modify any global state (e.g., the settings object remains unchanged). Its return value is determined as follows: if any of the attributes `settings.llm.api_key`, `settings.llm.base_url`, `settings.llm.name`, or `settings.llm.provider` is falsy (None, empty string, etc.), the function returns None. Otherwise, it returns a dictionary with exactly one top-level key 'provider', whose value is a dictionary containing exactly one key equal to `settings.llm.provider`. That inner dictionary has the keys 'npm', 'options', and 'models'. The value of 'npm' is '@ai-sdk/anthropic' if `settings.llm.api_style` equals the string 'anthropic', else '@ai-sdk/openai-compatible'. The value of 'options' is a dictionary with keys 'baseURL' (set to `settings.llm.base_url`) and 'apiKey' (set to the literal string `{env:LLM_API_KEY}`). The value of 'models' is a dictionary with a single key `settings.llm.name` whose value is an empty dictionary {}. Formally:

```
let llm = settings.llm in
((llm.api_key llm.base_url llm.name llm.provider) result = None)
((llm.api_key llm.base_url llm.name llm.provider)
 result = { 'provider': { llm.provider: {
 'npm': '@ai-sdk/anthropic' if llm.api_style = 'anthropic' else '@ai-sdk/openai-compatible',
 'options': { 'baseURL': llm.base_url, 'apiKey': '{env:LLM_API_KEY}' },
 'models': { llm.name: {} }
 } } })
```

- Code evidence:

```
Line 16: llm = settings.llm
Line 17: if not (llm.api_key and llm.base_url and llm.name and llm.provider):
```

- Trigger condition:

The code attempts to access attributes on a likely None value (settings.llm), raising an `AttributeError` instead of returning None as required when any of the required fields is not present. This violates the specification's requirement that no exceptions be raised and that the function return None when those fields are missing/falsy.

#### Bug validator

- Trigger summary: When settings.llm is None, the function dereferences None.api\_key raising `AttributeError` instead of returning None as required by the spec.
- Probe stdout:

CONFIRMED — `AttributeError: 'NoneType' object has no attribute 'api_key'`  
Expected: None (return None when fields are missing)  
Actual: `AttributeError('NoneType' object has no attribute 'api_key')`

#### INCR-MISMATCH-062 — `src--opencode_trace-py--_start_opencode_process`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/opencode_trace-py/_start_opencode_process.py`。
- Reasoner 结果: `logic_verification_results/src/opencode_trace-py/_start_opencode_process.json`。
- 详细报告: `src--opencode_trace-py--_start_opencode_process.md`。
- Probe: `probe_src--opencode_trace-py--_start_opencode_process.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/opencode_trace.py

_start_opencode_process(proj_dir, work_dir, event_id, command, trace_log_path) ->
(subprocess.Popen, threading.Thread, threading.Thread | None)

Pre-condition:
- proj_dir is a path to an existing directory on the filesystem
- work_dir is a path to an existing directory on the filesystem
- event_id is a non-empty unique string identifying this trace event
- command is an AgentCommand or a non-empty list of strings forming a valid CLI invocation
- trace_log_path is a filesystem path under work_dir whose parent directories exist

Post-condition:
- Launches command as a subprocess whose working directory is proj_dir, whose environment variables are derived from work_dir and event_id, and whose stdout and stderr are merged into a single pipeline for capture
- The subprocess receives input via a connected stdin pipe if and only if the command carries non-None stdin text; otherwise stdin is not connected to the subprocess
- The subprocess text stream encoding is UTF-8 with replacement on decode errors, guaranteeing no UnicodeDecodeError on output read
- Starts a background daemon thread that copies the subprocess merged output to trace_log_path as it is produced, ensuring every byte written by the subprocess is recorded
- If the command carries non-None stdin text, starts a background daemon thread that writes that text to the subprocess stdin pipe and then closes it; if the command carries no stdin text, no stdin-writing thread is started
- Returns a tuple of (process_handle, log_thread, stdin_thread) where log_thread is always a started threading.Thread, and stdin_thread is either a started threading.Thread or None
- All launched threads are daemon threads: they will not prevent the calling process from exiting
- The subprocess has not yet been waited on; its exit code is not available
[SPEC]
```

- SPEC claim:

- Launches command as a subprocess whose working directory is proj\_dir, whose environment variables are derived from work\_dir and event\_id, and whose stdout and stderr are merged into a single pipeline for capture
- The subprocess receives input via a connected stdin pipe if and only if the command carries non-None stdin text; otherwise stdin is not connected to the subprocess
- The subprocess text stream encoding is UTF-8 with replacement on decode errors, guaranteeing no UnicodeDecodeError on output read
- Starts a background daemon thread that copies the subprocess merged output to trace\_log\_path as it is produced, ensuring every byte written by the subprocess is recorded
- If the command carries non-None stdin text, starts a background daemon thread that writes that text to the subprocess stdin pipe and then closes it; if the command carries no stdin text, no stdin-writing thread is started
- Returns a tuple of (process\_handle, log\_thread, stdin\_thread) where log\_thread is always a started threading.Thread, and stdin\_thread is either a started threading.Thread or None
- All launched threads are daemon threads: they will not prevent the calling process from exiting
- The subprocess has not yet been waited on; its exit code is not available

- 推导 actual behavior:

Upon successful completion, the function returns a 3-tuple `(proc, log\_thread, stdin\_thread)` where:

- `proc` is a `subprocess.Popen` instance representing a child process that has been launched with the following properties:
  - Command arguments: `command\_argv(command)`.
  - Working directory: `proj\_dir`.
  - Environment: `\_opencode\_env(work\_dir, event\_id)` (a superset of the current process's environment).
  - Standard input: a pipe if `command\_stdin(command)` is not `None`, otherwise `None`.
  - Standard output: a pipe.
  - Standard error: merged with standard output (`subprocess.STDOUT`).
  - Text mode enabled with UTF-8 encoding and 'replace' error handling.
  - The underlying OS-level child process has been created and is either running or may have already terminated; `proc.pid` is set.
- `log\_thread` is a `threading.Thread` object that has been started. It is a daemon thread executing `\_copy\_opencode\_output(proc.stdout, trace\_log\_path)`, which will read lines from the subprocess's stdout and write them to the file at `trace\_log\_path` until the stream is exhausted (EOF), then flush and close the output file.
- `stdin\_thread` is:
  - If `command\_stdin(command)` is not `None`: a `threading.Thread` object that has been started, is daemonic, and executes `\_write\_command\_stdin(proc.stdin, command\_stdin(command))`. This will write the stdin text to the subprocess's stdin, flush it, and close the pipe, after which no further writes to `proc.stdin` are possible.
  - If `command\_stdin(command)` is `None`: `None`.

After the return, the caller must not read from `proc.stdout` (as it is consumed by `log\_thread`) nor write to `proc.stdin` if `stdin\_thread` is non-`None` (as the thread will eventually close it). The output of the subprocess will eventually be recorded in `trace\_log\_path` provided the daemon threads are not prematurely terminated by the process exiting.

Formally (let `stdin\_text = command\_stdin(command)` and `args = command\_argv(command)`):

```
...
(result == (proc, log_thread, stdin_thread))
(proc.pid == None)
(proc.args == args)
(proc.cwd == proj_dir)
(proc.env == _opencode_env(work_dir, event_id))
(proc.stdout is a readable pipe)
(proc.stderr == subprocess.STDOUT)
(proc.text == True)
(proc.encoding == 'utf-8')
(proc.errors == 'replace')
((stdin_text != None) (proc.stdin is a writable pipe)
 (stdin_thread is a Thread with target=_write_command_stdin,
 args=(proc.stdin, stdin_text), daemon=True, started=True))
((stdin_text == None) (proc.stdin == None) (stdin_thread == None))
(log_thread is a Thread with target=_copy_opencode_output,
 args=(proc.stdout, trace_log_path), daemon=True, started=True)
...
```

#### • Code evidence:

Line 7: stdin=subprocess.PIPE if stdin\_text is not None else None,

#### • Trigger condition:

When stdin\_text is None, the code sets stdin=None, which causes the subprocess to inherit the parent's stdin file descriptor instead of closing or disconnecting it. This violates the specification's requirement that 'otherwise stdin is not connected to the subprocess'.

#### Bug validator

- Trigger summary: When command has no stdin text (command\_stdin returns None), subprocess.Popen receives stdin=None, causing the child to inherit the parent's stdin fd instead of disconnecting it per the spec.
- Probe stdout:

CONFIRMED – subprocess.Popen received stdin=None, so the subprocess inherits the parent's stdin file descriptor instead of having stdin disconnected. Spec requires: 'otherwise stdin is not connected to the subprocess'.

## src/pipeline\_setup.py

### INCR-MISMATCH-063 — src--pipeline\_setup.py--\_deduplicate\_phases

- 人工审计：契约待确认。
- Validator： **confirmed**；尝试次数： 1。
- 原始函数与 SPEC： `src/pipeline_setup.py/_deduplicate_phases.py`。
- Reasoner 结果： `logic_verification_results/src/pipeline_setup.py/_deduplicate_phases.json`。
- 详细报告： `src--pipeline_setup.py--_deduplicate_phases.md`。
- Probe： `probe_src--pipeline_setup.py--_deduplicate_phases.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/pipeline_setup.py

_deduplicate_phases(phases_dir) -> dict

Pre-condition:
- phases_dir is a string path to a directory containing a valid
 phases.json file.
- phases.json conforms to the pipeline schema: a JSON object with a
 "phases" array; each phase has "phase" (int) and "modules" (array);
 each module has "name" (string) and "source_files" (array of strings).
- Each source_files element is a relative file path string from the
 project root.
- The phases.json file is readable and writable by the current process.

Post-condition:
- phases.json is overwritten with every source file path appearing in at
 most one module across all phases.
- For any source file path appearing in multiple modules (across the same
 or different phases), only the first occurrence – in ascending phase
 number order, then module order within each phase – is preserved; all
 subsequent occurrences are removed from their respective module's
 source_files list.
- The set of phases and modules is unchanged: no phase or module is
 removed, even when a module's source_files becomes empty.
- Phase numbers, module names, and the ordering of phases/modules within
 phases.json are preserved.
- Returns a dict with the key "modified_modules" mapping to a list of
 dicts, one per module from which at least one source file was removed.
 Each module dict contains: "phase" (int – the phase number), "module"
 (str – the module name), "removed_files" (list of strings – the
 deduplicated file paths that were removed), and "source_files" (list of
 strings – the module's remaining source files after deduplication).
- Returns {"modified_modules": []} when no duplicate source files exist
 across modules.

[SPEC]
```

#### Reasoner 差异

- SPEC claim:

```
- phases.json is overwritten with every source file path appearing in at
 most one module across all phases.
- For any source file path appearing in multiple modules (across the same
 or different phases), only the first occurrence in ascending phase
 number order, then module order within each phase is preserved; all
 subsequent occurrences are removed from their respective module's
 source_files list.
- The set of phases and modules is unchanged: no phase or module is
 removed, even when a module's source_files becomes empty.
```



- Phase numbers, module names, and the ordering of phases/modules within phases.json are preserved.
- Returns a dict with the key "modified\_modules" mapping to a list of dicts, one per module from which at least one source file was removed. Each module dict contains: "phase" (int the phase number), "module" (str the module name), "removed\_files" (list of strings the deduplicated file paths that were removed), and "source\_files" (list of strings the module's remaining source files after deduplication).
- Returns {"modified\_modules": []} when no duplicate source files exist across modules.

- 推导 actual behavior:

If the function returns a value R without raising an exception, the following properties hold: (1) The file at `os.path.join(phases_dir, 'phases.json')` has been overwritten with a JSON object data such that `data['phases']` is a list of the same length and order as in the original data; for each phase `p` in `data['phases']`, `p['phase']` equals the original integer, and `p['modules']` is the original list of modules in the same order; for each module `m` in `p['modules']`, `m['name']` is unchanged, and `m['source_files']` is the subsequence of the original `m['source_files']` containing exactly those files that were not in the set Seen constructed by iterating phases sorted by phase ascending and modules in their original order, with Seen growing as files are encountered for the first time; thus every source file path that ever appears in the original data appears in exactly one module's `source_files` list in data, specifically the first module (by phase, then module order) that originally claimed it, and the relative order of kept files in each module is preserved. (2) No phases or modules are added, removed, or reordered. (3) R is a dict with key 'modified\_modules'; R['modified\_modules'] is a list of objects, one per module whose `source_files` list changed (i.e., some file was removed), ordered by the same traversal; each object has 'phase': integer phase number, 'module': module name string (or '' if missing), 'removed\_files': list of removed file paths, and 'source\_files': the final deduplicated list for that module. All changed modules are included, and unchanged modules are omitted. (4) Side effect: for each duplicate file found, `logging.info()` was called with a message indicating the file path, phase, and module. (5) If an exception is raised before the 'with open(..., "w")' block, the original file remains unchanged and the function does not return. If an exception is raised during the final write, the file state is unspecified (may be partially written or truncated) and the function does not return.

- Code evidence:

```
Line 32: removed_files = [sf for sf in original if sf not in deduped]
```

- Trigger condition:

`removed_files` may contain duplicate file paths if the original `source_files` list in a module contained duplicate entries of a file that is entirely removed from that module. The specification requires the list of removed files to be deduplicated, i.e., each distinct file path should appear at most once.

#### Bug validator

- Trigger summary: When a module's `source_files` contains duplicate entries of a file that is entirely removed, `removed_files` preserves those duplicates instead of deduplicating them.
- Probe stdout:

```
CONFIRMED – duplicate removed_files for module_b: actual=['a.py', 'a.py'] | expected (unique)=['a.py']
```

#### INCR-MISMATCH-064 — `src--pipeline_setup-py--_phase_plan_complete`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/pipeline_setup-py/_phase_plan_complete.py`。
- Reasoner 结果: `logic_verification_results/src/pipeline_setup-py/_phase_plan_complete.json`。

- 详细报告: `src--pipeline_setup-py--_phase_plan_complete.md`。
- Probe: `probe_src--pipeline_setup-py--_phase_plan_complete.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/pipeline_setup-py/_phase_plan_complete.py

_phase_plan_complete(work_dir) -> bool

Pre-condition:
- work_dir is a string path to an existing directory.

Post-condition:
- Returns True when the file phases.json exists under work_dir, is a
 regular file, its content parses as valid JSON, and it conforms to the
 required schema (as determined by _phase_plan_schema_errors).
- Returns False when phases.json does not exist under work_dir, is not a
 regular file, does not parse as valid JSON, or does not conform to the
 required schema.
- The return value is idempotent for the same filesystem state: repeated
 calls with the same work_dir and same file content yield the same boolean
 result.
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns True when the file phases.json exists under work_dir, is a
 regular file, its content parses as valid JSON, and it conforms to the
 required schema (as determined by _phase_plan_schema_errors).
- Returns False when phases.json does not exist under work_dir, is not a
 regular file, does not parse as valid JSON, or does not conform to the
 required schema.
- The return value is idempotent for the same filesystem state: repeated
 calls with the same work_dir and same file content yield the same boolean
 result.
```

- 推导 actual behavior:

After a call to `_phase_plan_complete(work_dir)` finishes execution, no side effects have occurred and no exceptions have been raised. The return value `r` satisfies: `r` is `True` if and only if the file obtained by `os.path.join(work_dir, "phases.json")` exists, is readable, is valid JSON, and its decoded content fully conforms to the required schema (i.e., `_phase_plan_schema_errors` returns an empty list); otherwise `r` is `False`. In formal terms: let `p = os.path.join(work_dir, "phases.json")`. Then  $\left( \textit{result} = \mathbf{True} \right) \iff \left( \textit{isfile}(p) \wedge \textit{readable}(p) \wedge \textit{valid\_json}(\textit{read}(p)) \wedge \textit{schema\_conforms}(\textit{parse\_json}(\textit{read}(p))) \right)$ , which is equivalent to  $\left( \textit{result} = \mathbf{True} \right) \iff \textit{phase\_plan\_schema\_errors}(p) = [\,] \, \vee \, \text{false}$ .

- Code evidence:

```
Line 4: return not _phase_plan_schema_errors(phases_path)
```

- Trigger condition:

The specification requires that `phases.json` be a regular file to return `True`. The code delegates entirely to `_phase_plan_schema_errors`, which, according to its post-condition, does not check whether the file is a regular file. If a non-regular file (e.g., a FIFO) exists, is readable, and contains valid schema-conforming JSON, `_phase_plan_schema_errors` returns an empty list, causing `_phase_plan_complete` to return `True`, violating the specification.

## Bug validator

- Trigger summary: When phases.json is a FIFO (non-regular file) containing valid schema-conforming JSON, `_phase_plan_complete` returns True instead of the spec-required False.
- Probe stdout:

CONFIRMED – actual: True | expected: False

INCR-MISMATCH-065 — `src--pipeline_setup-py--_phase_plan_schema_errors`

- 人工审计: 实现缺陷候选。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/pipeline_setup-py/_phase_plan_schema_errors.py`。
- Reasoner 结果: `logic_verification_results/src/pipeline_setup-py/_phase_plan_schema_errors.json`。
- 详细报告: `src--pipeline_setup-py--_phase_plan_schema_errors.md`。
- Probe: `probe_src--pipeline_setup-py--_phase_plan_schema_errors.py`。

完整生成 SPEC

[SPEC]  
Unit: `src/pipeline_setup-py/_phase_plan_schema_errors.py`  
  
`_phase_plan_schema_errors(phases_path) -> list[str]`  
  
Pre-condition:  
- `phases_path` is a string.  
  
Post-condition:  
- Returns a list of human-readable error message strings.  
  
- When the file at `phases_path` cannot be read (any OS error including absent file or permission denied), the returned list is non-empty and contains a single element describing the OS error.  
  
- When the file at `phases_path` is readable but its content is not valid JSON, the returned list is non-empty and contains a single element describing the JSON parse error.  
  
- When the file is readable and its content is valid JSON, the returned list is empty if and only if the decoded JSON structure satisfies all of the following requirements:  
  a) The top-level decoded value is a JSON object (dict).  
  b) The object has a key "phases" whose value is a JSON array.  
  c) Every element of the "phases" array is a JSON object.  
  d) Every object in "phases" has a key "modules" whose value is a JSON array.  
  e) Every element of a "modules" array is a JSON object.  
  f) Every object in "modules" has a key "source\_files" whose value is a JSON array.  
  g) Every element of a "source\_files" array is a JSON string.  
  
- When the decoded JSON violates any requirement from (a) through (g), the returned list is non-empty. Each element of the list describes exactly one violation using JSON-path notation (zero-based array indices in brackets). When a module object has a non-empty "name" key, violation messages for that module include the name value for identification.  
  
- The function does not modify the file at `phases_path` or any other persistent state.  
- The function always returns within finite time regardless of inputs.  
[SPEC]

Reasoner 差异

- SPEC claim:

- Returns a list of human-readable error message strings.  
  
- When the file at `phases_path` cannot be read (any OS error including absent file or permission denied), the returned list is non-empty and contains a single element describing the OS error.  
  
- When the file at `phases_path` is readable but its content is not valid JSON, the

returned list is non-empty and contains a single element describing the JSON parse error.

- When the file is readable and its content is valid JSON, the returned list is empty if and only if the decoded JSON structure satisfies all of the following requirements:
  - a) The top-level decoded value is a JSON object (dict).
  - b) The object has a key "phases" whose value is a JSON array.
  - c) Every element of the "phases" array is a JSON object.
  - d) Every object in "phases" has a key "modules" whose value is a JSON array.
  - e) Every element of a "modules" array is a JSON object.
  - f) Every object in "modules" has a key "source\_files" whose value is a JSON array.
  - g) Every element of a "source\_files" array is a JSON string.
- When the decoded JSON violates any requirement from (a) through (g), the returned list is non-empty. Each element of the list describes exactly one violation using JSON-path notation (zero-based array indices in brackets). When a module object has a non-empty "name" key, violation messages for that module include the name value for identification.
- The function does not modify the file at phases\_path or any other persistent state.
- The function always returns within finite time regardless of inputs.

- **推导 actual behavior:**

The function attempts to read and parse the JSON file at `phases\_path`. If an `OSError` occurs (e.g., file not found, permission denied), it returns a list containing a single string describing the error (formatted as 'phases.json could not be read: ...'). If the file is read but contains invalid JSON, it returns a list with a single string describing the decode error (formatted as 'phases.json is not valid JSON: ...'). Otherwise, the file is successfully parsed into a Python object; the function then validates its structure and returns a list of human-readable schema error strings (possibly empty). The validation rules are: (1) the top-level value must be a dictionary, else an error is reported; (2) the dictionary must contain a `"phases"` key whose value is a list, else an error is reported; (3) each element of that list must be a dictionary; (4) each phase dictionary must contain a `"modules"` key whose value is a list; (5) each element of the modules list must be a dictionary; (6) each module dictionary must contain the key `"source\_files"`; (7) the value of `"source\_files"` must be a list, and every element of that list must be a string. All discovered errors are appended to the result list in order of traversal. If no errors are found, the returned list is empty. The function does not modify any global state or the file system, and it always closes the file if it was opened successfully. No unhandled exceptions propagate to the caller.

- **Code evidence:**

```
Line 3: try:
Line 4: with open(phases_path, "r") as f:
Line 5: data = json.load(f)
Line 6: except OSError as exc:
Line 7: return [f"phases.json could not be read: {exc}"]
Line 8: except json.JSONDecodeError as exc:
Line 9: return [f"phases.json is not valid JSON: {exc}"]
```

- **Trigger condition:**

The specification requires that the function always returns within finite time regardless of inputs, but the code only catches `OSError` and `JSONDecodeError`. A file that cannot be decoded from text (e.g., invalid UTF-8) causes a `UnicodeDecodeError`, which is not handled, so the function raises an exception and never returns a list, violating the specification.

## Bug validator

- **Trigger summary:** Opening a file containing invalid UTF-8 bytes causes an unhandled `UnicodeDecodeError` instead of returning an error list.
- **Probe stdout:**

CONFIRMED – `_phase_plan_schema_errors` raised `UnicodeDecodeError` instead of returning a list (spec requires 'always returns within finite time regardless of inputs')

- 人工审计：实现缺陷候选。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/pipeline_setup-py/_phases_cover_current_sources.py`。
- Reasoner 结果: `logic_verification_results/src/pipeline_setup-py/_phases_cover_current_sources.json`。
- 详细报告: `src--pipeline_setup-py--_phases_cover_current_sources.md`。
- Probe: `probe_src--pipeline_setup-py--_phases_cover_current_sources.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/pipeline_setup-py/_phases_cover_current_sources.py

_phases_cover_current_sources(phases_json, proj_dir, submodules=None) -> bool

Pre-condition:
- phases_json is a string path
- proj_dir is an existing directory
- submodules is None or an iterable of subdirectory name strings

Post-condition:
- Returns True when all of the following hold: (a) phases_json is a readable file
 whose content parses as valid JSON, (b) the JSON contains at least one source file
 entry across all phases and modules, (c) every source file path listed in the JSON
 resolves to an existing file under proj_dir, (d) when submodules is neither None
 nor an empty iterable, every listed source file path falls under at least one of
 the specified submodule directories (as determined by _is_under_submodules),
 and (e) every source file under the project directories scoped by submodules
 (or under all of proj_dir when submodules is None) appears in the JSON
- Returns False when any of (a)-(e) fails
- Backslash separators in source file paths within the JSON are treated as forward
 slashes for path comparison and file existence resolution
- The function does not create, modify, delete, or rename any file or directory
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns True when all of the following hold: (a) phases_json is a readable file
 whose content parses as valid JSON, (b) the JSON contains at least one source file
 entry across all phases and modules, (c) every source file path listed in the JSON
 resolves to an existing file under proj_dir, (d) when submodules is neither None
 nor an empty iterable, every listed source file path falls under at least one of
 the specified submodule directories (as determined by _is_under_submodules),
 and (e) every source file under the project directories scoped by submodules
 (or under all of proj_dir when submodules is None) appears in the JSON
- Returns False when any of (a)-(e) fails
- Backslash separators in source file paths within the JSON are treated as forward
 slashes for path comparison and file existence resolution
- The function does not create, modify, delete, or rename any file or directory
```

- 推导 actual behavior:

```
After execution, the return value is True iff all the following hold: (i) the file at
phases_json is successfully opened and parsed as JSON (no OSError/ValueError); (ii) the parsed
data yields a non-empty set listed of source file paths, each with backslashes converted to
forward slashes, extracted from the phasesmodulelessource_files hierarchy; (iii) if submodules is
not None, every path in listed satisfies _is_under_submodules(sf, submodules) (i.e., contains
one of the submodule strings as a path component); (iv) every path in listed corresponds to an
existing file in proj_dir (i.e., os.path.exists(join(proj_dir, sf))); (v) the set of all project
source files collected by _collect_project_source_files(proj_dir, submodules) is a subset of
listed. If any of these conditions fails, or if the read/parse fails, the function returns
False. No mutable state is modified. Formally: ret_val = True (read_parse_success(phases_json)
listed (submodules=None sf)listed: _is_under_submodules(sf, submodules)) sf)listed:
os.path.exists(os.path.join(proj_dir, sf)) _collect_project_source_files(proj_dir, submodules)
listed).
```

- Code evidence:

```
Line 17: if any(not os.path.exists(os.path.join(proj_dir, sf)) for sf in listed):
```

- Trigger condition:

The existence check does not ensure the file is under `proj_dir`; absolute paths (or relative paths with `'..'`) can refer to files outside `proj_dir`, causing the function to return `True` when the specification requires `False`.

#### Bug validator

- Trigger summary: Absolute file paths in `phases.json` pass `os.path.exists()` check via `os.path.join()` bypass, allowing files outside `proj_dir` to be accepted when spec requires they be under `proj_dir`.
- Probe stdout:

```
CONFIRMED -- actual: True | expected: False | absolute path /etc/hostname (exists) passed the
existence check via os.path.join(proj_dir, '/etc/hostname') = '/etc/hostname', but the spec
requires False because the file is not under proj_dir.
```

### INCR-MISMATCH-067 — `src--pipeline_setup-py--_run_generate_phases`

- 人工审计: 实现缺陷候选。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/pipeline_setup-py/_run_generate_phases.py`。
- Reasoner 结果: `logic_verification_results/src/pipeline_setup-py/_run_generate_phases.json`。
- 详细报告: `src--pipeline_setup-py--_run_generate_phases.md`。
- Probe: `probe_src--pipeline_setup-py--_run_generate_phases.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/pipeline_setup-py/_run_generate_phases.py

_run_generate_phases(proj_dir, work_dir, script_dir, is_incremental=False, resume=False,
submodules=None) -> None

Pre-condition:
- proj_dir, work_dir, and script_dir refer to existing directory paths
- is_incremental is a boolean; when truthy, a pre-existing phases.json under work_dir is
updated in place rather than regenerated from scratch
- resume is a boolean
- submodules is None or a non-empty iterable of subdirectory name strings relative to proj_dir

Post-condition:
- On normal return: phases.json exists under work_dir and conforms to the phases.json schema
- When resume is truthy and phases.json already satisfies the pipeline's completeness
criteria, the function returns without producing or modifying any file
- When submodules is provided: phases.json covers all source files under the specified
subdirectories of proj_dir; source files outside those subdirectories are neither added nor
required to be present
- When is_incremental is truthy: a valid phases.json already present under work_dir may be
accepted without modification if it covers all current source files, even when its modification
timestamp has not changed
- If valid phases.json is not produced or confirmed after a configurable maximum number of
retry attempts, the function prints a diagnostic message to stdout identifying the failed stage
and the trace directory, then calls sys.exit(1)
- When a non-final attempt fails to produce valid phases.json, the function does not call
sys.exit(1) – it waits a fixed interval before retrying
[SPEC]
```

- SPEC claim:

- On normal return: phases.json exists under work\_dir and conforms to the phases.json schema
  - When resume is truthy and phases.json already satisfies the pipeline's completeness criteria, the function returns without producing or modifying any file
  - When submodules is provided: phases.json covers all source files under the specified subdirectories of proj\_dir; source files outside those subdirectories are neither added nor required to be present
  - When is\_incremental is truthy: a valid phases.json already present under work\_dir may be accepted without modification if it covers all current source files, even when its modification timestamp has not changed
  - If valid phases.json is not produced or confirmed after a configurable maximum number of retry attempts, the function prints a diagnostic message to stdout identifying the failed stage and the trace directory, then calls sys.exit(1)
  - When a non-final attempt fails to produce valid phases.json, the function does not call sys.exit(1) it waits a fixed interval before retrying

- 推导 actual behavior:

Natural language:

After executing lines 81-130 starting from a state where the previous try block completed normally (i.e., run\_opencode\_traced returned successfully, prompt, prompt\_file, command are set, trace event recorded, attempt = 1, and phases.json may or may not exist), one of three mutually exclusive outcomes occurs:

1. Break (line 103): phase\_plan\_ready becomes True. The value of phase\_plan\_errors is computed as  $\text{if phases.json exists, else ["phases.json is missing"]}$ . The variables failure and missing are not set. The outer loop is exited; execution continues after that loop.
2. Sys.exit (line 130): if phase\_plan\_ready is False and  $\text{attempt} \geq \text{OPENCODE\_MAX\_RETRIES}$ , the program prints an error message and terminates with exit code 1. No further program state exists.
3. Retry (line 123): if phase\_plan\_ready is False and  $\text{attempt} < \text{OPENCODE\_MAX\_RETRIES}$ , the block prints a warning, sleeps for 10 seconds, and then completes (the code outside the block will increment attempt and re-enter the loop). In this case, phase\_plan\_ready remains False, failure is set to "update phases.json" if is\_incremental else "produce phases.json", missing is set accordingly, and phase\_plan\_errors holds its computed list.

Formal logic:

Let  $\text{attempt} = 1$ ,  $\text{phases\_json}$  a path,  $\text{is\_incremental}$  a Boolean,  $\text{submodules}$  possibly a nonempty list or None,  $\text{prev\_mtime}$  a float,  $\text{OPENCODE\_MAX\_RETRIES}$  a positive integer constant. Define helper predicates from the environment:

$\text{exists}(\text{phases\_json})$  true iff the file exists.  
 $\text{cover}(\text{phases\_json}, \text{proj\_dir}, S)$  denotes  $\text{if } S \text{ is a subset of the sources in } \text{proj\_dir} \text{ then } S \text{ is covered by } \text{phases\_json}$ .  
 $\text{schema\_errs}(\text{phases\_json})$  denotes  $\text{the list of schema errors in } \text{phases\_json}$ .  
 $\text{mtime}(\text{phases\_json})$  is  $\text{os.path.getmtime}(\text{phases\_json})$  (defined only if file exists).

Then the postcondition  $Q_{81-130}$  is a disjunction of three cases, where the state after the block satisfies exactly one of:

$$Q_{81-130} \equiv \begin{aligned} & \left( \exists e \in \{ \text{if } \text{exists}(\text{phases\_json}) \text{ then } \text{schema\_errs}(\text{phases\_json}) \text{ else } ["\text{phases.json is missing}"] \} \right. \\ & \quad \wedge \text{True, False} \wedge f \in \text{strings} \wedge m \in \text{strings} . \\ & \quad \bigl( e = \text{if } \text{exists}(\text{phases\_json}) \text{ then } \text{schema\_errs}(\text{phases\_json}) \text{ else } ["\text{phases.json is missing"}] \bigr) \\ & \quad \wedge \bigl( \text{if } e = [] \wedge \text{submodules is not None and } \text{submodules} \neq [] \text{ then } \\ & \quad \quad \text{cover}(\text{phases\_json}, \text{proj\_dir}, \text{submodules}) \\ & \quad \wedge \text{elif } e = [] \wedge \text{is\_incremental} \text{ then } \\ & \quad \quad r = (\text{exists}(\text{phases\_json}) \wedge \text{mtime}(\text{phases\_json}) \neq \text{prev\_mtime}) \vee \text{cover}(\text{phases\_json}, \text{proj\_dir}) \\ & \quad \wedge \text{elif } e = [] \wedge \neg \text{is\_incremental} \text{ then } \\ & \quad \quad r = \text{True} \\ & \quad \text{else } \\ & \quad \quad r = \text{False} \\ & \quad \bigr) \\ & \quad \wedge \bigl( \\ & \quad \quad \text{quad } (r = \text{True} \rightarrow \text{break executed} \wedge \text{phase\_plan\_ready} = \text{True} \wedge \text{phase\_plan\_errors} = e) \\ & \quad \quad \vee (r = \text{False} \wedge \text{attempt} \geq \text{OPENCODE\_MAX\_RETRIES} \rightarrow \text{sys.exit}(1) \text{ called, program terminates}) \\ & \quad \quad \vee (r = \text{False} \wedge \text{attempt} < \text{OPENCODE\_MAX\_RETRIES} \rightarrow \\ & \quad \quad \quad f = \text{if is\_incremental then "update phases.json" else "produce phases.json"} \\ & \quad \quad \quad \wedge m = \text{if } e \neq [] \text{ then "phases.json schema validation failed: " } \end{aligned}$$

```

\mathbin{+} \text{join}(e, "; \backslash)
\\ \qquad \qquad \qquad \text{else if is_incremental then "phases.json was not updated" else
"phases.json missing or invalid"}
\\ \qquad \qquad \qquad \text{print/ log executed, time.sleep(10) executed}
\\ \qquad \qquad \qquad \text{phase_plan_ready} = \text{False} \ \text{and} \ \text{phase_plan_errors} = e
\\ \qquad \qquad \qquad \text{failure} = f \ \text{and} \ \text{missing} = m
\\ \qquad \qquad \qquad \text{the block ends, execution continues outside the block (e.g., to}
\text{increment attempt and retry)}}
\biggr)
\backslash)

```

- Code evidence:

```

Line 95: elif is_incremental:
Line 96: phase_plan_ready = (
Line 97: os.path.getmtime(phases_json) != prev_mtime
Line 98: or _phases_cover_current_sources(phases_json, proj_dir)
Line 99:)

```

- Trigger condition:

When `is_incremental` is true, the code sets `phase_plan_ready` to True solely because the file modification time changed, even if `_phases_cover_current_sources` returns False. The specification requires that a valid `phases.json` be accepted in incremental mode only when it covers all current source files; a timestamp change alone does not satisfy that requirement.

#### Bug validator

- Trigger summary: When `is_incremental=True` and `phases.json` mtime changes but `_phases_cover_current_sources` returns False, the OR operator sets `phase_plan_ready=True`, incorrectly accepting the incomplete `phases.json`.
- Probe stdout:

CONFIRMED – buggy condition (OR) produces `phase_plan_ready=True`, but spec-correct condition (AND) produces `phase_plan_ready=False`. `phases.json` was modified (mtime changed) but still does not cover all source files (missing `real_source.py`). The OR incorrectly accepts it as ready.  
 Bug location: `src/pipeline_setup.py`, lines 1003-1007  
 Current: `phase_plan_ready = (mtime_changed OR _phases_cover_current_sources(...))`  
 Should be: `phase_plan_ready = (mtime_changed AND _phases_cover_current_sources(...))`

#### INCR-MISMATCH-068 — `src--pipeline_setup.py--_setup_outputs_complete`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/pipeline_setup.py/_setup_outputs_complete.py`。
- Reasoner 结果: `logic_verification_results/src/pipeline_setup-py/_setup_outputs_complete.json`。
- 详细报告: `src--pipeline_setup.py--_setup_outputs_complete.md`。
- Probe: `probe_src--pipeline_setup.py--_setup_outputs_complete.py`。

#### 完整生成 SPEC

```

[SPEC]
Unit: src/pipeline_setup.py/_setup_outputs_complete.py

_setup_outputs_complete(work_dir) -> bool

Pre-condition:
- work_dir is a valid directory path that may or may not contain pipeline output files.

Post-condition:
- Returns True when phases.json, engine_overview.txt, and at least one file whose name matches
the pattern phase_NN_types.txt (where NN is one or more digits) all exist as regular files under

```



```
work_dir.
- Returns False when any one of the three required output categories is absent from work_dir.
[SPEC]
```

Reasoner 差异

- SPEC claim:

```
- Returns True when phases.json, engine_overview.txt, and at least one file whose name matches
the pattern phase_NN_types.txt (where NN is one or more digits) all exist as regular files under
work_dir.
- Returns False when any one of the three required output categories is absent from work_dir.
```

- 推导 actual behavior:

After the function returns, the boolean result R satisfies: R = True if and only if (a) a regular file "phases.json" exists under "work\_dir", its content parses as valid JSON and conforms to the required schema, and (b) both "engine\_overview.txt" and at least one regular file matching the pattern "phase\_NN\_types.txt" (with NN one or more digits) exist under "work\_dir". Otherwise R = False. The "work\_dir" path is unchanged.

- Code evidence:

Line 3

- Trigger condition:

The code returns False if phases.json is not valid JSON/schema-conformant, but the specification only requires the file's existence. Hence, for an input where phases.json exists but is invalid JSON, the code returns False while the specification mandates True.

Bug validator

- Trigger summary: phases.json exists but contains invalid JSON; \_phase\_plan\_complete checks JSON validity+ schema, but spec only requires file existence, so \_setup\_outputs\_complete returns False instead of True.
- Probe stdout:

CONFIRMED -- actual: False | expected: True

## src/prompts-py

INCR-MISMATCH-069 — `src--prompts-py--_generate_block_post_condition`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/prompts-py/_generate_block_post_condition.py`。
- Reasoner 结果: `logic_verification_results/src/prompts-py/_generate_block_post_condition.json`。
- 详细报告: `src--prompts-py--_generate_block_post_condition.md`。
- Probe: `probe_src--prompts-py--_generate_block_post_condition.py`。

```
[SPEC]
Unit: src/prompts-py/_generate_block_post_condition.py

_generate_block_post_condition(block, pre_condition, knowledge, language, trace_dir=None,
trace_meta=None)

Pre-condition:
- block is a non-empty string containing code statements, possibly with "Line N:" prefixes
- pre_condition is a non-empty string describing the logical state assumed to hold before
block begins execution
- knowledge is an optional string providing additional contextual information that may be
included in the prompt; may be empty, None, or otherwise falsy
- language is a non-empty string identifying the source programming language of the code in
block
- trace_dir, when not None, is a directory path used for persisting trace records
- trace_meta, when not None, is a dict of metadata for trace annotation

Post-condition:
- Returns a string describing the strongest post-condition that must hold after executing
block from the given pre-condition, covering all execution paths through the block including
normal flow-through, early returns, and exceptional exits
- Returns None when the post-condition could not be determined from the given inputs
- The returned post-condition is expressed in natural language suitable for subsequent logical
implication checks against specification post-conditions
[SPEC]
```

## Reasoner 差异

- SPEC claim:

```
- Returns a string describing the strongest post-condition that must hold after executing block
from the given pre-condition, covering all execution paths through the block including normal
flow-through, early returns, and exceptional exits
- Returns None when the post-condition could not be determined from the given inputs
- The returned post-condition is expressed in natural language suitable for subsequent logical
implication checks against specification post-conditions
```

- 推导 actual behavior:

After normal execution (no exception), the function returns the value produced by calling `\_llm\_json\_call` with the following arguments: client = `\_llm\_provider\_client`, model = `REASONER\_POST\_CONDITION\_MODEL`, messages = a list containing two dictionaries (a system message with content combining the expert role description and language-specific semantics, and a user message with the programming language, pre-condition, code block, and optional knowledge context), parse\_fn = `\_parse\_post\_condition\_json`, default = `{"post\_condition": "non-empty string"}`, trace\_dir = the input `trace\_dir`, and trace\_meta = a dictionary with `purpose`, `summary`, and any input `trace\_meta` merged. The return value is either a non-empty string (the extracted post-condition) or `None` if the LLM response could not be parsed. If `\_llm\_json\_call` raises an exception, that exception propagates upward and the function terminates abnormally. No input arguments are mutated, and the only possible side effect is the trace persistence in `trace\_dir` if provided.

Formally:

```
let info_str = if (knowledge is truthy) then "\nAdditional context:\n" + knowledge else "" in
let messages = [
 {role: "system", content: "You are an expert in formal verification of " + language + "
programs. Given a " + language + " code block and its pre-condition, generate the post-condition
that describes the program state after the code block finishes execution. Cover all execution
paths including early returns, exceptions, and normal flow-through. Apply " + language + "-
specific semantics (ownership, lifetimes, error handling, etc.) as appropriate. Be precise and
unambiguous. Express the post-condition in natural language and formal logic."},
 {role: "user", content: "Programming language: " + language + "\n\nPre-condition:\n" +
pre_condition + "\n\nCode block:\n`" + language_lower + "\n" + block + "\n`\n" + info_str +
"\nGenerate the post-condition. Return only a valid JSON object with this required field:
{\"post_condition\": \"...\\"}. Do not include Markdown, tags, or prose outside the JSON
object."}]
] in
let meta = {"purpose": "generate_block_post_condition", "summary": "Generated post-condition for
code block"} + (trace_meta or {}) in
(result = _llm_json_call(_llm_provider_client, REASONER_POST_CONDITION_MODEL, messages,
_post_post_condition_json, "{\"post_condition\": \"non-empty string\"}", trace_dir, meta)
 (result String {None})
 (result None result is a non-empty string))
```

```
(Exception E : E is raised by _llm_json_call the current function raises E)
```

- Code evidence:

```
Line 28: return _llm_json_call(...)
```

- Trigger condition:

The specification requires that the function returns None when the post-condition cannot be determined, implying all failure modes should be handled gracefully. The code does not catch exceptions from `_llm_json_call`, so any runtime error (e.g., network failure) results in an unhandled exception, violating the specification.

#### Bug validator

- Trigger summary: Call `_generate_block_post_condition` while `_llm_json_call` raises a `RuntimeError` (simulating network failure) — the exception propagates uncaught instead of returning None as the spec requires.
- Probe stdout:

```
CONFIRMED — exception propagated instead of returning None | exception type: RuntimeError |
message: Simulated LLM API failure — network error
```

---

#### INCR-MISMATCH-070 — `src--prompts-py--_parse_spec_check_json`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/prompts-py/_parse_spec_check_json.py`。
- Reasoner 结果: `logic_verification_results/src/prompts-py/_parse_spec_check_json.json`。
- 详细报告: `src--prompts-py--_parse_spec_check_json.md`。
- Probe: `probe_src--prompts-py--_parse_spec_check_json.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/prompts-py/_parse_spec_check_json.py

_parse_spec_check_json(response)

Pre-condition:
- response is a non-empty string

Post-condition:
- Raises ValueError if response is not valid JSON text
- Raises ValueError if the parsed JSON value is not a mapping (dict)
- Raises ValueError if the parsed mapping does not contain all of the required keys:
"verdict", "counterexample", "offending_statements", "reason"
- Raises ValueError if the "verdict" value, after conversion to uppercase, is neither "MATCH"
nor "MISMATCH"
- Raises ValueError if "counterexample" is present and not null-valued but is not a string
- Raises ValueError if "offending_statements" is present and not null-valued but is not a
string
- Raises ValueError if "reason" is not a string value
- For "MISMATCH" verdict: raises ValueError if any of counterexample, offending_statements, or
reason is empty or consists only of whitespace; otherwise returns a tuple (True,
offending_statements_with_leading_trailing_whitespace_removed, reason_with_whitespace_removed,
data) where data is the parsed dict with verdict uppercased, counterexample set to the stripped
value, offending_statements set to the stripped value, and reason set to the stripped value
- For "MATCH" verdict: raises ValueError if counterexample or offending_statements is a non-
empty string; otherwise returns a tuple (False, None, None, data) where data is the parsed dict
with verdict uppercased, counterexample set to None, offending_statements set to None, and
```

reason set to its stripped value  
[SPEC]

## Reasoner 差异

- SPEC claim:

```
- Raises ValueError if response is not valid JSON text
- Raises ValueError if the parsed JSON value is not a mapping (dict)
- Raises ValueError if the parsed mapping does not contain all of the required keys:
"verdict", "counterexample", "offending_statements", "reason"
- Raises ValueError if the "verdict" value, after conversion to uppercase, is neither "MATCH"
nor "MISMATCH"
- Raises ValueError if "counterexample" is present and not null-valued but is not a string
- Raises ValueError if "offending_statements" is present and not null-valued but is not a
string
- Raises ValueError if "reason" is not a string value
- For "MISMATCH" verdict: raises ValueError if any of counterexample, offending_statements, or
reason is empty or consists only of whitespace; otherwise returns a tuple (True,
offending_statements_with_leading_trailing_whitespace_removed, reason_with_whitespace_removed,
data) where data is the parsed dict with verdict uppercased, counterexample set to the stripped
value, offending_statements set to the stripped value, and reason set to the stripped value
- For "MATCH" verdict: raises ValueError if counterexample or offending_statements is a non-
empty string; otherwise returns a tuple (False, None, None, data) where data is the parsed dict
with verdict uppercased, counterexample set to None, offending_statements set to None, and
reason set to its stripped value
```

- 推导 actual behavior:

Given a non-empty string `response`, the function `_parse_spec_check_json` either raises a `ValueError` or returns a tuple `(match_flag, offending, reason, data)`. The possible outcomes are:

1. If `_load_spec_check_json(response)` raises a `JSONDecodeError`, a `ValueError` with message `'spec-check response is not valid JSON: {exc}'` is raised.
2. Otherwise, let `data` be the parsed JSON object. If `data` is not a dictionary (`not isinstance(data, dict)`), a `ValueError` with message `'spec-check JSON must be an object'` is raised.
3. If any of the required fields `'verdict'`, `'counterexample'`, `'offending_statements'`, `'reason'` are missing from `data`, a `ValueError` is raised with message `'spec-check JSON missing required field(s): ...'` listing the missing keys.
4. Otherwise, let `verdict_raw = data['verdict']`, `counterexample_raw = data.get('counterexample')`, `offending_statements_raw = data.get('offending_statements')`, `reason_raw = data.get('reason')`. Let `verdict = verdict_raw.upper()` if `isinstance(verdict_raw, str)` else `verdict_raw`. If `verdict not in ('MATCH', 'MISMATCH')`, a `ValueError` is raised with message `'spec-check JSON verdict must be MATCH or MISMATCH'`.
5. If `counterexample_raw` is not `None` and not `isinstance(counterexample_raw, str)`, raise `ValueError('spec-check JSON field counterexample must be a string or null')`. If `offending_statements_raw` is not `None` and not `isinstance(offending_statements_raw, str)`, raise `ValueError('spec-check JSON field offending_statements must be a string or null')`. If `not isinstance(reason_raw, str)`, raise `ValueError('spec-check JSON field reason must be a string')`.
6. Update `data['verdict'] = verdict`.
  - **Case MISMATCH** (`verdict == 'MISMATCH'`):
    - Define `valid = lambda x: isinstance(x, str) and bool(x.strip())`. If any of `counterexample_raw`, `offending_statements_raw`, `reason_raw` fails `valid(x)`, raise `ValueError('spec-check MISMATCH JSON missing non-empty field(s): ...')` listing those failing.
    - Else, set `data['counterexample'] = counterexample_raw.strip()`, `data['offending_statements'] = offending_statements_raw.strip()`, `data['reason'] = reason_raw.strip()`. Return `(True, data['offending_statements'], data['reason'], data)`.
  - **Case MATCH** (`verdict == 'MATCH'`):
    - If `valid(counterexample_raw)` or `valid(offending_statements_raw)`, raise `ValueError('spec-check MATCH JSON must not include counterexample or offending_statements')`.
    - Else, set `data['counterexample'] = None`, `data['offending_statements'] = None`, `data['reason'] = reason_raw.strip()`. Return `(False, None, None, data)`.

Formally:

```
\[
\begin{aligned}
&\&\text{pre: } response \in \Sigma^+ \\\
&\&\text{post: } \left(\begin{aligned}
&\&(\neg \text{valid_json}(response) \Rightarrow \text{raise ValueError}) \\\
&\&(\text{valid_json}(response) \wedge D = \text{parse}(response) \wedge \neg \text{isinstance}(D, \text{dict}) \\
&\&\Rightarrow \text{raise ValueError}) \\\
&\&(\text{isinstance}(D, \text{dict}) \wedge \text{keys_missing}(D) \Rightarrow \text{raise ValueError}) \\\
&\&(\text{isinstance}(D, \text{dict}) \wedge \neg \text{keys_missing}(D) \wedge \neg \text{valid_verdict}(D) \Rightarrow \\
&\&\text{raise ValueError}) \\\
\end{aligned} \right)
\end{aligned}
```

```

&\land (\text{valid fields and verdict} \land \neg \text{valid_counterexample_type}(D) \Rightarrow
\text{raise ValueError}) \land
&\land (\text{valid fields and verdict} \land \neg \text{valid_offending_type}(D) \Rightarrow
\text{raise ValueError}) \land
&\land (\text{valid fields and verdict} \land \neg \text{valid_reason_type}(D) \Rightarrow
\text{raise ValueError}) \land
&\land (\text{valid types} \land \text{verdict_up} = \text{MISMATCH} \land \neg
\text{all_nonempty_strings}(D) \Rightarrow \text{raise ValueError listing failing fields}) \land
&\land (\text{valid types} \land \text{verdict_up} = \text{MISMATCH} \land \text{all_nonempty_strings}(D)
\Rightarrow \text{return } (True, \text{strip}(0), \text{strip}(R), D')) \land
&\land (\text{valid types} \land \text{verdict_up} = \text{MATCH} \land (\text{nonempty_str}(C) \lor
\text{nonempty_str}(0)) \Rightarrow \text{raise ValueError}) \land
&\land (\text{valid types} \land \text{verdict_up} = \text{MATCH} \land \neg \text{nonempty_str}(C) \land
\neg \text{nonempty_str}(0) \Rightarrow \text{return } (False, None, None, D'))
\end{aligned} \right)
\end{aligned}
\]
where predicates follow the Python code semantics.

```

- Code evidence:

```

Line 47: if _nonempty_string(counterexample) or _nonempty_string(offending_statements):
Line 48: raise ValueError(
Line 49: "spec-check MATCH JSON must not include counterexample or offending_statements"
Line 50:)

```

- Trigger condition:

The specification states: for MATCH verdict, raises ValueError if counterexample or offending\_statements is a non-empty string. A whitespace-only string (e.g., ' ') is a non-empty string (length > 0), so it should raise ValueError. The code's \_nonempty\_string checks bool(value.strip()), treating whitespace-only strings as empty, and does not raise an error, violating the spec.

#### Bug validator

- Trigger summary: Whitespace-only counterexample/offending\_statements strings bypass the MATCH-verdict ValueError check because \_nonempty\_string uses bool(value.strip()) instead of len(value) > 0.
- Probe stdout:

```

CONFIRMED - _nonempty_string treats whitespace-only as empty, but spec requires ValueError for
any non-empty string. Actual: returned tuple (False, None, None) (no error) | Expected:
'ValueError'

```

## src/reasoner-py

### INCR-MISMATCH-071 — src--reasoner-py--\_compute\_brace\_depth\_per\_line

- 人工审计：实现缺陷候选。
- Validator: **confirmed**；尝试次数: 1。
- 原始函数与 SPEC: [src/reasoner-py/\\_compute\\_brace\\_depth\\_per\\_line.py](#)。
- Reasoner 结果: [logic\\_verification\\_results/src/reasoner-py/\\_compute\\_brace\\_depth\\_per\\_line.json](#)。
- 详细报告: [src--reasoner-py--\\_compute\\_brace\\_depth\\_per\\_line.md](#)。
- Probe: [probe\\_src--reasoner-py--\\_compute\\_brace\\_depth\\_per\\_line.py](#)。

#### 完整生成 SPEC

```

[SPEC]
Unit: src/reasoner-py/_compute_brace_depth_per_line.py

_compute_brace_depth_per_line(lines) -> list[int]

```

Pre-condition:

- lines is a non-empty list of strings, each containing one line of function body text without "Line N:" prefixes
- The input represents a function body with balanced braces, i.e., every '}' has a matching '{' earlier in the sequence, so the cumulative depth never becomes negative.

Post-condition:

- Returns a list of integers whose length equals len(lines)
- For each index i, the integer is the cumulative net count of '{' minus '}' characters encountered across lines[0] through lines[i], inclusive, after applying the following exclusion rules:
  - \* Characters inside a double-quoted string literal (delimited by unescaped '"', where backslash escapes the next character) are excluded.
  - \* Characters inside a single-quoted character literal (delimited by unescaped "'", with backslash escape) are excluded.
  - \* From the point where the sequence "/\*" occurs outside any literal, all remaining characters on that line are excluded.
  - \* From the point where the sequence "/\*" occurs outside any literal, all remaining characters on that line are excluded (the simplified implementation does not propagate the block comment into subsequent lines).
- Because of the balanced-brace precondition, each returned integer is non-negative.

[SPEC]

## Reasoner 差异

- SPEC claim:

- Returns a list of integers whose length equals len(lines)
  - For each index i, the integer is the cumulative net count of '{' minus '}' characters encountered across lines[0] through lines[i], inclusive, after applying the following exclusion rules:
    - \* Characters inside a double-quoted string literal (delimited by unescaped '"', where backslash escapes the next character) are excluded.
    - \* Characters inside a single-quoted character literal (delimited by unescaped "'", with backslash escape) are excluded.
    - \* From the point where the sequence "/\*" occurs outside any literal, all remaining characters on that line are excluded.
    - \* From the point where the sequence "/\*" occurs outside any literal, all remaining characters on that line are excluded (the simplified implementation does not propagate the block comment into subsequent lines).
  - Because of the balanced-brace precondition, each returned integer is non-negative.

- 推导 actual behavior:

The function returns a list `depths` such that:

- `len(depths) == len(lines)`.
- For each i (0 ≤ i < len(lines)), let prefix depth d = 0 if i == 0 else depths[i-1]. Then depths[i] is the depth after scanning lines[i] starting from d, where the scan processes characters left-to-right, skipping:
  - \* double-quoted string literals (enclosed by '"', with backslash escape sequences `\\` causing the next character to be skipped),
  - \* single-quoted character literals (enclosed by "'", same escape rule),
  - \* line comments starting with `//` (skipping the rest of the line),
  - \* block comments starting with `/\*`: characters are skipped until `\*/` is found on the same line; if `\*/` is not found, the rest of the line is skipped.
- Outside these skipped regions, each `{` increments the depth by 1 and each `}` decrements the depth by 1. Nothing else changes the depth.
- Because the input represents a function body with balanced braces (every '}' has a matching '{' earlier, cumulative depth never negative), the final depth after processing the last line is 0, i.e., `depths[-1] == 0`. Formally, let `scan(s, d)` be the result of applying the above rules to string `s` with starting depth `d`. Then for all i: `depths[i] = scan(lines[i], 0 if i == 0 else depths[i-1])`, and `depths[-1] == 0`.

- Code evidence:

```

Line 13: if ch == '"':
Line 14: i += 1
Line 15: while i < len(line):
Line 16: if line[i] == '\\':
Line 17: i += 2
Line 18: continue
Line 19: if line[i] == '"':
Line 20: i += 1

```

```
Line 21: break
Line 22: i += 1
Line 23: continue
```

- Trigger condition:

The specification requires that characters inside a double-quoted string literal be excluded regardless of line boundaries (a string literal is defined by unescaped double quotes without a line limit). The code's string-scanning loop (lines 13-23) only scans until the end of the current line; if no closing quote is found on that line, the string state is reset at the next line. This causes braces inside a multi-line string to be incorrectly counted, violating the specification. For the concrete input `["", "{", "}"]`, the code produces depths `[0,1,0]` while the specification requires `[0,0,0]`.

#### Bug validator

- Trigger summary: Multi-line string not tracked: unterminated double quote on line 0 causes braces on subsequent lines to be counted instead of excluded.
- Probe stdout:

CONFIRMED – actual: `[0, 1, 0]` | expected: `[0, 0, 0]`

---

#### INCR-MISMATCH-072 — `src--reasoner-py--_split_into_blocks_braced`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/reasoner-py/_split_into_blocks_braced.py`。
- Reasoner 结果: `logic_verification_results/src/reasoner-py/_split_into_blocks_braced.json`。
- 详细报告: `src--reasoner-py--_split_into_blocks_braced.md`。
- Probe: `probe_src--reasoner-py--_split_into_blocks_braced.py`。

#### 完整生成 SPEC

```
[SPEC]
Unit: src/reasoner-py/reasoner.py

_split_into_blocks_braced(func, language) -> list[str]

Pre-condition:
- func is a non-empty string containing the body of a function, each line optionally prefixed
 with "Line N: " followed by the source text
- language is a string identifying the source programming language

Post-condition:
- Returns a non-empty list of strings, each being a contiguous, non-overlapping segment of the
 function body, appearing in the same order as in the original body
- For brace-delimited languages: every boundary between consecutive returned segments occurs
 at a line whose brace-nesting depth equals the brace-nesting depth at the first meaningful brace
 level of the function body. Each segment is therefore syntactically self-contained with respect
 to brace structure – it begins and ends at the same nesting depth as the function's entry depth
- For languages that use indentation for syntactic structure (classified as Python-like by the
 function): segments are split at boundaries that respect indentation-level changes, with each
 segment being a syntactically contiguous block
- The ordered concatenation of all returned strings, joined with newline characters,
 reconstructs the original function body text after stripping leading and trailing whitespace
[SPEC]
```

#### Reasoner 差异

- SPEC claim:

- Returns a non-empty list of strings, each being a contiguous, non-overlapping segment of the function body, appearing in the same order as in the original body
  - For brace-delimited languages: every boundary between consecutive returned segments occurs at a line whose brace-nesting depth equals the brace-nesting depth at the first meaningful brace level of the function body. Each segment is therefore syntactically self-contained with respect to brace structure it begins and ends at the same nesting depth as the function's entry depth
  - For languages that use indentation for syntactic structure (classified as Python-like by the function): segments are split at boundaries that respect indentation-level changes, with each segment being a syntactically contiguous block
  - The ordered concatenation of all returned strings, joined with newline characters, reconstructs the original function body text after stripping leading and trailing whitespace

- 推导 actual behavior:

The function returns a non-empty list of strings `blocks`. Let `preprocessed` be the string obtained by taking `func`, stripping leading and trailing whitespace, splitting into lines, removing any leading "Line N: " prefix (where N is a number) from each line, left-stripping the remainder, and joining the results with newline characters. Then `'\n'.join(blocks) == preprocessed`. If `language.lower()` is in the set `{"python"}` or if the brace depth entry point is zero (i.e., after computing brace depth per line, the first non-zero depth is not found and the initial depth is zero), then `blocks` is exactly the result of calling `_split_into_blocks(func)`. Otherwise, let `lines` be the list of preprocessed lines, let `depths = _compute_brace_depth_per_line(lines)`, and let `entry` be `depths[0]` if `depths[0] > 0`, else the first positive value in `depths` (or 0 if none). Then `blocks` is the partition of `lines` generated by the following algorithm: start with index `i = 0`. While `i < len(lines)`, if the number of remaining lines `len(lines) - i` is less than or equal to `2 * GRANULARITY`, append all remaining lines as the final block and stop. Else, set `target = i + GRANULARITY`; search for the smallest index `j >= target` such that `depths[j] == entry`. If such `j` exists, append lines `i` through `j` (inclusive) as a block, set `i = j + 1`; otherwise, append all remaining lines from `i` as the final block and stop. No block is ever empty.

- Code evidence:

```
Line 23: entry_depth = depths[0] if total > 0 else 0; Line 24-25: entry_depth = next((d for d in
depths if d > 0), 0); Line 40-43: for j in range(target, total): if depths[j] == entry_depth:
split_point = j; break
```

- Trigger condition:

The specification requires that each segment begins and ends at the brace-nesting depth equal to the function's entry depth. The code computes `entry_depth` as `depths[0]` if positive, otherwise the first positive depth. When `depths[0]` is 0 and a positive depth exists later, `entry_depth` becomes that positive depth. The splitting algorithm then searches for a line with depth equal to `entry_depth` starting from `i+GRANULARITY`. For the first block, line 0 has depth 0, but the block ends at a line with depth == `entry_depth`. Thus the first block starts at depth 0 and ends at depth `entry_depth`, violating the requirement that it begins at entry depth. For the given counterexample (C function body with opening brace on the second line, `GRANULARITY=1`), the code splits after line 1, producing a first block `["int main()", "{}"]` that starts at depth 0 and ends at depth 1, not syntactically self-contained w.r.t. entry depth.

#### Bug validator

- Trigger summary: C function body with opening brace on second line: `entry_depth=1` but first block starts at depth 0, violating spec requirement that segments begin at entry depth.
- Probe stdout:

```
CONFIRMED - first block starts at depth 0, not entry_depth 1. Block: 'int main()\n{' |
all_blocks: ['int main()\n{', ' return 0;\n}'] | depths: [0, 1, 1, 0]
```

## src/verification-py



- 人工审计：实现缺陷候选。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/verification-py/streaming_reasoner.py`。
- Reasoner 结果: `logic_verification_results/src/verification-py/streaming_reasoner.json`。
- 详细报告: `src--verification-py--streaming_reasoner.md`。
- Probe: `probe_src--verification-py--streaming_reasoner.py`。

## 完整生成 SPEC

```
[SPEC]
Unit: src/verification-py/streaming_reasoner.py

streaming_reasoner(input_dir, output_dir, file_list=None, proj_dir=None, work_dir=None,
poll_interval=2, spec_procs=None, already_processed=None, resume=False) -> set

Pre-condition:
- input_dir is a directory path containing extracted function files, where each
 file is expected to eventually have [SPEC]/[INFO] blocks prepended
```

## Reasoner 差异

- SPEC claim:

```
- Every file in input_dir (scoped to file_list when provided) whose is_file_ready()
 returns True will be submitted to _verify_single_file exactly once; a readied
 file that was in already_processed is NOT resubmitted
- A verification result JSON is written to output_dir for every submitted file,
 mirroring the relative path structure of input_dir; the verdict field in each
 result is one of: "MATCH", "MISMATCH", "ERROR", "SKIPPED"
- For every verified file whose verdict is "MISMATCH" and proj_dir is not None,
 a bug-validation task is submitted via _validate_single_bug; the validation
 writes a result JSON at proj_dir/bug_validation/<bug_id>.result.json where
 bug_id is derived from the result JSON path by stripping the
 fm_agent/logic_verification_results/ prefix, removing ".json", and replacing
 "/" with "--"
- Progress output is printed for each completed verification: MATCH and SKIPPED
 files are marked with a green check, confirmed bugs with a red cross; each
 line is prefixed with "[<N>/<total>] <relative_path>: <label>"
- When all expected files have been verified, all reasoning futures are done,
 and no validation futures remain in-flight, the loop exits normally
- When spec_procs is provided and every process has exited via _spec_task_done,
 and not all expected files are ready: the function exits with a warning.
 If no files received specs at all, the warning states no [SPEC]/[INFO] markers
 were observed; otherwise it reports how many files are missing specs and
 lists each as "[pending]"
- On KeyboardInterrupt: all in-flight reasoning and validation futures are
 waited on to completion before the function returns
- If proj_dir is not None, _generate_validation_summary is called after all
 processing ends (normal exit, early-exit on stalled specs, or interrupt),
 producing proj_dir/bug_validation/summary.json
- Returns a set of file paths (absolute paths within input_dir) that were
 successfully verified; when already_processed is provided, the returned set
 includes those files as well (it is a superset). The returned set is a distinct
 object (a copy) from already_processed.
```

- 推导 actual behavior:

```
validation_futures = (V0 S) \ Done, where S = {(vf, (fpath, rel_path, result_json_rel,
completed_count))} if and only if verdict = "MISMATCH" and the assignment validation_futures[vf]
= ... succeeded before any exception, otherwise S = . Done = { f | f dom(V0 S) f.done() }.
All other variables (processed, submitted, reasoning_futures, completed_count, num_functions,
executor, work_dir, proj_dir, resume, expected_files, poll_interval, spec_procs,) retain their
preblock values. No exception propagates out of the block; logs, file I/O, and print output may
have occurred but do not affect the abstract state.
```

- Code evidence:

```
Line 133: `if _all_procs is not None and all(_spec_task_done(p) for p in _all_procs):`
Line 134: `unready = (expected_files or set()) - processed`
Line 135: `if unready and not reasoning_futures and not validation_futures:`
```

- Trigger condition:

The early exit condition triggered when all spec\_procs have exited does not check whether the unprocessed expected files are actually ready (i.e., have the required markers). If files already contain the [SPEC] and [INFO] markers before any processing occurs, and the spec\_procs list is empty or all processes have finished, the loop breaks prematurely, leaving ready files unsubmitted. This violates the requirement that every file whose is\_file\_ready() returns True must be submitted to \_verify\_single\_file exactly once.

#### Bug validator

- Trigger summary: Early exit when all spec\_procs finish does not call is\_file\_ready() on remaining expected files; ready files are skipped and never submitted for verification.
- Probe stdout:

```
Functions pending verification: 2
WARNING:root:Spec generation process(es) exited (codes [0]) but no files received [SPEC]/[INFO]
markers.
CONFIRMED - missed ready file(s): ['ready_file.py', 'ready_file2.py'] | processed: []
```

## 产物位置

- 本轮增量 SPEC 更新列表: [incremental\\_updated\\_specs.json](#)。
- 全部逻辑结果: [logic\\_verification\\_results/](#)。
- Bug validator 汇总: [bug\\_validation/summary.json](#)。
- 逐条报告与 probe: [bug\\_validation/](#)。
- 完整运行日志: [/tmp/fm-agent-incremental-main-20260721.log](#) (位于本机 /tmp, 不在 target 仓库内)。

## 人工复核追加：最可能的实际 Bug

### 复核口径与总括

本节不把 Bug validator 的 **confirmed** 直接等同于“FM-Agent 源码中存在真实 Bug”。**confirmed** 只说明 probe 构造出的输入能够使实现偏离同一轮生成的 SPEC；如果 SPEC 擅自扩大了输入域、规定了并不存在的返回格式，或者 probe 依赖 monkeypatch 制造不可能状态，仍然可能是假阳性。

这里仅保留下列同时具备独立证据的候选：

1. 触发输入符合真实调用链，或可能由 LLM、并发写文件、环境缺失等正常运行条件产生；
2. 不依赖生成 SPEC，也能从源码注释、路径约束、协议语义或下游无保护调用推出错误后果；
3. 能指出具体的错误分支以及可观察影响，而不只是返回值与 SPEC 的文字不同。

按这个标准，73 个 mismatch 中目前最值得处理的是 **7 项**：**5 项高置信度**、**1 项中高置信度**、**1 项中等置信度**。其中只有 [\\_phase\\_plan\\_schema\\_errors](#) 的问题明确来自本轮比较区间内的新提交 [29a4c57](#)；其余大多是旧代码中的潜伏问题，因为“增量验证覆盖到某函数”并不等于“该 Bug 是两个版本之间新引入的”。

| 优先级 | 人工结论                                                     | 对应 mismatch       | 置信度 | 核心后果                                                                                |
|-----|----------------------------------------------------------|-------------------|-----|-------------------------------------------------------------------------------------|
| P0  | phase 路径未限制在项目根目录内                                       | INCR-MISMATCH-066 | 高   | 可读取项目外源码，并把提取结果写到 <code>fm_agent/</code> 之外                                         |
| P1  | phase schema 校验器接受会令下游崩溃的结构                              | INCR-MISMATCH-065 | 高   | <code>malformedphases.json</code> 被视为完整，随后触发 <code>KeyError</code> ；非法 UTF-8 还会直接逃逸 |
| P1  | 增量 phase 计划只要 mtime 改变就可绕过覆盖检查                           | INCR-MISMATCH-067 | 高   | 漏列当前源码的计划仍被接受，后续函数不提取、不验证                                                           |
| P1  | codegraph 不可用时先删除旧索引                                     | INCR-MISMATCH-036 | 高   | 本可继续使用的 <code>.codegraph/</code> 被不可恢复地丢弃，随后静默降级                                    |
| P1  | brace-depth 扫描没有保存跨行词法状态                                 | INCR-MISMATCH-071 | 高   | 注释/续行字符串内的花括号污染分块，进而造成推理误报或漏报                                                       |
| P2  | spec 进程结束检查存在最后一次扫描竞态                                    | INCR-MISMATCH-073 | 中高  | 已写完 SPEC 的函数可能被留在 pending，直到下次恢复才处理                                                 |
| P2  | Erlang LSP 行索引错误地采用 Python 的广义 <code>splitlines()</code> | INCR-MISMATCH-045 | 中   | 含 VT/FF 等控制字符时，ELP range 映射到错误源码片段                                                  |

## ACTUAL-001 (P0) — `phases.json` 可用绝对路径或 `..` 逃逸项目目录

- 对应报告：INCR-MISMATCH-066 / `_phases_cover_current_sources`。
- 主要代码位置：`src/pipeline_setup.py:731-751`、`src/extract.py:715-776`。
- 人工结论：高置信度实际 Bug，而且实际影响比 `validator` 报告的“错误接受绝对路径”更严重。

独立依据如下：

1. `workflow` 明确要求 `source_files` 是相对项目根目录的路径（`pipeline_setup.py:879-884`），但 `schema` 校验只检查它是字符串。
2. `_phases_cover_current_sources()` 在 `pipeline_setup.py:749` 使用 `os.path.exists(os.path.join(proj_dir, sf))`。当 `sf` 是绝对路径时，`os.path.join()` 会丢弃 `proj_dir`；当 `sf` 是 `../outside.c` 时，也没有做 `realpath/commonpath` 边界校验。
3. 只要 `plan` 同时列出项目当前源码，最后的 `current_sources.issubset(listed)` 就会成立；多出来的项目外路径不会被拒绝。
4. 下游 `run_extraction()` 在 `extract.py:722` 再次直接 `join(proj_dir, src_rel)`，因此会读取项目外文件。更严重的是，`extract.py:742` 用绝对 `src_dir` 拼接 `output_base` 时，`output_base` 同样会被丢弃；随后 `os.makedirs()` 和 `open(..., "w")`（753、775 行）可能在 `fm_agent/` 之外创建目录和函数文件。

真实触发条件不是手工传入异常内部参数，而是 LLM 生成或更新 `phases.json` 时误写绝对路径/父目录路径。该文件本来就是非可信的模型产物，所以仅靠 `prompt` 约束不足以形成安全边界。

建议修复时同时做两层保护：在 `phase` 校验阶段拒绝绝对路径、空路径和任何解析后不位于 `realpath(proj_dir)` 下的路径；在 `run_extraction()` 写输出前再次验证 `src_path` 与 `out_dir` 的 `containment`，避免单点校验被绕过。

## ACTUAL-002 (P1) — `_phase_plan_schema_errors()` 的“完整 schema”并不完整

- 对应报告: `INCR-MISMATCH-065` / `_phase_plan_schema_errors`。
- 主要代码位置: `src/pipeline_setup.py:624-681`、`src/pipeline_setup.py:247-280`、`src/generate_topdown_layers.py:28-38`。
- 引入范围: 校验器由本轮比较区间内的 `29a4c57` (`fix(setup): retry malformed phase plans`) 加入。
- 人工结论: 高置信度实际 **Bug**; `validator` 的非法 **UTF-8** 例子成立, 但还存在更容易到达的结构漏检。

该函数声称验证 `required schema`, 却只验证 `phases`、`modules` 和 `source_files` 的容器类型, 没有要求:

- 每个 `phase` 必须有可用的 `phase` 编号;
- 每个 `module` 必须有字符串 `name`;
- `source_files` 必须是项目内、受支持且规范化的相对路径。

因此下面这种 **JSON** 会返回空错误列表, 被 `_phase_plan_complete()` 当作完整计划:

```
{"phases": [{ "modules": [{ "source_files": ["src/a.py"]}]}]}
```

但后续 `_deduplicate_phases()` 会在 `pipeline_setup.py:266` 访问 `p["phase"]`, `generate_topdown_layers._collect_phase_files()` 会在 `generate_topdown_layers.py:37` 访问 `module["name"]`, 都会对已经“通过 `schema`”的数据抛出 `KeyError`。这证明问题不依赖 **LLM SPEC**, 而是校验器与其直接消费者的真实契约不一致。

原 `mismatch` 指出的编码分支也是真问题: `open(..., "r").json.load()` 可能抛 `UnicodeDecodeError`, 当前只捕获 `OSError` 和 `JSONDecodeError`。`_run_generate_phases()` 在进入修复重试前就调用该函数, 因此一个截断或非 **UTF-8** 的模型产物可直接中止 `pipeline`, 而不是进入“报告 `schema` 错误并重试”的设计路径。

建议把 `phase`、`name`、路径规范以及必要字段一次性纳入 `schema`; 读取异常至少覆盖 `UnicodeError`。校验后仍应让下游使用防御式 `.get()` 或给出带字段路径的明确错误, 避免再次形成“校验器说合法、消费者却崩溃”的缝隙。

## ACTUAL-003 (P1) — 增量 `phase` 计划的 `mtime` 与覆盖完整性被错误地写成 **OR**

- 对应报告: `INCR-MISMATCH-067` / `_run_generate_phases`。
- 代码位置: `src/pipeline_setup.py:997-1007`。
- 人工结论: 高置信度实际 **Bug**。

增量分支当前判定为:

```
phase_plan_ready = (
 os.path.getmtime(phases_json) != prev_mtime
 or _phases_cover_current_sources(phases_json, proj_dir)
)
```

`mtime` 改变只能证明文件被写过, 不能证明更新后的计划覆盖当前源码。这里的两项实际上应满足两个不同必要条件: 模型确实更新过旧计划, 并且更新结果仍完整。当前 **OR** 允许模型只改一个描述、空白或删除某些 `source_files` 后立即通过。

下游 `run_extraction()` 仅遍历 `phases.json` 中列出的 `source_files` (`extract.py:703-715`), 没有一个全项目补扫步骤。因此被漏掉的源文件不会提取函数, 也不会进入 **SPEC** 生成和逻辑验证; 最终表

现可能是“增量运行成功，但一部分改动完全没被验证”。这与增量模式自己的 prompt（要求添加新文件、移除已删除文件并保留仍准确条目）直接冲突。

该 **OR** 在比较基点 **95fd9ed...** 前已经存在；本轮之所以发现它，是因为相关 **setup** 函数重新进入了增量验证范围，并不代表它由当前 **merge** 新引入。

建议将“是否发生更新”和“更新后是否完整”分开记录；在需要强制本轮更新时使用 **mtime\_changed and covers\_current\_sources**，若允许原计划本来就完整则显式设计成两个有名称的分支，不能用一个无区分的 **OR** 吞掉覆盖失败。

## ACTUAL-004 (P1) — **try\_codegen\_init(force=True)** 在确认可重建前删除旧数据库

- 对应报告：**INCR-MISMATCH-036 / try\_codegen\_init**。
- 代码位置：**src/languages/codegraph.py:500-537**。
- 真实调用点示例：**src/incremental\_reasoner.py:794-799**。
- 人工结论：**高置信度实际 Bug**。

当 **.codegen/codegraph.db** 已存在且 **force=True** 时，代码在 526 行先执行 **shutil.rmtree(codegraph\_dir)**，到 530-536 行才解析并运行可执行文件。如果配置的 **pinned binary** 不存在、**PATH** 中也没有 **codegen**，**subprocess.run()** 抛 **FileNotFoundError** 后函数静默返回。此时旧数据库已经消失。

这与函数 **docstring** 中“未安装时静默跳过，让 **pipeline** 回退”的行为不等价：真正安全的跳过应不修改已有状态。对增量流程尤其明显，**run\_incremental\_pipeline** 在重新提取前调用该函数；一次环境变动或安装损坏会把可用索引删掉，随后 **C/C++** 等语言静默切换到精度较低的 **regex** 路径，使本轮函数边界和调用图与上一轮不一致。

建议先验证 **executable** 可执行，再把新 **index** 构建到临时目录；只有 **codegen init** 成功后才原子替换旧目录。至少也应把旧目录先重命名为备份，并在启动失败或非零退出时恢复。

## ACTUAL-005 (P1) — **brace-depth** 计算器没有跨行保存注释/字符串状态

- 对应报告：**INCR-MISMATCH-071 / \_compute\_brace\_depth\_per\_line**。
- 代码位置：**src/reasoner.py:25-79**、下游使用位置 **src/reasoner.py:82-147**。
- 人工结论：**高置信度实际 Bug**。**validator** 使用“跨行字符串”作为例子略有语言差异，但跨行块注释给出了不依赖 **SPEC** 的标准 **C/C++** 反例。

函数在每一行开始时都重置 **i=0**，没有 **in\_block\_comment**、**in\_string** 等跨行状态。遇到 **/\*** 后只扫描当前行；如果本行没有 **\*/**，下一行仍按普通代码处理。源码第 71 行的注释声称“block comment spans lines 时忽略其中花括号”，但实现实际上没有做到。

例如：

```
int f() {
 /* comment starts
 */
 return 0;
}
```

注释里的 **}** 会被 75-76 行当成真实闭括号，深度提前从 1 降到 0，函数末尾再降到 -1。

**\_split\_into\_blocks\_braced()** 随后把这份错误 **depth** 数组当成“安全语法边界”，可能在注释或嵌套块附近错误切分。**reasoner()** 又会按这些 **block** 串联 LLM 生成的 **post-condition**，所以影响不是显示层错误，而是会直接增加推理误报与漏报。

建议使用一次跨行词法扫描并显式维护状态；更稳妥的是按语言复用 `tree-sitter/tokenizer`，而不是继续扩充一个同时处理 C、C++、Java、Rust、JavaScript 等语言的简化字符扫描器。

## ACTUAL-006 (P2) — spec producer 退出与 watcher 最后扫描之间存在竞态

- 对应报告：INCR-MISMATCH-073 / `streaming_reasoner`。
- 代码位置：扫描在 `src/verification.py:108-132`，提前退出在 `src/verification.py:205-226`。
- 人工结论：中高置信度实际并发 Bug，但 `validator` 的 `mock` 只是确定性模拟；真实窗口很窄。

每轮先扫描文件并调用 `is_file_ready()`，随后才检查所有 `spec_procs` 是否退出。如果 `producer` 恰好在某文件本轮 `readiness` 检查返回 `False` 后写完最后的 `[SPEC]/[INFO]` 标记并退出，那么同一轮的 207-226 行会看到“全部进程已结束、没有 `future`、仍有 `unprocessed` 文件”并直接 `break`。代码没有在退出前对 `unready` 再做一次 `readiness` 扫描。

因此 `probe` 中“第一次 `False`、随后 `True`”并非天然证明 Bug，但它对应一个真实可发生的进程时序，而不是不合法参数。结果是日志把已经 `ready` 的文件报告成“missing specs/pending”，当前 `invocation` 不再验证它；只有外层重试或下次 `--resume` 才可能补上。

建议 `producer` 全部退出后先进行一次最终稳定扫描；只有确认剩余文件仍不 `ready` 才退出。若 `producer` 采用非原子原地写入，还可要求文件 `mtime/size` 连续两个轮询周期稳定后再判断，避免读取半成品。

## ACTUAL-007 (P2) — Erlang `_SourceIndex` 把非换行控制字符当成 LSP 新行

- 对应报告：INCR-MISMATCH-045 / `_SourceIndex.build`。
- 代码位置：`src/languages/erlang.py:320-355`，ELP `range` 的实际消费者在 `src/languages/erlang.py:505`。
- 人工结论：中等置信度实际 Bug；语义明确，但真实 Erlang 源码中出现原始 VT/FF 等字符的概率较低。

Python `str.splitlines()` 不只识别 `\n\r\n`，还会把垂直制表符 `\v`、换页符 `\f`、NEL 及若干 Unicode 分隔符当作行界。ELP/LSP 返回的 `line/UTF-16 character` 坐标却是编辑器文档行坐标。源码字符串或注释中若含原始 VT，`_SourceIndex.build()` 会多造一行，后续 `position_to_offset()` 从该点开始全部偏移，最终 `_source_for_range()` 截出错误函数体。

这个问题不像其他 Erlang mismatch 那样依赖人为破坏内部状态：`_SourceIndex` 的输入就是从磁盘读取的源码，控制字符仍是合法字符串内容。只是触发样本罕见，所以优先级低于 `phase` 路径、增量完整性和 `codegraph` 数据丢失。

建议按 LSP 文档实际支持的行结束符显式切分，并保留精确偏移；不要直接使用 Python 的“通用文本行”定义。

## 为什么其余 `confirmed` 暂未列为实际 Bug

其余多数 mismatch 至少存在一种明显的假阳性来源，例如：给 Pydantic `str` 字段塞入 `bytes`/整数、把 `None` 传给真实调用链始终提供路径的内部函数、把常量 `monkeypatch` 成 0、要求 `defaultdict` 不具备标准的缺键插入语义、规定所有后端文件必须出现在 `argv` 而忽略文件清单实际通过 `stdin` 传递，或继续使用旧 SPEC 认定一对 `[SPEC]/[INFO]` 就应当 `ready`。

这些条目不是断言“绝对无 Bug”，而是当前证据只证明实现不满足生成 SPEC，没有形成足以指导修复的真实产品契约。优先修复上面的 7 项后，再针对剩余候选补充真实 `caller`、合法配置模型和独立 `oracle`，会比按 `confirmed` 数量逐项修改更可靠。